Deep Learning

Artificial Intelligence

Jacopo Parretti

II Semester 2025-2026

Contents

Part I

Introduction to Deep Learning

1 What is Deep Learning?

1.1 Positioning within AI

Deep Learning is a subset of machine learning and, more specifically, of **representation learning** (also called *feature learning*). The fundamental goal is to *learn underlying features directly from data*, eliminating the need for hand-engineered feature extractors.

The standard machine learning pipeline separates two concerns: **feature extraction**, designed by a domain expert, and **classification**, learned from labeled examples. Deep learning collapses this pipeline — the network performs both steps jointly, learning task-relevant representations end-to-end from raw input.

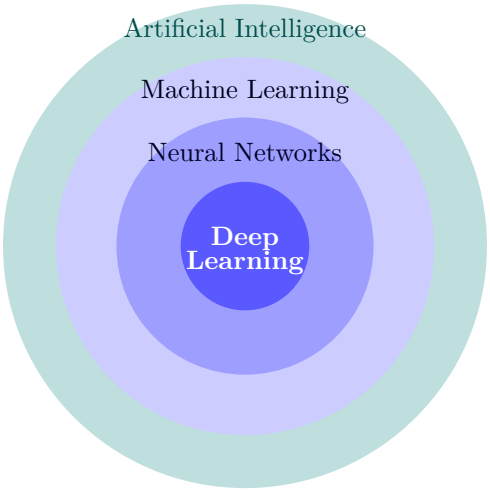

Figure 1: Deep Learning as a nested subset of AI.

1.2 Why Deep Learning?

A critical empirical observation motivates deep learning: as the volume of available data grows, most classical learning algorithms **plateau** in performance, while deep neural networks **continue to improve**. This favorable scaling behavior, combined with large-scale datasets and modern hardware accelerators, explains the field's current dominance.

1.3 Deep Learning as Representation Learning

Deep learning is explicitly a form of **representation learning**: the network learns a hierarchy of representations, layer by layer, from raw input to task-relevant abstractions. In visual recognition, the hierarchy proceeds roughly as:

- **Visible layer**: raw pixel values
- **1st hidden layer**: low-level features (edges, color blobs)
- **2nd hidden layer**: mid-level features (corners, contours)
- **3rd hidden layer**: high-level features (object parts)

- **Output layer**: class identity (car, person, animal, ...)

The deeper the network, the more abstract and task-specific the learned representations.

1.4 Why Representation Matters

The choice of representation fundamentally affects the difficulty of learning. A canonical example: two point clouds that cannot be separated by a linear boundary in Cartesian coordinates (x, y) become trivially linearly separable after converting to polar coordinates (r, θ). Deep networks learn such transformations automatically, avoiding the need for manual feature engineering.

2 A Brief History of Neural Networks

The current success of deep learning is recent, but its intellectual roots extend to the 1940s. Three factors explain the modern renaissance:

1. **Big data**: availability of massive labeled datasets (e.g., ImageNet, web-scale corpora).
2. **Improved software**: modern frameworks (PyTorch, TensorFlow) with automatic differentiation and GPU support.
3. **Hardware**: GPUs and TPUs provide the parallelism required to train large models in tractable time.

Key milestones (1943–present):

- **1943**: McCulloch & Pitts model a neuron with electrical circuits.
- **1949**: Hebb formalizes synaptic plasticity in *The Organization of Behavior*.
- **1956**: The Dartmouth Summer Research Project launches AI as a field.
- **1957–58**: Rosenblatt develops the **Perceptron** — the oldest neural network still in common use today.
- **1959**: Widrow & Hoff develop ADALINE and MADALINE — the first neural network applied to a real-world problem.
- **1969**: Minsky & Papert prove the Perceptron's fundamental limitations (*Perceptrons*); neural network research stalls.
- **1981–82**: Hopfield presents energy-based associative memory; interest revives.
- **1985**: Backpropagation popularized (Rumelhart, Hinton, Williams).
- **1997**: LSTM proposed by Schmidhuber & Hochreiter.
- **1998**: LeCun publishes *Gradient-Based Learning Applied to Document Recognition*.
- **Now**: neural networks are ubiquitous; discussions about their future are prevalent.

3 The Perceptron

The **perceptron** is the basic building block of neural networks, introduced by Frank Rosenblatt in 1958. It is a computational abstraction of a biological neuron.

Definition 3.1 (Perceptron)**.** A perceptron takes m real-valued inputs $x_1, \ldots, x_m$ together with a constant bias w_0, computes their weighted sum, and passes the result through an activation function g:

$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i\, w_i\right)$$

In the original formulation, g is the Heaviside step function. Information flows strictly **forward**: from inputs to output, with no feedback connections.

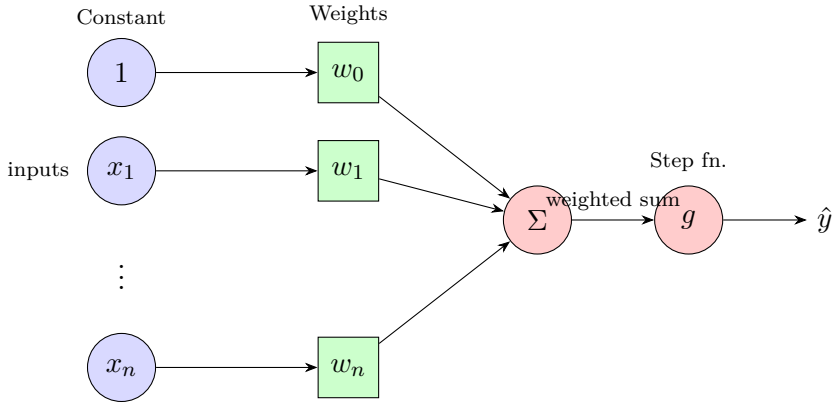

Figure 2: Perceptron: inputs are multiplied by learned weights, summed, and passed through an activation function.

4 Activation Functions

Activation functions introduce **non-linearity** into the network. Without them, stacking linear layers collapses to a single linear transformation, making the model incapable of representing non-linear decision boundaries. They are a crucial element of every architecture.

- **Sigmoid**: $\sigma(x) = \dfrac{1}{1+e^{-x}}$
  Output in $(0, 1)$; historically widespread but suffers from vanishing gradients at saturation.
- **Tanh**: $\tanh(x)$
  Zero-centered; output in $(-1, 1)$; generally preferred over sigmoid in hidden layers.
- **ReLU** (Rectified Linear Unit): $\max(0, x)$
  Sparse activations, computationally cheap; the default choice in modern architectures.
- **Leaky ReLU**: $\max(0.1x,\, x)$
  Mitigates the dying ReLU problem by allowing a small gradient for negative inputs.
- **ELU** (Exponential Linear Unit):

  $$\mathrm{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

  Smooth for negative inputs; can push mean activations toward zero.
- **Maxout**: $\max(w_1^T x + b_1,\; w_2^T x + b_2)$
  Generalizes ReLU and Leaky ReLU; learns the activation shape at the cost of doubling the parameter count.

5 From Perceptron to Neural Networks

5.1 The Multi-Layer Idea

The goal is to build a **larger, non-linear model**. The key idea: concatenate many perceptron systems and insert a **non-linear activation function** between computation layers. A two-layer network (one hidden layer) computes:

$$\mathbf{y}_1 = \mathbf{x}W_1 + \mathbf{b}_1, \qquad \mathbf{y}_2 = \phi(\mathbf{y}_1)\, W_2 + \mathbf{b}_2, \qquad \hat{\mathbf{y}} = s(\mathbf{y}_2)$$

where ϕ is a non-linear activation (e.g., ReLU) and s is the output activation (e.g., softmax for classification).

5.2 Dimensions

For a problem with F input features, H hidden neurons, and C output classes:

- $\mathbf{x} \in \mathbb{R}^{1\times F}$, $W_1 \in \mathbb{R}^{F\times H}$, $\mathbf{b}_1 \in \mathbb{R}^{1\times H}$, $\mathbf{y}_1 \in \mathbb{R}^{1\times H}$
- $W_2 \in \mathbb{R}^{H\times C}$, $\mathbf{b}_2 \in \mathbb{R}^{1\times C}$, $\mathbf{y}_2 \in \mathbb{R}^{1\times C}$

Example: a 28×28 grayscale image ($F = 784$), $H = 1024$ hidden neurons, $C = 3$ classes:

$$\underbrace{\mathbf{x}}_{1\times 784} \cdot \underbrace{W_1}_{784\times 1024} + \underbrace{\mathbf{b}_1}_{1\times 1024} = \underbrace{\mathbf{y}_1}_{1\times 1024} \xrightarrow{\text{ReLU}} \underbrace{W_2}_{1024\times 3} + \underbrace{\mathbf{b}_2}_{1\times 3} = \underbrace{\mathbf{y}_2}_{1\times 3} \rightarrow s(\mathbf{y}_2)$$

5.3 Multilayer Perceptron (MLP)

The **Multilayer Perceptron** (MLP) extends this to an arbitrary number of stacked hidden layers. For input $x = (\chi_1, \ldots, \chi_p)^T$:

1. **Hidden layer** with weight matrix W^h and activation σ:

$$u_j = \sum_i W^h_{ji}\, \chi_i, \qquad h_j = \sigma(u_j)$$

2. **Output layer** with weight matrix W^y:

$$v_k = \sum_j W^y_{kj}\, h_j, \qquad y_k = \sigma(v_k)$$

The network is **fully connected**: every neuron in layer ℓ connects to every neuron in layer $\ell+1$.

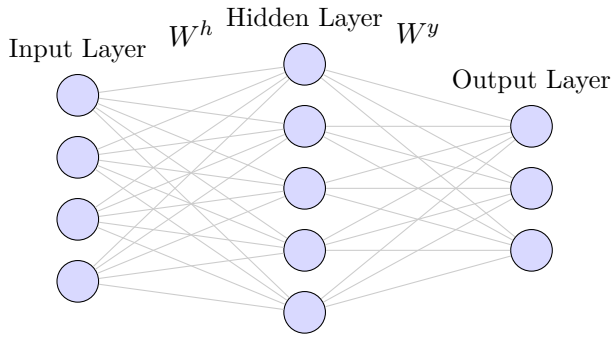

Figure 3: Multilayer Perceptron with one hidden layer.

6 Training Neural Networks

6.1 Training and Testing Stages

Neural network training proceeds in two distinct phases:

- **Training stage**: input data is passed through the network and compared to the ground truth. The network parameters are adjusted to minimize the discrepancy.
- **Testing stage**: new, unseen input is passed through the *trained* (frozen) network to produce a prediction (best guess).

6.2 Forward and Backward Pass

Each training step consists of two passes:

1. **Forward pass**: propagate the input through the network to compute a prediction.
2. **Backward pass (backpropagation)**: compute the loss between prediction and ground truth; propagate the error signal backward through the network via the chain rule; update weights to reduce the error.

7 Loss Functions

A loss function quantifies the discrepancy between the network's prediction p and the ground truth t. The choice is **task-dependent**.

7.1 Standard Loss Functions

1. **Regression** — Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum(y - \hat{y})^2$$

2. **Binary classification** — Binary Cross-Entropy (BCE):

$$\text{BCE}(t, p) = -\big(t \cdot \log(p) + (1 - t) \cdot \log(1 - p)\big)$$

3. **Multiclass classification** — Categorical Cross-Entropy (CCE):

$$\text{CCE}(p, t) = -\sum_{c=1}^{C} t_{o,c} \log(p_{o,c})$$

7.2 Cross-Entropy and Information Theory

Cross-entropy loss derives from the notion of **entropy** $H(X)$, which measures the uncertainty of a probability distribution:

$$H(X) = \begin{cases} -\displaystyle\int_x p(x) \log p(x)\, dx & \text{if } X \text{ is continuous} \\ -\displaystyle\sum_x p(x) \log p(x) & \text{if } X \text{ is discrete} \end{cases}$$

(Logarithm in base 2.)

- Higher $H(X) \Rightarrow$ greater uncertainty in the distribution.

- A **uniform** $p(x) \Rightarrow$ maximum entropy (maximum uncertainty).
- A **Dirac delta** $p(x) \Rightarrow$ minimum entropy (zero uncertainty).

Minimizing cross-entropy is equivalent to concentrating the predicted distribution on the correct class. Hence: **the lower the loss, the better the model**.

7.3 Loss as an Optimization Objective

Because loss functions are **differentiable**, the learning problem can be cast as an **optimization problem**.

Key Insight

By training the network, we want to find the weights that achieve **minimal loss**. The weight update algorithm is called **Gradient Descent**, implemented in practice via the **backpropagation procedure**.

8 Backpropagation and Gradient Descent

8.1 General Idea

Backpropagation iteratively updates **weights** w and **biases** b to decrease the loss $J(w)$. The procedure decomposes into three sub-tasks:

1. **Forward pass**: compute the network output and evaluate the loss.
2. **Backward pass**: compute gradients $\partial J/\partial w$ by applying the chain rule layer by layer (from output to input).
3. **Weight update**: subtract a fraction of the gradient from each weight. The fraction is controlled by the **learning rate** η — a critical hyperparameter.

The update corresponds geometrically to moving in the direction of **steepest descent** on the loss surface.

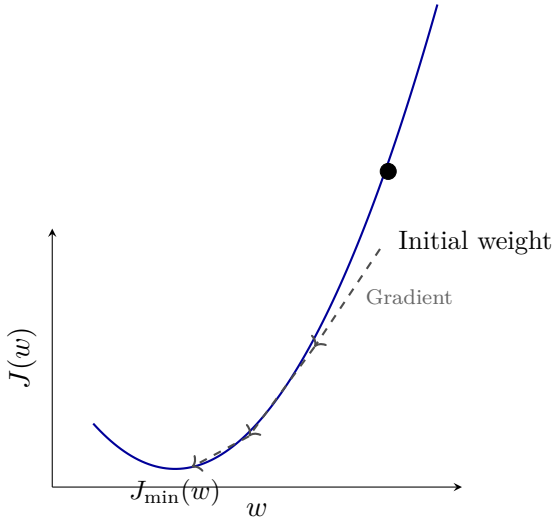

Figure 4: Gradient descent on a 1D loss surface. Each step moves in the direction of the negative gradient toward the global minimum.

8.2 Batch Gradient Descent

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
   (a) Compute the gradient $\frac{\partial J(w)}{\partial w}$ over the *entire* training set.
   (b) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w.

Drawback: computing the gradient over all training samples is very expensive when the dataset contains millions of examples.

8.3 Stochastic Gradient Descent (SGD)

SGD reduces the cost by estimating the gradient from a *single* training sample i at each step:

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
   (a) Pick a single data point i.
   (b) Compute the gradient $\frac{\partial J_i(w)}{\partial w}$.
   (c) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w.

Drawback: the gradient estimate is very noisy since it relies on a single instance.

8.4 Mini-Batch Gradient Descent

Mini-batch gradient descent is the standard approach in practice — the middle ground between batch GD and SGD:

1. Initialize weights randomly: $w \sim \mathcal{N}(0, \sigma^2)$.
2. Repeat until convergence:
   (a) Pick a batch B of data points.
   (b) Compute the averaged gradient:

$$\frac{\partial J_B(w)}{\partial w} = \frac{1}{B} \sum_{k=1}^{B} \frac{\partial J_k(w)}{\partial w}$$

   (c) Update: $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$.
3. Return w.

Mini-batch gradient descent yields **smoother convergence**, **more stable gradients**, and allows for **larger learning rates** compared to pure SGD — making it the default optimization strategy in deep learning.

Part II

Backpropagation: Derivation and Training Dynamics

9 Expressive Capacity of Multilayer Networks

9.1 What a Single Perceptron Can Represent

A perceptron with n inputs defines a hyperplane of dimension $n-1$ that partitions the input space into two half-spaces:

$$\sum_{i=1}^{n} w_i x_i - t = 0$$

This limits its classification ability to **linearly separable** problems. The canonical failure case is XOR — no values w_0, w_1, w_2 can satisfy the four constraints simultaneously.

9.2 Capacity with Hidden Layers

Adding hidden layers dramatically increases expressive power. In the two-dimensional case:

- Each hidden neuron defines a boundary line (half-plane).
- With h hidden neurons, the output neuron takes their conjunction: three neurons yield a triangle, four a quadrilateral, and so on.

The following theorems quantify this:

Theorem 9.1 (Huang & Lipmann, 1988)**.** *A feed-forward network with a single hidden layer can approximate almost all separation surfaces between two classes. The qualifier "almost all" means that by adding a second output neuron, non-convex and disconnected regions become representable too.*

Theorem 9.2 (Gibson & Lowan, 1990)**.** *There exist regions that a single-hidden-layer network cannot approximate — for example, two triangular regions that touch at exactly one point.*

Theorem 9.3 (Lipmann, 1987)**.** *A feed-forward network with **two** hidden layers can approximate any region in input space. Approximation accuracy is controlled by the number of neurons in the intermediate layers.*

Remark. These results are existence theorems, not constructive recipes. In practice, there are no fixed rules for choosing architecture; empirical heuristics and cross-validation guide the design.

10 Formal Derivation of Backpropagation

10.1 Setup and Notation

Consider a two-layer feed-forward network with n inputs, h hidden neurons, and q output neurons. Let g be a differentiable activation function (e.g. the logistic). Define:

I_i	i-th input
w_{ji}	weight from input i to hidden neuron j
$a_j = \sum_{i=1}^{n} w_{ji}\, I_i$	pre-activation of hidden neuron j
$Z_j = g(a_j)$	output of hidden neuron j
W_{kj}	weight from hidden neuron j to output neuron k
$a_k = \sum_{j=1}^{h} W_{kj}\, Z_j$	pre-activation of output neuron k
$O_k = g(a_k)$	output of network for class k

The squared error for a single training example with target y_k:

$$E(W) = \frac{1}{2} \sum_{k=1}^{q} \left(y_k - O_k\right)^2$$

10.2 Derivatives for the Output Layer

We want $\partial E / \partial W_{kj}$. By the chain rule:

$$\frac{\partial E}{\partial W_{kj}} = \frac{\partial E}{\partial O_k} \cdot \frac{\partial O_k}{\partial a_k} \cdot \frac{\partial a_k}{\partial W_{kj}}$$

Computing each factor:

$$\frac{\partial E}{\partial O_k} = -(y_k - O_k), \qquad \frac{\partial O_k}{\partial a_k} = g'(a_k), \qquad \frac{\partial a_k}{\partial W_{kj}} = Z_j$$

Define the **output delta**:

$$\delta_k := (O_k - y_k)\, g'(a_k)$$

Then:

$$\boxed{\frac{\partial E}{\partial W_{kj}} = \delta_k\, Z_j}$$

The weight update (gradient descent) for the output layer is:

$$\Delta W_{kj} = -\eta\, \delta_k\, Z_j$$

10.3 Derivatives for the Hidden Layer

We want $\partial E / \partial w_{ji}$. By the chain rule:

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial a_j} \cdot \frac{\partial a_j}{\partial w_{ji}}$$

The second factor is simply $\partial a_j / \partial w_{ji} = I_i$. For the first factor we must account for the fact that a_j feeds all q output neurons through Z_j:

$$\frac{\partial E}{\partial a_j} = \sum_{k=1}^{q} \frac{\partial E}{\partial a_k} \cdot \frac{\partial a_k}{\partial Z_j} \cdot \frac{\partial Z_j}{\partial a_j} = \sum_{k=1}^{q} \delta_k\, W_{kj} \cdot g'(a_j) = g'(a_j) \sum_{k=1}^{q} \delta_k\, W_{kj}$$

Define the **hidden delta**:

$$\delta_j \coloneqq g'(a_j) \sum_{k=1}^{q} \delta_k \, W_{kj}$$

Then:

$$\boxed{\frac{\partial E}{\partial w_{ji}} = \delta_j \, I_i}$$

The weight update for the hidden layer is:

$$\Delta w_{ji} = -\eta \, \delta_j \, I_i$$

10.4 General Weight Update Rule

Both updates share the same structure. For any weight connecting neuron q (input side) to neuron p (output side):

$$\Delta W_{pq} = -\eta \, \delta_p \cdot V_q$$

where V_q is the activation value at the input neuron and δ_p is the error signal at the output neuron, computed by:

- **Output layer**: $\delta_k = (O_k - y_k) \, g'(a_k)$
- **Hidden layer**: $\delta_j = g'(a_j) \sum_k \delta_k \, W_{kj}$ (weighted sum of upstream deltas)

This recursive structure is what gives the algorithm its name: error signals δ are *propagated backward*, from output to input, layer by layer.

10.5 Algorithm

Algorithm 1 Backpropagation (online, one training example per step)

1: Initialize all weights randomly (small values).
2: **repeat**
3: **Forward pass**: present input $\mathbf{x}$; compute a_j, Z_j, a_k, O_k for all hidden and output neurons; store activations.
4: **Compute output deltas**: $\delta_k \leftarrow (O_k - y_k) \, g'(a_k)$ for all k.
5: **Backpropagate**: $\delta_j \leftarrow g'(a_j) \sum_k \delta_k \, W_{kj}$ for all hidden neurons j.
6: **Update output weights**: $W_{kj} \leftarrow W_{kj} - \eta \, \delta_k \, Z_j$.
7: **Update hidden weights**: $w_{ji} \leftarrow w_{ji} - \eta \, \delta_j \, I_i$.
8: **until** convergence

10.6 Complexity

Computing the gradient via finite differences (difference quotient) costs $\mathcal{O}(W^2)$: for each of the W weights, the network must be evaluated anew. Backpropagation reduces this to $\mathcal{O}(W)$ by reusing intermediate activations and computing all partial derivatives in a single backward sweep. This is the key efficiency gain that makes training large networks tractable.

An additional structural benefit is **locality**: the update ΔW_{pq} depends only on the activation at neuron q and the delta at neuron p — exactly the two endpoints of the connection. This locality makes the algorithm trivially parallelizable.

11 Optimization Challenges

11.1 Learning Rate Sensitivity

The learning rate η is a critical hyperparameter. Its effect is qualitatively different in two failure regimes:

- η **too small**: convergence is slow; training may require an impractical number of epochs.
- η **too large**: the update overshoots the minimum, causing the weights to oscillate between two configurations without converging.

11.2 Local Minima

The loss surface of a deep network is non-convex, with many local minima. Gradient descent is only guaranteed to converge to a *local* minimum; the outcome depends on the initialization. If the starting configuration $W^{(0)}$ is far from the global minimum, the algorithm may settle at a suboptimal solution.

Momentum. A practical remedy is to add a **momentum term** that incorporates inertia from the previous step:

$$\Delta W^{(k)} = -\eta \left.\frac{\partial E}{\partial W}\right|_{W^{(k)}} + \alpha\big(W^{(k)} - W^{(k-1)}\big)$$

where $\alpha \in (0, 1)$ is the momentum parameter (commonly $\alpha = 0.9$). The effect is to dampen oscillations in directions of high curvature and accelerate movement along low-curvature directions. Other global optimization strategies (simulated annealing, genetic algorithms, reactive Tabu search) can also be applied.

11.3 Saturation

When an activation function is in its saturated regime — inputs far from zero for sigmoid or tanh — its derivative g' is nearly zero. Consequently, $\delta \propto g'$ is nearly zero as well, and weight updates become negligibly small. Training effectively stalls.

Mitigation: initialize weights to small random values. Near zero, the activation functions are approximately linear and their derivatives are large, so gradients are well-conditioned at the start of training. As learning proceeds, weights grow only as needed.

12 Generalization

12.1 Definition

A network's **generalization capacity** is its ability to correctly classify inputs *not* seen during training. Minimizing training loss is a means to an end, not the end itself. A model that perfectly memorizes training examples but fails on new ones has not learned anything useful.

12.2 Overfitting

Overfitting (also called overtraining) occurs when the network has been trained for too many epochs: it has memorized the training patterns, including their noise, at the expense of flexibility on unseen inputs. The telltale sign is a divergence between training and test error:

- **Training error**: monotonically decreasing.

- **Test error**: initially decreasing, then increasing after some epoch e_0.

The optimal stopping point is e_0, before the test error begins to rise. This technique is known as **early stopping**.

Key Principle

Minimizing training error is **not** equivalent to training a good neural network. The minimum training error does not correspond to the best generalization. Overfitting must always be monitored.

13 Cross-Validation Strategies

In practice, labeled data is limited. The following strategies manage the training-evaluation tradeoff under this constraint.

13.1 Resubstitution

Train and evaluate on the *same* set (all available data). This produces an **optimistic** (lower-bound) error estimate: the model has already seen every example during training, so measured error is artificially low.

13.2 Holdout

Randomly partition the dataset into two disjoint halves: one for training, one for testing. This provides an **upper-bound** error estimate, since the model sees only half the data during training. The result is sensitive to the particular partition chosen.

13.3 Averaged Holdout

Repeat the holdout procedure multiple times with different random partitions and average the resulting errors. This reduces the variance introduced by a single arbitrary split. Partitions can be sampled randomly or enumerated exhaustively.

13.4 Leave-One-Out (LOO)

Given N examples, train on $N-1$ and test on the single excluded example; repeat for every example and average. This maximally utilizes the available data and provides a near-unbiased error estimate. The computational cost is N training runs, which can be prohibitive for large datasets or slow-converging models.

13.5 k-Fold Cross-Validation

A practical compromise between holdout and LOO. Partition the data into k disjoint segments (folds) of equal size. For each fold:

1. Train on the remaining $k-1$ folds.
2. Evaluate on the held-out fold.

Average the k test errors. Leave-one-out corresponds to $k = N$. A common choice in practice is $k = 5$ or $k = 10$, balancing computational cost against variance.

Method	Training size	Bias	Variance
Resubstitution	N	High (optimistic)	Low
Holdout	$N/2$	Moderate	High
Averaged Holdout	$N/2$	Moderate	Lower
Leave-One-Out	$N-1$	Low	High
k-Fold	$(k-1)N/k$	Low–moderate	Moderate

14 When to Use Neural Networks

Neural networks are an appropriate choice when:

1. **The problem is instance-based**: examples are given as (attributes, class) pairs. Raw inputs should be preprocessed — real-valued features scaled to $[0, 1]$, discrete features one-hot encoded.
2. **The training set is large**: generalization improves monotonically with data; small datasets favor simpler, more data-efficient models.
3. **Long training times are acceptable**: iterative optimization over large parameter spaces is inherently expensive.
4. **Interpretability is not required**: the learned weight matrices are opaque; if human-readable discriminant functions are needed, decision trees or linear models are preferable.

Part III

Convolutional Neural Networks

15 Regularization

Deep networks have high expressive power and will readily memorize training data. **Regularization** constrains the optimization problem to encourage generalization on unseen inputs, rather than overfitting to training noise.

15.1 The Bias-Variance Trade-off

Model quality is governed by two competing sources of error:

- **Bias**: error from overly simplistic assumptions; the model underfits the data.
- **Variance**: error from excessive sensitivity to training data; the model overfits.

As model complexity increases, bias decreases but variance grows. The optimal model lives at the minimum of their sum. Deep networks are high-variance models by default — regularization is what keeps them on the right side of this trade-off.

15.2 Early Stopping

Stop training as soon as the validation loss stops improving. In practice:

- **Training loss**: monotonically decreasing.
- **Validation loss**: decreases early, then rises as overfitting begins.

The checkpoint at the minimum validation loss is retained as the final model. This is the simplest and most widely used regularizer.

15.3 L2 Regularization (Ridge / Weight Decay)

Add an L2 penalty on all weights to the loss:

$$\mathcal{L}' = \mathcal{L} + \frac{\beta}{2}\|W\|_2^2 = \mathcal{L} + \frac{\beta}{2}\sum_i w_i^2$$

where $\beta > 0$ is the penalty factor. The added term penalizes large weights, forcing the network to distribute the learned signal evenly and discouraging any single connection from dominating. In the gradient update this appears as a constant multiplicative decay on weights at each step, hence the name *weight decay*.

15.4 Dropout

Dropout prevents neurons from *co-adapting* — relying on the presence of specific other neurons and thus becoming brittle. During each training step, each neuron is independently retained with probability p (typically $p \geq 0.5$ for later layers; higher for early layers and input) and dropped (set to zero) with probability $1 - p$.

The effect: each training step is equivalent to training a different sub-network. At test time all neurons are active but their outputs are scaled by p to match the expected activation. This ensemble-like behaviour provides strong regularization at negligible cost.

15.5 Batch Normalization

As training progresses, the distribution of inputs to each hidden layer shifts as the parameters of preceding layers change — a phenomenon called **internal covariate shift**. Batch normalization (Ioffe & Szegedy, ICML 2015) mitigates this by normalizing each feature dimension across the current mini-batch:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

To preserve representational power, two learnable parameters $\gamma^{(k)}$ and $\beta^{(k)}$ are introduced per feature:

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}$$

Batch normalization is inserted between the linear transformation and the activation function. It accelerates training, acts as a mild regularizer, and is practically indispensable in very deep architectures.

16 Convolutional Neural Networks

16.1 Motivation: Receptive Fields and Weight Sharing

The mammalian visual cortex is organized so that individual neurons respond only to stimuli within a restricted region of the visual field — their **receptive field**. Nearby neurons have overlapping receptive fields that tile the entire visual scene. **Convolutional Neural Networks** (CNNs) are loosely inspired by this architecture.

The fundamental observation is that the same local pattern detector (e.g., an edge detector) is useful everywhere in an image — there is no reason to learn a separate detector for each spatial location. CNNs exploit this by **sharing weights** across space: a single filter is convolved with the entire input, producing a **feature map** that encodes where the detected pattern appears.

16.2 The Convolution Operation

A filter (kernel) K of size $k \times k$ is slid over the input with a chosen **stride** s. At each position, the element-wise product of the filter and the corresponding input patch is summed to produce a single output value. For a 2D input X and filter K:

$$(X * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i \cdot s + m,\ j \cdot s + n] \cdot K[m, n]$$

Example 16.1. A 5×5 input convolved with a 3×3 filter at stride 1 produces a 3×3 feature map. The filter is positioned at each valid location, element-wise multiplied with the input patch, and summed:

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix} * \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{pmatrix}$$

Output spatial size. For an input of height/width H, filter size k, stride s, and zero-padding p:

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

16.3 Feature Maps and Filters

A convolutional layer applies d different filters to the same input, producing d feature maps stacked along the **depth** dimension (not to be confused with the depth of the network). Each filter learns to detect a distinct local pattern.

- **Shallow filters** (early layers): detect simple patterns such as oriented edges and color gradients.
- **Deeper filters**: combine the activations of previous layers to detect increasingly abstract patterns — textures, object parts, complete objects.

A layer therefore maps an input volume of shape $H \times W \times d_{\text{in}}$ to an output volume of shape $H' \times W' \times d_{\text{out}}$, where d_{out} equals the number of filters.

16.4 Non-linearity

After each convolution, a non-linear activation is applied element-wise. In modern CNNs this is almost universally **ReLU**:

$$g(z) = \max(0, z)$$

ReLU is applied independently to each element of the feature map, setting all negative values to zero. Without non-linearity, stacked convolutions would collapse to a single linear filter.

16.5 Pooling

Pooling layers reduce spatial dimensions, thereby:

- decreasing the number of parameters and computation,
- introducing limited spatial invariance (small translations of the input produce the same output),

- controlling overfitting.

The most common variant is **max pooling**: a $k \times k$ window slides over the feature map with stride s and outputs the maximum value in each window. A 2×2 max pool with stride 2 is the standard, halving both spatial dimensions.

16.6 Fully Connected Layer and Softmax

After a sequence of convolution and pooling operations, the spatial dimensions are small enough that the feature map is **flattened** into a vector and passed through one or more fully connected layers. The final FC layer has as many neurons as there are classes.

For multiclass classification, the output is passed through **softmax**:

$$P(y = j \mid \mathbf{x}) = \frac{e^{z_j}}{\sum_{k=1}^{C} e^{z_k}}$$

which converts raw scores $\mathbf{z}$ into a proper probability distribution over C classes. Combined with categorical cross-entropy loss, softmax is the standard output for classification CNNs.

16.7 Standard Architecture Pattern

The canonical CNN architecture stacks convolutional blocks followed by fully connected layers:

$$\text{INPUT} \to [[\text{CONV} \to \text{ReLU}]^N \to \text{POOL}]^M \to [\text{FC} \to \text{ReLU}]^K \to \text{FC (softmax)}$$

Typical choices: $N \leq 3$, $M \geq 1$, $K \leq 2$. The convolutional blocks learn spatial features; the FC layers perform classification.

Summary: CNN Layers

Layer	Operation	Purpose
CONV	Sliding dot product with shared filters	Local feature extraction
ReLU	$\max(0, x)$ elementwise	Non-linearity
POOL	Max/avg over spatial window	Downsampling, invariance
BN	Normalize by batch statistics	Training stability
FC	Matrix multiply + bias	Classification / regression
Softmax	Normalized exponential	Probability over classes

17 Landmark Architectures

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) catalyzed a decade of architectural innovation. The following architectures represent the major milestones, each introducing a design principle that influenced the field.

17.1 LeNet (1998)

Developed by LeCun et al. for handwritten digit recognition on MNIST (28×28 grayscale, 10 classes, 60k training examples). LeNet established the canonical pattern of interleaved convolution and subsampling:

$$\text{INPUT } (32 \times 32) \;\to\; C_1 \text{ (6@28} \times \text{28)} \;\to\; S_2 \text{ (6@14} \times \text{14)} \;\to\; C_3 \text{ (16@10} \times \text{10)}$$
$$\to\; S_4 \text{ (16@5} \times \text{5)} \;\to\; F_5 \text{ (120)} \;\to\; F_6 \text{ (84)} \;\to\; \text{OUTPUT (10)}$$

Filters were 5×5 at stride 1; subsampling was average pooling 2×2 at stride 2; activation functions were sigmoid/tanh.

17.2 AlexNet (2012)

AlexNet (Krizhevsky et al.) won ILSVRC 2012 with a top-5 error of 15.3%, compared to 26% for the runner-up. It was the first deep CNN applied at scale to ImageNet:

- 5 convolutional layers + 3 fully connected layers; 60 million parameters.
- **First use of ReLU** in a competitive CNN — replacing sigmoid/tanh and substantially accelerating training.
- **Dropout** (rate 0.5) on FC layers to control overfitting.
- Data augmentation (random crops, horizontal flips, color jitter).
- Trained on two GTX 580 GPUs for one week.
- Hyperparameters: SGD with momentum 0.9, learning rate 10^{-2} (manually decayed), L2 weight decay 5×10^{-4}, mini-batch 128.

Architecture (for input $227 \times 227 \times 3$):

1. CONV_1: 96 filters, 11×11, stride $4 \to 55 \times 55 \times 96$; MAX POOL 3×3 s2 $\to 27 \times 27 \times 96$
2. CONV_2: 256 filters, 5×5, pad $2 \to 27 \times 27 \times 256$; MAX POOL $\to 13 \times 13 \times 256$
3. CONV_3: 384 filters, 3×3, pad $1 \to 13 \times 13 \times 384$
4. CONV_4: 384 filters, 3×3, pad $1 \to 13 \times 13 \times 384$
5. CONV_5: 256 filters, 3×3, pad $1 \to 13 \times 13 \times 256$; MAX POOL $\to 6 \times 6 \times 256$
6. FC6: 4096 neurons; FC7: 4096 neurons; FC8: 1000 neurons (class scores)

17.3 ZFNet (2013)

Won ILSVRC 2013. Zeiler & Fergus improved AlexNet by:

- reducing the first convolution from 11×11 stride 4 to 7×7 stride 2 (finer spatial resolution early on),
- expanding the number of feature maps in the middle convolutional layers.

ZFNet also introduced **deconvolution-based visualization** to understand which input patterns activate each filter, providing the first systematic tool for inspecting what CNNs learn.

17.4 VGGNet (2014)

Runner-up at ILSVRC 2014 (Simonyan & Zisserman). VGGNet's central thesis: **depth alone, with small filters, drives performance**. Design rules:

- All convolutions are 3×3 at stride 1, padding 1.
- All pooling is 2×2 max pooling at stride 2.

- 16 weight layers (VGG-16) or 19 (VGG-19); 138 million parameters.

Two stacked 3×3 convolutions have the same receptive field as one 5×5, but fewer parameters and an extra non-linearity. Three 3×3 convolutions match a 7×7 receptive field with a further reduction in parameters. This insight — *replace large filters with stacks of small ones* — became a standard architectural principle.

Limitation: most memory is consumed by early convolutional layers; most parameters reside in the late FC layers (two FC layers of 4096 units each account for the majority of the 138M parameters).

17.5 GoogLeNet / Inception (2014)

Won ILSVRC 2014 (Szegedy et al., Google). The central innovation is the **Inception module**: rather than choosing a single filter size, apply 1×1, 3×3, and 5×5 convolutions and 3×3 max pooling *in parallel* on the same input, then concatenate their outputs along the depth dimension.

- 1×1 **convolutions** (bottleneck projections) reduce channel depth before the expensive 3×3 and 5×5 convolutions, dramatically cutting computation.
- 22 layers total; only 5 million parameters (vs. AlexNet's 60M) — **12× fewer**.
- No large FC layers; replaced by **global average pooling**, eliminating the parameter bottleneck.
- Two **auxiliary classifiers** attached at intermediate layers combat vanishing gradients during training and provide additional regularization.

Later versions (v2, v3, v4) refined the inception blocks with further factorization (e.g., replacing 5×5 with two 3×3, or $n \times 1$ followed by $1 \times n$) and incorporated batch normalization.

17.6 ResNet (2015)

Won ILSVRC 2015 with 3.57% top-5 error — surpassing human-level performance (5.1%). Developed by He et al. (Microsoft Research). ResNet's contribution: **residual (skip) connections** that make very deep networks trainable.

The residual block. Instead of learning the mapping $H(\mathbf{x})$ directly, the block learns the **residual** $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$, and the skip connection adds the input back:

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

If the optimal transformation is close to the identity, it is easier for the network to push $F(\mathbf{x}) \to 0$ than to learn a near-identity from scratch. Skip connections also provide a direct gradient path to early layers, alleviating vanishing gradients.

Bottleneck block. For deeper networks, each residual block uses a 1×1-3×3-1×1 sequence: the first 1×1 reduces channel depth (e.g. $256 \to 64$), the 3×3 performs spatial convolution, and the second 1×1 restores depth ($64 \to 256$). This reduces computation by roughly $10\times$ compared to two 3×3 convolutions with the same channel count.

Training details. Batch normalization after every CONV; He (Xavier/2) initialization; SGD with momentum 0.9; initial LR 0.1, divided by 10 at plateaus; weight decay 10^{-5}; no dropout.

Scale. The ILSVRC-winning model used 152 layers yet trained faster than VGGNet-16 due to efficient bottleneck blocks and the absence of large FC layers.

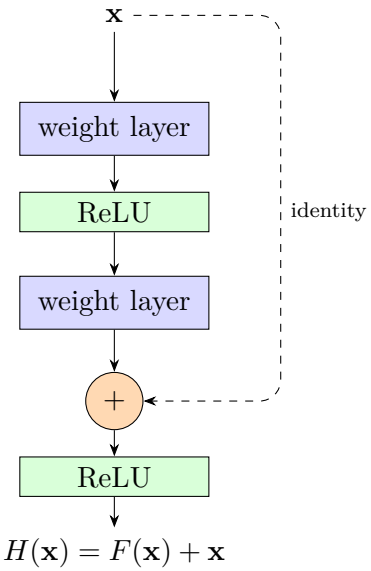

Figure 5: Residual block. The skip connection adds the block's input directly to its output, making it easy for the block to represent the identity if that is the optimal mapping.

17.7 Performance Summary

Architecture	Year	Layers	Parameters	Top-5 error
AlexNet	2012	8	61M	15.3%
ZFNet	2013	8	62M	11.2%
VGGNet-16	2014	16	138M	7.32%
GoogLeNet	2014	22	5M	6.67%
ResNet-152	2015	152	25M	3.57%
Human expert	—	—	—	5.1%

The trend is clear: more layers consistently improve performance, provided the optimization problem is properly controlled (via residual connections, batch normalization, and regularization). GoogLeNet demonstrates that parameter count and accuracy are not coupled — architectural efficiency matters as much as depth.

18 Transfer Learning

Training a deep CNN from scratch on a new task typically requires millions of labeled examples and weeks of GPU time. **Transfer learning** bypasses this by reusing a network pretrained on a large source dataset (typically ImageNet) as a starting point.

18.1 ConvNet as Fixed Feature Extractor

Remove the last fully connected layer (which encodes source-task class scores) and treat the remaining network as a **fixed feature extractor**. For each input in the new dataset, propagate it forward and extract the activation vector from the penultimate layer (e.g. a 4096-dimensional vector from FC7 in AlexNet). Train a lightweight classifier (linear SVM or logistic regression) on these features. No backpropagation into the pretrained weights is performed.

When to use: new dataset is small and similar to the source dataset. Fine-tuning is risky due to overfitting; the high-level features from the pretrained network are already relevant.

18.2 Fine-tuning

Initialize with pretrained weights, then continue training by backpropagating on the new labeled data. Three common variants:

1. **Fine-tune only the classifier** (last FC layer): equivalent to feature extractor above but trained end-to-end with a very small learning rate. Used when the new dataset is small and similar to the source.
2. **Fine-tune the top few layers**: unfreeze the last 1–3 convolutional blocks in addition to the FC layers. Used when the new dataset is moderately large or somewhat different from the source.
3. **Fine-tune the entire network**: unfreeze all layers and train with a small learning rate. Used when the new dataset is large (low overfitting risk) and substantially different from the source, so source features are not well aligned.

Practical Rule of Thumb

Dataset size	Similarity to source	Strategy
Small	Similar	Linear classifier on CNN codes
Large	Similar	Fine-tune full network
Small	Different	Linear classifier on intermediate features
Large	Different	Fine-tune full network (or train from scratch)

Early layers learn generic features (edges, textures) that transfer broadly; later layers encode increasingly task-specific representations. This is why freezing early layers is almost always safe, regardless of the transfer scenario.

19 Beyond Supervised Classification

19.1 Autoencoders

An **autoencoder** is trained to reconstruct its input:

$$\mathbf{x} \xrightarrow{\text{encoder}} \mathbf{h} \xrightarrow{\text{decoder}} \hat{\mathbf{x}} \approx \mathbf{x}$$

The bottleneck representation $\mathbf{h}$ (latent code) is forced to capture the most informative structure in the data. Autoencoders enable unsupervised feature learning, dimensionality reduction, and anomaly detection. In practice, discriminative tasks tend to yield better embeddings than pure reconstruction.

19.2 Generative Adversarial Networks

A **GAN** (Goodfellow et al., NeurIPS 2014) trains two networks in adversarial competition:

- **Generator** G: maps random noise $\mathbf{z} \sim p(\mathbf{z})$ to synthetic samples $G(\mathbf{z})$.
- **Discriminator** D: classifies inputs as real (from data) or fake (from G).

G tries to fool D; D tries to distinguish real from generated samples. At equilibrium, $G(\mathbf{z})$ is indistinguishable from real data. The minimax objective is:

$$\min_G \max_D \; \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

GANs have been applied to image synthesis, style transfer, image-to-image translation (pix2pix, CycleGAN), and data augmentation.

19.3 Recurrent Networks and LSTMs

Recurrent Neural Networks (RNNs) process sequential data by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step:

$$\mathbf{h}_t = f(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

Vanilla RNNs struggle with long-range dependencies due to vanishing gradients over many time steps.

Long Short-Term Memory (LSTM) networks address this with a gated cell state $\mathbf{c}_t$ that flows through the sequence with only additive interactions — a *conveyor belt* for information. Three learned gates control the information flow:

- **Forget gate**: decides which portions of $\mathbf{c}_{t-1}$ to discard.
- **Input gate**: decides what new information to add to $\mathbf{c}_t$.
- **Output gate**: decides what to expose as the hidden state $\mathbf{h}_t$.

The gating mechanism allows LSTMs to maintain relevant information over arbitrarily long sequences, making them the standard for text, speech, and time-series tasks before the advent of transformers.

19.4 Transformers

Transformers (Vaswani et al., 2017) replace recurrence with **self-attention**, allowing every position in a sequence to attend directly to every other position:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where Q (queries), K (keys), and V (values) are linear projections of the input. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing large and saturating the softmax.

Multi-head attention runs h attention mechanisms in parallel and concatenates their outputs, enabling the model to attend to different aspects of the input simultaneously.

Key architectural components:

- **Positional encoding**: injects position information (since self-attention is permutation-invariant).
- **Feed-forward sublayer**: applied position-wise after each attention block.
- **Add & Norm**: residual connections and layer normalization after each sublayer.

Transformers are now the dominant architecture for NLP (BERT, GPT) and are increasingly applied to computer vision (Vision Transformer, ViT).

Part IV

Image Classification

20 The Image Classification Problem

20.1 Task Definition and Applications

Image classification is the task of assigning a discrete semantic label $y \in \{1, \ldots, C\}$ to an input image $\mathbf{x}$. The label is drawn from a fixed, finite vocabulary of C categories. Applications span a broad range:

- **Medical imaging**: distinguishing benign from malignant tissue (Levy et al., 2016), cell type identification, pathology grading.
- **Scientific discovery**: galaxy morphology classification (Dieleman et al., 2014), satellite imagery analysis.
- **Wildlife monitoring**: individual animal recognition (e.g., Kaggle whale recognition challenges).

Classification also serves as the **canonical building block** for more complex visual tasks:

- **Object detection**: apply a classifier at every candidate bounding box.
- **Image segmentation**: assign a class to every pixel.
- **Image captioning** and **visual question answering**: build on image-level representations learned for classification.
- **Video understanding**: classify per frame or temporal clip.

20.2 Statistical Learning Framework

The standard framework formalizes image classification as supervised learning:

$$y = f(\mathbf{x})$$

where $\mathbf{x}$ is a feature representation of the image, y is the output label, and f is the **prediction function** to be learned.

Training phase. Given a labeled training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$, learn f by minimizing prediction error on the training set.

Testing phase. Given an unlabeled test instance $\mathbf{x}$, predict $y = f(\mathbf{x})$. The learned function must generalize to inputs not seen during training.

Pipeline. The standard supervised pipeline is:

$$\text{Training images} \xrightarrow{\text{image features}} \text{Training} \rightarrow \text{Learned model}$$

At test time: test image $\xrightarrow{\text{image features}}$ Prediction (using learned model) $\rightarrow$ label.

21 Classic Recognition Pipeline

Before deep learning, recognition systems decoupled feature extraction from classification into two independent stages:

$$\text{Image (pixels)} \longrightarrow \underbrace{\text{Feature representation}}_{\text{hand-crafted}} \longrightarrow \underbrace{\text{Trainable classifier}}_{\text{off-the-shelf}} \longrightarrow \text{Class label}$$

Hand-crafted features had to encode the visual properties that discriminate classes while being robust to nuisance variation (illumination, viewpoint, scale, occlusion). Representations were organized in a hierarchy:

- **Low level**: SIFT (scale-invariant feature transforms), HoG (histogram of oriented gradients), GIST, edge maps.
- **Mid level**: Bag of words (Fisher vectors, VLAD), sliding window detectors, deformable part models.
- **High level**: Contextual dependence, scene-level priors.

The trainable classifier (SVM, logistic regression, random forest) operated on these fixed features. The critical insight from Torralba & Efros is that **three factors** govern recognition performance, roughly in decreasing order of impact:

1. **Data**: more labeled data is almost always better (quality matters more than quantity in the low-data regime). Annotation is the bottleneck.
2. **Representation**: a richer, more invariant feature hierarchy matters more than the choice of classifier.
3. **Learning technique**: the choice of classifier or inference method is the least decisive factor once data and features are fixed.

Remark. Deep learning merges stages 1 and 2: the network learns both the feature hierarchy and the classifier end-to-end from data, replacing hand-crafted features with *deep learned features*.

22 Classification Fundamentals

22.1 Binary and Multiclass Classification

A classifier assigns an input $\mathbf{x} \in \mathbb{R}^d$ to one of C classes. A **decision rule** partitions the input space into **decision regions** — one per class — separated by **decision boundaries**.

- **Binary classification** ($C = 2$): a single decision boundary separates two regions.
- **Multiclass classification** ($C > 2$): the input space is partitioned into C regions. One-vs-one or one-vs-all strategies reduce multiclass to binary; alternatively, a single model can output a C-dimensional score vector.

22.2 Classification Methods

The main families of classification methods, roughly in historical and complexity order:

Method	Characteristic
k-Nearest Neighbours	Non-parametric; decision boundary from local vote
Linear classifiers	Hyperplane boundary; closed-form or convex optimization
Support Vector Machines	Maximum-margin linear classifier; kernel trick for non-linearity
Random Forests	Ensemble of decision trees; handle discrete features naturally
Neural Networks	Learnable non-linear decision boundaries; trained by SGD
Deep Networks	Deep hierarchical feature learning; state of the art on vision

23 Neural Networks for Classification

23.1 Linear Machine

The simplest parameterized classifier maps the input $\mathbf{x} \in \mathbb{R}^d$ to a score vector $\mathbf{s} \in \mathbb{R}^C$ via a linear **score function**:

$$\mathbf{s} = W\mathbf{x}$$

where $W \in \mathbb{R}^{C\times d}$ is the weight matrix. Each row of W is a **class template**: the score for class c is the dot product of $\mathbf{x}$ with the c-th template.

Example 23.1. For a 32×32 RGB image, the input is flattened to $\mathbf{x} \in \mathbb{R}^{3072}$. With $C = 10$ classes, $W \in \mathbb{R}^{10\times 3072}$ has 30,720 parameters. The output $\mathbf{s} \in \mathbb{R}^{10}$ gives one score per class.

23.2 Two-Layer and Three-Layer Networks

Adding one hidden layer with ReLU non-linearity gives a **2-layer neural network**:

$$\mathbf{s} = W_2 \max(0,\, W_1\mathbf{x})$$

where the hidden vector $\mathbf{h} = \max(0, W_1\mathbf{x})$ has dimension H. The network is also called a *1-hidden-layer network*.

A **3-layer (2-hidden-layer) network** adds a further non-linear stage:

$$\mathbf{s} = W_3 \max\big(0,\; W_2 \max(0,\, W_1\mathbf{x})\big)$$

These stacked non-linearities are collectively referred to as **Multi-Layer Perceptrons** (MLPs).

Example 23.2. For CIFAR-10: $d = 3072$, $H = 100$ hidden units, $C = 10$ classes.

$$W_1 \in \mathbb{R}^{100\times 3072} \quad (100 \times 3072 \text{ parameters})$$
$$W_2 \in \mathbb{R}^{10\times 100} \quad (10 \times 100 \text{ parameters})$$

Total: $3{,}072 \times 100 + 100 \times 10 = 308{,}200$ parameters.

Neural Networks: Pros and Cons

Pros	Flexible and general function approximation framework
	Scalable depth: adding layers increases model capacity
Cons	Hard to analyze theoretically; training prone to local optima
	Requires large amounts of training data and compute
	Huge hyperparameter space (architecture, learning rate, ...)

24 Softmax Classifier and Cross-Entropy Loss

24.1 From Scores to Probabilities

The **softmax classifier** (multinomial logistic regression) interprets the raw score vector $\mathbf{s} = f(\mathbf{x}_i; W) \in \mathbb{R}^C$ as **unnormalized log-probabilities** (logits). To obtain a proper probability distribution, apply the **softmax function**:

$$P(Y = k \mid X = \mathbf{x}_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

Example 24.1. Given raw scores $\mathbf{s} = [3.2,\ 5.1,\ -1.7]$ for classes (cat, car, frog):

class	score (logit)	unnorm. prob. (e^s)	prob.
cat	3.2	24.5	0.13
car	5.1	164.0	0.87
frog	-1.7	0.18	0.00

If the true class is "cat", the predicted probability is 0.13.

24.2 Cross-Entropy Loss

Training minimizes the **negative log-likelihood** (cross-entropy) of the correct class:

$$L_i = -\log P(Y = y_i \mid X = \mathbf{x}_i) = -\log \frac{e^{s_{y_i}}}{\sum_j e^{s_j}}$$

Continuing the example with true class "cat" ($y_i = \text{cat}$):

$$L_i = -\log(0.13) = 2.04$$

The loss is low when the model assigns high probability to the correct class, and high otherwise. Minimizing L_i is equivalent to **maximum likelihood estimation**: choose W to maximize $\prod_i P(Y = y_i \mid X = \mathbf{x}_i)$.

24.3 Dataset Loss

The full training objective averages per-sample losses over the dataset:

$$L(W) = \frac{1}{N} \sum_{i=1}^{N} L_i(f(\mathbf{x}_i, W),\ y_i)$$

Training finds $W^* = \arg\min_W L(W)$ by gradient descent:

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W}$$

Remark. Cross-entropy compares the predicted probability vector with the **one-hot** ground-truth vector (1 for the correct class, 0 elsewhere). Perfect predictions yield $L_i = 0$; random predictions over C classes give $L_i = \log C$.

25 CNN Spatial Arithmetic

25.1 Stride and the Integer-Fit Constraint

A filter of size F applied to an input of size N with stride s (no padding) produces an output of size:

$$\left\lfloor \frac{N-F}{s} \right\rfloor + 1 \quad \text{(must be an integer)}$$

Example 25.1. $N = 7$, $F = 3$:

$$s = 1 : \frac{7-3}{1} + 1 = 5 \quad \text{(valid)}$$
$$s = 2 : \frac{7-3}{2} + 1 = 3 \quad \text{(valid)}$$
$$s = 3 : \frac{7-3}{3} + 1 = 2.33 \quad \text{(does not fit — stride 3 cannot tile this input evenly)}$$

Not all combinations of N, F, s are valid. In practice, s and F must be chosen so that $(N-F)/s$ is an integer.

25.2 Zero Padding

To prevent the spatial dimensions from shrinking at every convolutional layer, it is common to **zero-pad** the input with p pixels on each border before applying the filter. With padding, the output size generalizes to:

$$\frac{N + 2p - F}{s} + 1$$

Size-preserving padding. To keep the spatial dimensions unchanged at stride $s = 1$, set $p = (F-1)/2$:

$$\frac{N + 2 \cdot \frac{F-1}{2} - F}{1} + 1 = N$$

This requires F to be odd. Common cases:

F	p (to preserve size at stride 1)
3	1
5	2
7	3

Example 25.2. Input $32 \times 32 \times 3$, 10 filters of size 5×5, stride 1, padding 2.

$$\text{Output spatial size} : \frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32 \quad \Rightarrow \quad 32 \times 32 \times 10$$
$$\text{Parameters} : \text{each filter has } 5 \times 5 \times 3 + 1 = 76 \text{ weights } \Rightarrow\ 10 \times 76 = 760$$

Remark. Without padding, repeated convolutions shrink the spatial dimensions rapidly. A 32×32 input convolved with 5×5 filters at stride 1 becomes $28 \times 28 \to 24 \times 24 \to \cdots$. This is too aggressive for deep networks, making zero-padding essential.

25.3 1×1 Convolutions

A 1×1 convolutional layer applies a filter of spatial size $1 \times 1 \times C_{\text{in}}$ at every spatial location. Each filter computes a **C_{in}-dimensional dot product**, mixing channels without touching neighboring spatial positions. This effectively performs a **linear channel projection**: $C_{\text{in}} \to C_{\text{out}}$ depth-wise at each pixel independently.

Example 25.3. Input $56 \times 56 \times 64$, apply 32 filters of size $1 \times 1 \times 64$:

$$\text{output} = 56 \times 56 \times 32$$

Spatial dimensions are preserved; channel depth is reduced from 64 to 32. Parameters: $32 \times (1 \times 1 \times 64 + 1) = 2{,}080$.

1×1 convolutions are used as bottleneck projections in GoogLeNet/Inception and ResNet to reduce computation before expensive 3×3 or 5×5 convolutions.

25.4 Translation Equivariance

A convolutional layer is **translation equivariant**: applying a spatial translation to the input and then convolving produces the same result as convolving and then applying the same translation to the output. Formally, if T_δ denotes translation by offset δ:

$$\text{CONV}(T_\delta(\mathbf{x})) \approx T_\delta(\text{CONV}(\mathbf{x}))$$

(exact at interior positions; boundary effects break equivariance at the edges due to padding).

This is a direct consequence of weight sharing: the same filter detects the same pattern regardless of where it appears in the input. Equivariance is distinct from **invariance** (where the output does not change at all with translation, as max-pooling approximately achieves).

25.5 Receptive Field

The **receptive field** of a unit in a feature map is the region of the input image that influences its value. Two factors enlarge the receptive field:

- **Number of layers**: each additional layer increases the effective receptive field.
- **Filter size**: larger filters see more of the input at each layer.

Parameter Efficiency: Stacked Small Filters

Two stacked 3×3 conv layers have the same 5×5 receptive field as a single 5×5 layer, but with fewer parameters and an extra non-linearity:

$$\begin{aligned} \text{One } 5 \times 5 \text{ layer}: &\quad 5 \times 5 + 1 = 26 \text{ parameters per filter} \\ \text{Two } 3 \times 3 \text{ layers}: &\quad 2 \times (3 \times 3 + 1) = 20 \text{ parameters per filter} \end{aligned}$$

This is the design principle behind VGGNet.

Effect of max-pooling on receptive field. Max-pooling with stride > 1:

- **Increases** the effective receptive field of subsequent layers (each unit now covers a larger input region).
- **Decreases** spatial resolution (fewer feature map positions).
- **Discards** precise spatial information — the exact location of the maximum within a pooling window is lost.

This trade-off is fundamental to CNN design: high-resolution features are needed for localization tasks, while large receptive fields are needed for recognition. Fully convolutional architectures (strided convolutions instead of pooling) can preserve more spatial information.

26 Training Best Practices

26.1 Train / Validation / Test Split

The goal of training is a classifier with good **generalization** to never-before-seen data. The standard protocol enforces three non-overlapping data splits:

1. **Training set**: learn model *parameters* (weights) by minimizing the training loss.
2. **Validation set** (held-out): tune *hyperparameters* (architecture, learning rate, regularization, ...) by evaluating validation performance. Iterate steps 1 and 2.
3. **Test set**: evaluate final model performance *once*. Looking at test performance during development is data snooping and invalidates the evaluation.

Remark. The validation set acts as a proxy for the test set during development. Selecting hyperparameters to maximize validation accuracy *does* use the validation set, so validation accuracy is an optimistic estimate of generalization. The test set, used exactly once, gives an unbiased estimate.

26.2 Training Dynamics

During training, four curves are monitored:

- **Training loss**: decreases monotonically as the optimizer minimizes the objective.
- **Validation loss**: decreases initially, then rises when overfitting begins.
- **Training accuracy**: increases; should reach near 100% for an expressive model on a small dataset.
- **Validation accuracy**: increases initially, then plateaus or decreases.

A growing gap between training and validation loss/accuracy is the diagnostic signature of **overfitting**. The optimal stopping point (early stopping) is the epoch at which validation loss is minimal.

Part V

Recurrent Neural Networks and LSTM

27 Sequential Data and Problem Formulation

27.1 Dealing with Sequential Data

Standard feedforward networks assume i.i.d. inputs with fixed size. Many real-world modalities are instead **sequences**: audio signals, natural language text, time series (IoT sensors, ECG), financial data, video frames. Processing these requires architectures that respect temporal structure.

27.2 Approaches to Sequence Modelling

Two classical statistical approaches exist before neural sequence models:

- **Autoregressive models**: predict X_{t+1} directly from a window of recent observations X_{t-1}, X_t using learned weights w_{t-1}, w_t.

- **Hidden state models**: introduce a latent *hidden* variable that summarises past information; the next observation is generated from the hidden state.

A good sequence model must handle:

- Sequences of **different lengths**.
- Both **short** and **long-term dependencies**.
- **Order** between elements.
- **Parameter sharing** across positions in the sequence.

27.3 Problem Formulations

Taking text as the running example, the same architecture can be instantiated in four regimes:

Regime	Input	Output / Example
One-to-many	single vector	sequence (image captioning)
Many-to-one	sequence	single vector (sentiment analysis)
Direct many-to-many	sequence	aligned sequence (video captioning)
Delayed many-to-many	sequence	offset sequence (machine translation)

27.4 Representing Text: Word Embeddings

ML cannot directly consume raw text; a numerical descriptor is required. The pipeline is:

1. Build a **vocabulary** from the corpus.
2. **Index** each word (e.g. `a`$\rightarrow$0, `dog`$\rightarrow$2, `walk`$\rightarrow N$).
3. Represent each word as a vector.

Two representation strategies:

- **One-hot encoding**: a vocabulary-length binary vector with a single 1 at the word's index. Very sparse, high-dimensional, hard-coded, and orthogonal — no semantic similarity between words.
- **Word embeddings**: dense, lower-dimensional vectors *learned from data*. Semantically similar words are close in embedding space (e.g. *happy*/*sad* cluster together; *dog*/*cat* cluster together).

27.5 Why Fixed-Window Approaches Fail

Three naive approaches to next-word prediction illustrate the fundamental difficulties:

1. **Small fixed window + one-hot**: concatenate one-hot vectors for the last k words and feed to a classifier. *Problem*: the window may not contain the informative context (long-range dependency).
2. **Bag of all available words**: encode the full preceding context as a single binary vector. *Problem*: word order is discarded ("food was good, not bad" vs. "food was bad, not good" have the same representation).
3. **Large fixed window**: concatenate one-hot vectors for all preceding words. *Problem*: no parameter sharing — knowledge about a word at position i does not transfer to the same word appearing at position j.

The fundamental requirements — variable-length input, order preservation, parameter sharing, long-range dependencies — motivate the RNN.

28 Recurrent Neural Networks

28.1 The Recurrence Relation

An RNN applies the **same function and the same weights** at every time step, maintaining a **hidden state** h_t that summarises all past information:

Definition 28.1 (RNN recurrence)**.**

$$h_t = f_W(h_{t-1},\, x_t)$$

where W are shared weight matrices, x_t is the input at step t, and h_{t-1} is the hidden state from the previous step.

The explicit forward-pass equations are:

$$h_t = \sigma(W_{hh}\, h_{t-1} + W_{xh}\, x_t) \tag{1}$$
$$y_t = W_{hy}\, h_t \tag{2}$$

where W_{hh} weights the recurrent connection, W_{xh} weights the input, and W_{hy} maps the hidden state to the output.

28.2 Key Properties

- **Memory**: recurrence adds memory — the decision at step $t-1$ influences the decision at step t.
- **Causal modelling**: the network models causal relationships between observations.
- **Two sources of input**: the present (x_t) and the recent past (h_{t-1}).
- **Weight sharing**: W_{hh}, W_{xh}, and W_{hy} are shared across all time steps. They act as *filters* that determine how much importance to give to the present input and the past hidden state.

28.3 Unrolling Over Time

An RNN with a self-loop can equivalently be viewed as a sequence of multiple communicating copies of the same network, one per time step. This is called **unrolling** (or unfolding) and makes the computational graph acyclic, enabling standard backpropagation.

28.4 Loss and Backpropagation Through Time (BPTT)

In the many-to-many setting, each step t produces a local loss L_t. The total loss is:

$$L = \sum_{t=1}^{T} L_t$$

The gradient of L_t with respect to the shared weight W_{hh} requires differentiating through all earlier hidden states (chain rule over time):

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial y_t}\frac{\partial y_t}{\partial h_t}\left(\sum_{k=1}^{t}\frac{\partial h_t}{\partial h_k}\frac{\partial h_k}{\partial W_{hh}}\right), \qquad \frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}}$$

The term $\partial h_t/\partial h_k$ is computed as a **product of consecutive Jacobians** along the time axis, one per step from k to t.

28.5 Gradient Pathologies

This product of Jacobians causes two numerical instabilities:

- **Vanishing gradients**: if many factors are < 1, the product collapses to zero. Gradient signals from distant time steps vanish, so model weights are updated only with respect to recent effects. Long-term dependencies cannot be learned.
- **Exploding gradients**: if many factors are > 1, the product diverges. SGD steps become arbitrarily large, destabilising training. A practical fix is **gradient clipping**: if the gradient norm exceeds a threshold τ, rescale it to τ before the update:

$$\Theta_{\text{new}} = \Theta_{\text{old}} - \alpha \nabla_\Theta L(\Theta), \quad \text{with } \nabla_\Theta \leftarrow \frac{\tau}{\|\nabla_\Theta\|} \nabla_\Theta \text{ if } \|\nabla_\Theta\| > \tau$$

The solution to vanishing gradients requires a fundamentally different cell architecture: **gated cells**.

28.6 RNN Applications

RNNs have been successfully applied to: NLP, language modelling, text generation ($x_{t+1} = o_t$), machine translation, image caption generation, speech recognition, and action recognition in video.

29 Long Short-Term Memory (LSTM)

29.1 Motivation

Proposed by Hochreiter and Schmidhuber (1997), the LSTM addresses the vanishing gradient problem by introducing:

- **Connection weights that may change at each time step** — unlike vanilla RNNs with fixed W_{hh}.
- A **cell state** c_t that accumulates information over long durations, separate from the hidden state h_t.
- **Gated units** that learn *when* to forget old state and when to store new information.

29.2 Gated Cells

A **gate** is a sigmoid layer followed by a pointwise multiplication. The sigmoid outputs values in $[0, 1]$: a value of 0 means "let nothing through"; a value of 1 means "let everything through". Gates are differentiable, so their parameters are learned end-to-end.

29.3 LSTM Cell State

The LSTM cell maintains two vectors of length n:

- **Cell state** c_t: the long-term memory. LSTM can erase, write, and read from c_t via three corresponding gates.
- **Hidden state** h_t: the short-term memory / output, analogous to the vanilla RNN hidden state.

Gate values are dynamic: they are computed from the current context (h_{t-1}, x_t) at each step, not fixed.

29.4 The Four Steps of an LSTM Cell

Step 1 — Forget. The **forget gate** f_t decides which information to discard from the previous cell state:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t)$$

$f_t \in (0,1)^n$; element-wise multiplication $c_{t-1} \cdot f_t$ selectively erases components of the cell state.

Step 2 — Store input. The **input gate** i_t decides what new information to write into the cell state, and $\tilde{c}_t$ is the candidate cell content:

$$i_t = \sigma(W_i h_{t-1} + U_i x_t) \tag{3}$$
$$\tilde{c}_t = \tanh(W_c h_{t-1} + U_c x_t) \tag{4}$$

i_t answers "is the new data worth remembering?"; $\tilde{c}_t$ determines how much each component of the cell state should be updated.

Step 3 — Update cell state. The new cell state combines the selectively forgotten old state with the selectively written new content:

$$c_t = c_{t-1} \cdot f_t + i_t \cdot \tilde{c}_t$$

The element-wise multiplication by f_t erases; the addition of $i_t \cdot \tilde{c}_t$ writes.

Step 4 — Output hidden state. The **output gate** o_t filters which parts of the (now updated) cell state are exposed as the hidden state:

$$o_t = \sigma(W_o h_{t-1} + U_o x_t) \tag{5}$$
$$h_t = \tanh(c_t) \cdot o_t \tag{6}$$

$\tanh(c_t)$ squashes the cell state to $[-1,1]$; o_t gates which components are passed on. h_t is then used to produce the output y_t.

LSTM full equations summary

$$\begin{aligned}
f_t &= \sigma(W_f h_{t-1} + U_f x_t) && \text{(forget gate)}\\
i_t &= \sigma(W_i h_{t-1} + U_i x_t) && \text{(input gate)}\\
\tilde{c}_t &= \tanh(W_c h_{t-1} + U_c x_t) && \text{(candidate cell)}\\
c_t &= f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t && \text{(cell state update)}\\
o_t &= \sigma(W_o h_{t-1} + U_o x_t) && \text{(output gate)}\\
h_t &= \tanh(c_t) \cdot o_t && \text{(hidden state)}
\end{aligned}$$

Remark. The cell state c_t flows through the network with only element-wise multiplications and additions — no matrix multiplications along the temporal axis. This additive structure is why gradients do not vanish as severely as in vanilla RNNs.

30 Variants

30.1 Gated Recurrent Unit (GRU)

Introduced by Cho et al. (2014), the **Gated Recurrent Unit (GRU)** simplifies the LSTM by:

- Merging the forget and input gates into a single **update gate** z_t.
- Adding a **reset gate** r_t that controls how much past information to forget.
- Merging the cell state and hidden state into a single hidden state h_t.

The GRU equations are:

$$z_t = \sigma(W_z \cdot [h_{t-1},\, x_t]) \tag{7}$$
$$r_t = \sigma(W_r \cdot [h_{t-1},\, x_t]) \tag{8}$$
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1},\, x_t]) \tag{9}$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \tag{10}$$

The resulting model has fewer parameters than a standard LSTM, is computationally cheaper, and has become increasingly popular in practice.

Reference

K. Cho, B. van Merriënboer et al. (2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". *EMNLP 2014*.

30.2 Transformers

The **Transformer** (Vaswani et al., 2017) replaces recurrence entirely with a self-attention mechanism. At each step, the model attends to *all* other positions in the sequence simultaneously, deciding which parts are most relevant. This eliminates the sequential bottleneck of RNNs, enables massively parallel training, and handles long-range dependencies more effectively.

Reference

A. Vaswani et al. (2017). "Attention Is All You Need". *NeurIPS 2017*. `arXiv:1706.03762`.

Part VI

Attention Mechanism & Transformers

31 From RNNs to Transformers

31.1 The Historical Context

Before Transformers, NLP tasks — especially **sequence-to-sequence** (seq2seq) tasks such as machine translation — relied on recurrent architectures: LSTMs and GRUs. In the visual domain, convolutions dominated for decades. The **attention mechanism** itself was introduced earlier, through "fast weight controllers" (Schmidhuber, 1992–1997), but it only became central to NLP architectures later.

The Transformer architecture, introduced in 2017 in "Attention Is All You Need" (Vaswani et al.), stands on the shoulders of the encoder-decoder paradigm but departs from it in a fundamental way: **recurrence is eliminated entirely**.

31.2 Problems with RNNs/LSTMs

- **Sequential processing**: inputs are processed one after the other, making it impossible to fully exploit modern GPUs designed for parallel computation.
- **Long-range dependencies**: information degrades as sequence length increases; the hidden state must compress an entire context into a fixed-size vector.
- **Vanishing gradients**: gradients shrink over many time steps, making distant context hard to learn.

31.3 Transformers: Key Advantages

- **No recurrence**: the entire sequence is processed in parallel.
- **Self-attention**: any two tokens can interact directly, regardless of their distance in the sequence.
- **Efficiency**: simple matrix operations that are fully parallelizable on modern hardware.
- **Performance**: state-of-the-art on virtually all NLP benchmarks, and increasingly in vision.

Core Idea

Transformers replace recurrence with **self-attention mechanisms**, which analyze the relationships between *all* elements in a sequence simultaneously. Self-attention assigns different importance (weights) to every token, capturing long-range dependencies efficiently and in parallel.

32 Transformer Architecture

The Transformer is an **encoder-decoder** architecture. Its high-level pipeline for a translation task proceeds as follows:

1. **Pre-processing**: tokenization, embedding, positional encoding.
2. **Encoder**: encodes relations among all input elements, producing contextualized representations.
3. **Auto-regressive decoding**: generated outputs are fed back as additional decoder inputs.
4. **Output generation**: the decoder produces a probability vector over the vocabulary for the next token.
5. **De-tokenization**: convert the predicted logits back into words.

32.1 Encoder–Decoder Structure

Definition 32.1 (Encoder)**.** The encoder receives the entire input sequence and outputs an encoded sequence of the *same length*. Its role is to build rich, contextualized representations of the input.

Definition 32.2 (Decoder)**.** The decoder predicts one token at a time, conditioned on (a) the encoder output and (b) the previously predicted tokens. This generation is **auto-regressive** — output elements are produced sequentially, so there is no parallelization at inference time.

Both the encoder and the decoder are **stacks of N identical layers** (the original paper uses $N = 6$). All encoder layers share the same structure; all decoder layers share the same structure. Each decoder layer receives input from both the immediately preceding decoder layer *and* the top encoder layer.

Example 32.1 (Translation)**.** Input: "How are you?" → Encoder produces a matrix representation → Decoder iteratively generates: "¿Cómo estás?"

33 Encoder in Detail

33.1 Components

An encoder transforms raw input tokens into **contextualized representations**. Each encoder layer consists of:

1. **Input Embeddings** (bottom encoder only)
2. **Positional Encoding**
3. **Multi-Head Self-Attention** sub-layer
4. **Add & Norm** (residual connection + layer normalization)
5. **Feed-Forward Neural Network** sub-layer
6. **Add & Norm**

33.2 Input Embeddings

Input tokens (e.g., words) are converted into **numerical vectors** using an embedding algorithm. The embedding layer captures **semantic meaning**: semantically similar words map to nearby vectors. Embedding happens only at the bottom-most encoder; all subsequent encoder layers receive the output of the encoder directly below them. Each encoder receives a list of fixed-size vectors (e.g., dimension 512).

33.3 Positional Encoding

The Transformer has no inherent notion of order: without additional information, shuffling the input tokens would produce the same representation. **Positional encoding** injects position/order information by adding a positional vector of the same dimension as the embedding.

The standard formula uses sine and cosine functions of different frequencies:

$$P(k,\, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \qquad P(k,\, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

where:

- k: position of the token in the input sequence, $0 \le k < L/2$
- d: dimension of the output embedding space
- n: user-defined scalar, set to 10 000 in the original paper
- i: column index, $0 \le i < d/2$; each i controls both a sine and a cosine dimension

Each dimension is characterized by a unique frequency and offset. Values range in $[-1, 1]$, effectively representing each position. The output of the positional encoding layer is a matrix in which each row is an embedded word *summed with* its positional information.

Effect of Positional Encoding

After adding positional encodings, two words will be *close* in the embedding space if they are both semantically similar **and** appear at nearby positions. Using combinations of sine and cosine functions allows the encoder to generalize to sequence lengths not seen during training.

33.4 Stack of Encoder Layers

Each encoder layer consists of **two sub-modules**:

1. A **multi-head self-attention mechanism**.
2. A **pointwise feed-forward network**.

Around each sub-module there is a **residual (skip) connection**, followed by **layer normalization**:

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Residual connections mitigate the vanishing gradient problem, enabling training of deep models.

34 Self-Attention Mechanism

Definition 34.1 (Self-Attention)**.** Self-attention is a mechanism for creating **arbitrarily long-range weighted dependencies** between tokens in a sequence. For each token being processed, it determines how much focus to place on every other token in the same sequence.

Self-attention relates each word in the input to all other words, allowing the encoder to focus on different parts of the input as it processes each token. For example, when encoding the word "it" in "The animal didn't cross the street because **it** was too tired", self-attention can link "it" to "animal" rather than "street".

34.1 Query, Key, Value

Self-attention operates through three vectors derived from each token's embedding:

- **Query** (Q): represents the current token "asking" for relevant context.
- **Key** (K): represents each token "announcing" what it contains.
- **Value** (V): the actual content associated with a key; used to construct the output.

When a query and a key are highly aligned (high dot product), the corresponding value is **emphasized** in the output. The query, key, and value vectors are obtained by multiplying the embedding X by three trainable weight matrices:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

34.2 Step-by-Step Computation

1. **Compute Q, K, V**: multiply embeddings by the learned weight matrices W^Q, W^K, W^V.
2. **Compute scores**: take the dot product of the query vector with the key vector of every other token. For token at position 1: $\text{score}_j = q_1 \cdot k_j$.
3. **Scale**: divide scores by $\sqrt{d_k}$ (e.g., $\sqrt{64} = 8$) to obtain more stable gradients.

4. **Softmax**: normalize the scaled scores so they are all positive and sum to 1. The softmax score indicates how much each word will be represented in the output at this position.
5. **Weight values**: multiply each value vector v_j by its softmax score.
6. **Sum**: sum up the weighted value vectors to produce the output z at this position.

34.3 Matrix Form

In practice, all tokens are processed simultaneously using matrix operations. Packing all embeddings as rows of X:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V}$$

- QK^T is the **score matrix**: entry (i, j) measures how much token i should attend to token j. Geometrically it is an unnormalized cosine similarity.
- Dividing by $\sqrt{d_k}$ prevents the dot products from growing too large in high dimensions, keeping gradients stable.
- Softmax normalizes each row into a valid probability distribution (attention weights).
- Multiplying by V produces the final attended representation.

Key Property

Soft attention is **differentiable**, which makes it directly integrable into end-to-end trained neural networks via backpropagation.

35 Multi-Head Self-Attention

35.1 Motivation

A single attention function might be dominated by the token itself (e.g., z_1 contains mostly information about position 1). **Multi-head attention** extends the model's ability to jointly attend to information from **different representation subspaces** at different positions.

Instead of one attention function, queries, keys, and values are **linearly projected h times** with different learned projections. Each projection defines one **attention head**; all heads operate in parallel, each producing an h-dimensional output.

35.2 Mechanics

With h heads and d_{model} model dimension:

- Each head i has its own weight matrices W_i^Q, W_i^K, W_i^V (randomly initialized, learned during training).
- Each head independently projects the inputs and computes attention, producing Z_i.
- The h output matrices $Z_0, \ldots, Z_{h-1}$ are **concatenated** and multiplied by a learned projection matrix W^O:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

The result is a single matrix that captures information from all heads. The original paper uses $h = 8$ heads.

Why Multi-Head?

Multiple heads allow the model to simultaneously focus on different aspects of context: one head might resolve coreference ("it" → "animal"), another might capture syntactic relations, and yet another might encode positional proximity.

36 Feed-Forward Network and Normalization

36.1 Pointwise Feed-Forward Network

After the (multi-head) attention sub-layer, the output passes through a **pointwise feed-forward network** — the same network applied independently to each position. In the original paper it consists of:

1. Linear layer
2. ReLU activation
3. Linear layer

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\, W_2 + b_2$$

The output merges with the input via another residual connection, followed by another layer normalization.

36.2 Add & Norm Summary

Each sub-layer (self-attention and feed-forward) in every encoder *and* decoder layer is wrapped with:

$$\mathrm{LayerNorm}(x + \mathrm{SubLayer}(x))$$

This residual + normalization pattern stabilizes training and enables stacking many layers.

37 Decoder in Detail

37.1 Structure

The decoder mirrors the encoder but has **three sub-layers** per layer instead of two:

1. **Masked Multi-Head Self-Attention**
2. **Add & Norm**
3. **Encoder-Decoder (Cross) Attention** — the decoder's queries attend to the encoder's keys and values
4. **Add & Norm**
5. **Feed-Forward Network**
6. **Add & Norm**

The decoder operates **auto-regressively**: it begins with a start token and generates tokens one by one, feeding each generated token back as part of its next input, until a special end token is produced.

37.2 Masked Multi-Head Self-Attention

Definition 37.1 (Masking)**.** Masking prevents each decoder position from attending to *future* positions. This is enforced by adding a masking matrix M to the scaled scores before softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

The masking matrix M contains:

- 0 for **allowed positions** (attending to past and current tokens).
- $-\infty$ for **masked positions** (future tokens); after softmax these become 0 attention weight.

Concretely, for a sequence of length 3, the look-ahead mask is:

$$M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}$$

This ensures that predictions at position t depend only on known outputs at positions $1, \ldots, t-1$, making the decoder's generation truly sequential and causal.

37.3 Encoder-Decoder Attention

In this sub-layer, the **queries come from the decoder** (previous decoder layer output), while the **keys and values come from the encoder** (top encoder layer output). This allows each decoder position to attend to all positions in the input sequence, integrating the encoder's rich contextual representations.

37.4 Output Stack

The final decoder layer feeds into:

1. A **linear layer** acting as a classifier projecting to vocabulary size.
2. A **softmax** that converts logits to a probability distribution over the entire vocabulary.

The token with the highest probability is selected (or sampled) as the next output token.

38 Language Pre-Training

Transformers are well-suited to **transfer learning**: pre-train on large unlabeled corpora in an unsupervised manner, then fine-tune on small labeled datasets for specific tasks.

Three broad model families emerge depending on which part of the architecture is retained:

Type	Examples	Use cases
Encoder-only	BERT, RoBERTa, Electra	Text classification, sentiment analysis, sentence similarity, NER
Decoder-only	GPT series, Transformer-XL, DialoGPT	Text generation, machine translation, summarization, chatbots
Encoder-Decoder	T5, BART	Seq2seq tasks: translation, summarization, Q&A

- **Encoder-only models** output hidden states for each token; they do not generate text on their own. They are best for tasks requiring a deep understanding of the full input context.
- **Decoder-only models** generate text sequentially in an auto-regressive fashion. GPT-3 has 175B parameters.
- **Encoder-decoder models** use the full pipeline and excel at transformation tasks where both understanding and generation are required.

Key References

- A. Vaswani et al. (2017). "Attention Is All You Need". *NeurIPS 2017*. `arXiv:1706.03762`.
- A. Dosovitskiy et al. (2021). "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale". *ICLR 2021*. `arXiv:2010.11929`.

Part VII

Beyond Supervised Learning: Unsupervised Learning, Clustering & Self-Supervised Methods

39 Types of Learning: A Taxonomy

39.1 Supervision Axes

Learning algorithms can be classified along two independent axes:

- **With teacher vs. without teacher**: does the system receive explicit labels or target signals?
- **Active vs. passive**: does the learner choose which examples to query, or is the data fixed?

This yields four quadrants:

	With Teacher	**Without Teacher**
Passive	Supervised learning	Unsupervised learning
Active	Active learning	Exploration (RL)

39.2 Motivation for Unsupervised Learning

LeCun's Cake Analogy

"If intelligence is a cake, the bulk of the cake is **unsupervised learning**, the icing on the cake is **supervised learning**, and the cherry on the cake is **reinforcement learning**."
— Yann LeCun

The information-theoretic argument: a typical supervised dataset provides only 10–10,000 bits per sample (one label). The raw data itself contains millions of bits. Supervised learning

therefore discards most of the available information. By contrast, unsupervised methods can exploit the *full* data distribution.

Why unsupervised learning is hard: unlike supervised learning, there is no clear objective function derived from labels. The "right" representation is task-dependent and not trivially defined.

Key Goals of Unsupervised Learning

- Learn the underlying structure of the data distribution.
- Build **general-purpose representations** that transfer to downstream tasks.
- Exploit the vastly larger pool of unlabeled data.

40 Self-Supervised Learning

40.1 Core Idea

Self-supervised learning (SSL) is a form of unsupervised learning in which the supervision signal is derived automatically from the data itself — no human annotations are required. The procedure is:

1. Design a **pretext task** whose labels can be generated automatically from the raw data.
2. Train a model to solve the pretext task.
3. Use the learned representation for downstream tasks (transfer learning).

Definition 40.1 (Pretext Task)**.** A **pretext task** is an auxiliary learning objective created from the data structure itself (e.g., predicting the relative position of image patches). Solving the pretext task forces the model to learn semantically useful representations, even though the task itself may not be the end goal.

40.2 Context Prediction (Doersch et al., ICCV 2015)

One of the earliest successful SSL methods for visual representation learning. The key idea: given a central image patch, predict the **relative spatial position** of a surrounding patch (one of 8 possible positions).

1. Sample a random 96×96 patch from an image.
2. Sample one of its 8 neighbours.
3. Train a CNN to classify which of the 8 positions the neighbour occupies.
4. The CNN must understand spatial layout and object semantics to solve the task.

Reference

C. Doersch, A. Gupta, A. Efros. "Unsupervised Visual Representation Learning by Context Prediction". *ICCV 2015*.

41 Learning by Clustering

41.1 General Framework

A natural approach to unsupervised representation learning: use cluster assignments as pseudo-labels to train a network.

Key Observation

Even a **randomly-initialized CNN** already produces a feature space with non-trivial structure. Clustering in that space yields assignments that, used as pseudo-labels, can bootstrap meaningful representations.

The iterative clustering loop:

1. **Feature extraction**: pass all images through the current CNN; collect feature vectors.
2. **Clustering**: run k-means (or similar) on the feature vectors to obtain k clusters.
3. **Pseudo-label assignment**: assign each image the index of its cluster.
4. **CNN fine-tuning**: train the CNN with cross-entropy loss on the pseudo-labels.
5. Repeat from step 1.

42 Progressive Unsupervised Learning (PUL)

42.1 Problem Setting: Unsupervised Person Re-Identification

Person re-identification (re-ID) is the task of matching images of the same person across cameras with non-overlapping fields of view. It is challenging due to viewpoint, illumination, and appearance changes.

In the **unsupervised** setting: no cross-camera identity labels are available. Only *within-camera* tracklets (sequences of frames) provide weak supervision.

42.2 PUL: Formulation

Progressive Unsupervised Learning (Fan et al., ACM TOMM 2018) addresses this with an iterative self-paced scheme combining k-means clustering with CNN fine-tuning and selective sample inclusion.

Definition 42.1 (PUL Setup)**.** Let $\mathcal{X} = \{x_i\}_{i=1}^N$ be the unlabeled cross-camera dataset, and let $\{\hat{y}_i\}$ be cluster pseudo-labels. The model learns a feature embedding f_θ such that same-identity images cluster together.

The three core optimization objectives:

(1) Cluster assignment — find pseudo-labels $\hat{y}$ by k-means:

$$\hat{y}_i = \arg\min_k \|f_\theta(x_i) - \mu_k\|_2^2 \tag{11}$$

where μ_k is the centroid of cluster k.

(2) Sample selection — include only samples close to their centroid:

$$\mathcal{S} = \{x_i : \|f_\theta(x_i) - \mu_{\hat{y}_i}\|_2^2 \leq \tau\} \tag{12}$$

where τ is a distance threshold. This is the **self-paced** component: start with easy (unambiguous) samples, progressively include harder ones as the model improves.

(3) CNN fine-tuning — minimize cross-entropy on selected samples:

$$\mathcal{L} = -\frac{1}{|\mathcal{S}|} \sum_{x_i \in \mathcal{S}} \log p(\hat{y}_i \mid x_i; \theta) \tag{13}$$

42.3 Algorithm 1: PUL

Algorithm: Progressive Unsupervised Learning

1. **Initialise** CNN f_θ (pre-trained on a source domain with labels).
2. **Extract features**: $F = \{f_\theta(x_i)\}_{i=1}^N$.
3. **Run** k**-means** on F; obtain pseudo-labels $\{\hat{y}_i\}$ and centroids $\{\mu_k\}$.
4. **Self-paced selection**: form $\mathcal{S}$ using the distance threshold τ.
5. **Fine-tune** f_θ on $\mathcal{S}$ with cross-entropy loss.
6. **Increase** τ (include more samples next round) and **decrease** k (merge clusters) progressively.
7. **Repeat** steps 2–6 until convergence.

42.4 Self-Paced Learning Intuition

Definition 42.2 (Self-Paced Learning)**. Self-paced learning** is a curriculum strategy where the model is trained first on the easiest (most confident) examples, and progressively trained on harder ones as learning progresses. This prevents early-stage noise from corrupting the learned representation.

In PUL:

- Early iterations: small $\tau \Rightarrow$ only samples very close to centroids are selected $\Rightarrow$ high-confidence pseudo-labels.
- Later iterations: τ is increased $\Rightarrow$ more samples included, including ambiguous ones near cluster boundaries.
- k is decreased progressively: coarse-to-fine clustering, merging related pseudo-identities.

42.5 Semi-Supervised Extension

PUL can be extended to the **semi-supervised** setting when a small number of labeled cross-camera pairs are available. The labeled pairs provide an additional supervised loss term:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{unsup}} + \lambda\, \mathcal{L}_{\text{sup}}$$

This anchors the feature space, significantly reducing the noise in early cluster assignments.

Reference

H. Fan, L. Zheng, C. Yan, Y. Yang. "Unsupervised Person Re-identification: Clustering and Fine-tuning". *ACM TOMM 2018*.

43 DeepCluster

43.1 Motivation

DeepCluster (Caron et al., ECCV 2018) scales the clustering-based SSL approach to ImageNet-scale data. The central observation: *clustering and representation learning are mutually reinforcing* — better features yield better clusters, and better pseudo-labels yield better features.

43.2 Pipeline

DeepCluster Pipeline

1. **Feature extraction**: forward-pass the full dataset through the current ConvNet; collect the penultimate-layer (FC2-like) features.
2. **Pre-processing**: apply **PCA** for dimensionality reduction, then **L2 normalisation**.
3. **k-means clustering**: cluster all feature vectors into k clusters (e.g., $k = 10{,}000$ for ImageNet).
4. **Pseudo-label assignment**: use cluster indices as class labels for the entire dataset.
5. **CNN training**: train the ConvNet with **cross-entropy loss** on the pseudo-labels via standard mini-batch SGD + backpropagation.
6. **Repeat** steps 1–5 for the next epoch.

43.3 Architecture and Implementation Details

- **Backbone**: AlexNet or VGG-16; ImageNet-scale experiments.
- **Number of clusters**: $k = 10{,}000$ — much larger than the 1,000 ImageNet ground-truth classes; allows finer granularity.
- **Initial label generation**: features from the FC2 layer of the randomly-initialised network are already structurally non-trivial; the first k-means run gives a noisy but non-degenerate starting point.
- **Data augmentation**: random crops and horizontal flips are applied during CNN training (consistent with supervised ImageNet training) to improve generalisation.
- **Trivial solution prevention**: without countermeasures, k-means may collapse features onto a small number of clusters. DeepCluster addresses this by:
  - Re-assigning empty clusters.
  - Uniform sampling within each cluster during mini-batch construction (to counter class-size imbalance).

43.4 Why It Works

Example 43.1 (DeepCluster Feature Quality)**.** After sufficient iterations, the convolutional filters of the DeepCluster-trained AlexNet closely resemble those of a supervised AlexNet — oriented Gabor filters, colour blobs, and higher-level part detectors emerge purely from the clustering signal. The representations transfer competitively to PASCAL VOC classification and detection benchmarks.

- **Iteration 0** (random init): features are unstructured; clusters are noisy.
- **After E epochs**: features encode semantic content; clusters become semantically coherent.
- The self-reinforcing loop drives the network toward semantically meaningful representations without any labels.

Reference

M. Caron, P. Bojanowski, A. Joulin, M. Douze. "Deep Clustering for Unsupervised Learning of Visual Features". *ECCV 2018*. `arXiv:1807.05520`.

Part VIII

Self-Supervised Learning: Pretext Tasks and Contrastive Methods

44 Motivation and Problem Setting

44.1 Two Practical Limitations of Supervised Deep Learning

Two intertwined limitations motivate self-supervision:

1. **DNNs need a lot of data to generalise well.** The classical remedy is **pretraining on a large dataset for a related task** (e.g. ImageNet) and then transferring/fine-tuning. This still requires a large *labelled* source dataset.
2. **Obtaining human annotations for large datasets is expensive or unfeasible.** The remedy is **unsupervised representation learning** — learning without human annotation.

Self-supervised learning (SSL) is the subset of unsupervised learning in which the supervision is not random or implicit, but is constructed automatically from the data itself.

Why Self-Supervision?

- Building custom labelled datasets for every task is expensive.
- Annotation is almost impossible in some domains, e.g. medical imaging.
- There is a vast supply of unlabelled images and videos (Facebook, YouTube, etc.).
- Cognitively plausible: it resembles how children learn — by interaction with the world rather than by being shown labels.

44.2 Definition of Self-Supervision

Definition 44.1 (Self-Supervised Learning)**.** A learning paradigm in which the supervision signal is **generated automatically from the data itself** (no human-provided labels). The general recipe is:

- **Hide** part of the data;
- Train a network to **predict the hidden part** given the visible part.

The auxiliary task must be designed so that, in order to solve it, the network is forced to learn a *semantic representation*.

45 Pretext and Downstream Tasks

45.1 Two-Stage Pipeline

Self-supervised feature learning proceeds in two stages:

Pretext + Downstream

1. **Self-supervised pretext task training** — on a large *unlabelled* dataset. The ConvNet is trained to solve a pre-designed auxiliary (**pretext**) task whose labels come for free.
2. **Knowledge transfer** — the learned weights are reused.
3. **Supervised downstream task training** — on a possibly small *labelled* dataset, using the pretrained features (linear evaluation, fine-tuning, etc.).

- **Pretext task**: pre-designed auxiliary objective; large data; no human annotation.
- **Downstream task**: the actual goal (image classification, semantic segmentation, object detection, action recognition, ...); typically requires human annotations and is also used to *evaluate* the quality of the learned features.
- In some cases the pretext and downstream task coincide.

45.2 Designing a Good Pretext Task

- Supervision **cannot come from humans** — it must be derivable from the data itself (or, more arguably, from another algorithm).
- The pretext task must require **visual reasoning**: solving it should be impossible without learning genuine visual or semantic features.

45.3 Taxonomy of Pretext Tasks

Following the survey by Jing & Tian (2019), four broad families exist:

Family	Examples
Generation-based (image)	Image generation (GAN), super-resolution, inpainting, colorisation.
Generation-based (video)	Video generation (GAN), video colorisation, future-frame prediction.
Context-based (spatial)	Image jigsaw puzzle, geometric transformation prediction (rotation), relative position, clustering (context similarity).
Context-based (temporal)	Frame-order verification, frame-order recognition.
Cross-modal	Optical-flow / RGB correspondence, audio-visual correspondence, ego-motion.

Remark. A separate category — *free semantic-label* methods (semantic/instance segmentation labels obtained by another algorithm) — is sometimes listed but is conceptually weaker, because the supervision comes from a model rather than from the data itself.

45.4 Typical Downstream Tasks

- **Images**: image classification, semantic segmentation (e.g. FCN), object detection (e.g. Fast R-CNN).
- **Videos**: human action recognition.

Reference

L. Jing, Y. Tian. "Self-supervised Visual Feature Learning with Deep Neural Networks: A Survey". `arXiv:1902.06162`, 2019.

46 Generation-Based Pretext Tasks (Images)

The common idea: train a network to *generate* or *reconstruct* an image (or part of it). To produce realistic outputs the encoder must capture the statistics and semantics of natural images.

46.1 Autoencoder

The autoencoder is the foundational generation-based pretext task:

- An **encoder** maps the input image to a low-dimensional **encoded representation**.
- A **decoder** reconstructs the input from this representation.

The pretext objective is reconstruction error (e.g., pixel-wise L_2). The bottleneck forces the encoder to compress the data, learning useful features for downstream tasks.

Reference

G. E. Hinton, R. R. Salakhutdinov. "Reducing the dimensionality of data with neural networks". *Science*, 2006.

46.2 Generative Adversarial Networks (GANs)

A **GAN** consists of two networks trained in opposition:

- A **generator** G takes random noise and produces a fake image.
- A **discriminator** D receives both real images (from the training set) and fake images, and must classify them as real or fake.

Training is a minimax game: G tries to fool D, while D tries to distinguish. At equilibrium, the generator captures the data distribution. The discriminator's intermediate features can be transferred as a representation.

Reference

I. Goodfellow et al. "Generative Adversarial Nets". *NIPS 2014*.

46.3 Image Colorisation

Pretext task: given a monochrome (lightness L) input, predict the per-pixel colour (a, b channels in Lab colour space).

- Implementation as a **classification** problem over a quantised (a, b) grid (313 bins), with **class re-balancing** based on ImageNet colour statistics to counter the dominance of grey/brown pixels.
- Architecture: stack of dilated (*à trous*) convolutions producing a $H/4 \times W/4 \times 313$ probability map; the predicted (a, b) image is then upsampled and recombined with the input L channel.

To predict colours plausibly the network must recognise objects (sky is blue, grass is green, faces are skin-toned), inducing semantic features.

Reference

R. Zhang, P. Isola, A. A. Efros. "Colorful image colorization". *ECCV 2016*.

46.4 Super-Resolution: SRGAN

SRGAN (Ledig et al., CVPR 2017) recovers a high-resolution image from a low-resolution input.

- **Generator**: a deep residual network with skip connections; the resolution is increased via PixelShuffler upsampling layers.
- **Discriminator**: a CNN with LeakyReLU activations and BatchNorm, ending in a sigmoid that outputs "real high-res" vs. "super-resolved".
- **Loss**: the generator is trained with an L_2 pixel loss (or perceptual loss) *plus* an adversarial loss; the discriminator uses binary cross-entropy.

Reference

C. Ledig et al. "Photo-realistic single image super-resolution using a generative adversarial network". *CVPR 2017*.

46.5 Image Inpainting (Context Encoders)

Pretext task: a square region is removed from the image; the network must predict the missing patch using only the surrounding context.

- Combination of L_2 reconstruction loss (encourages average colour and rough structure) **and** an adversarial loss (encourages sharp, plausible details).
- To produce a coherent inpainting the encoder must understand object structure and scene semantics.

Reference

D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, A. Efros. "Context encoders: Feature learning by inpainting". *CVPR 2016*.

46.6 Generation-Based Pretext Tasks (Videos)

The same generation principles extend to video:

- **Video generation with GANs** — VideoGAN (Vondrick et al., NIPS 2016).
- **Video colorisation** — end-to-end voxel-to-voxel prediction (Tran et al., CVPRW 2016).
- **Future-frame generation** — decomposing motion and content for natural video sequence prediction (Villegas et al., ICLR 2017).

47 Context-Based Pretext Tasks (Images)

These tasks exploit *spatial* structure: parts of the image are rearranged or transformed, and the network must recover the original arrangement.

47.1 Geometric Transformation: Image Rotations (RotNet)

Pretext task: an image is rotated by a multiple of 90° (0°, 90°, 180°, 270°) and the network must predict the angle (4-way classification).

- Each input image X generates four training samples X^0, X^1, X^2, X^3, one per rotation $g(X, y)$ with $y \in \{0, 1, 2, 3\}$.
- A single ConvNet $F(\cdot)$ is trained to maximise $F^y(X^y)$ for the correct label y.
- The intuition: to identify the canonical orientation the network must understand object semantics (sky is up, ground is down, animals stand in a particular pose).

Example 47.1 (RotNet vs. other SSL methods (PASCAL VOC 2007/2012))**.** With AlexNet, RotNet reaches **72.97% mAP** on classification (all layers fine-tuned), **54.4% mAP** on detection, and **39.1% mIoU** on segmentation, compared with 79.9 / 56.8 / 48.0 for fully supervised ImageNet labels and clearly above earlier SSL methods such as Egomotion, Context Encoders, Tracking, Context Prediction, Colorisation, Jigsaw, BiGAN, NAT, Split-Brain and Counting.

Reference

S. Gidaris, P. Singh, N. Komodakis. "Unsupervised representation learning by predicting image rotations". *ICLR 2018*.

47.2 Relative Position of Patches

Pretext task: extract a central patch and one of its 8 neighbours from an image; predict the relative position of the neighbour (8-way classification).

- Two AlexNet-style towers with shared weights process the two patches; their fc6 features (4096-dim) are concatenated and passed through fc7, fc8 (4096), and fc9 (8 outputs).
- Solving the task requires understanding both low-level continuity (texture, edges) and high-level object structure.

Reference

C. Doersch, A. Gupta, A. A. Efros. "Unsupervised visual representation learning by context prediction". *ICCV 2015*.

47.3 Solving a Jigsaw Puzzle

A generalisation of the relative-position task:

- An image is divided into $3 \times 3 = 9$ patches; the patches are shuffled according to a randomly selected permutation from a fixed set (e.g. 64 permutations).
- Nine identical CNN towers (shared weights) extract per-patch features; their concatenated representation feeds fc6/fc7/fc8 with a softmax over the permutation set.
- The network must reason about the global spatial layout, learning rich object-level features.

Reference

M. Noroozi, P. Favaro. "Unsupervised learning of visual representations by solving jigsaw puzzles". *ECCV 2016*.

47.4 Clustering as Context Similarity

A context-based pretext task in which the "context" is the global feature space:

- The ConvNet extracts features from all images; **clustering** (e.g., k-means) yields pseudo-labels.
- The pseudo-labels are used as classification targets; the ConvNet is updated by backpropagation.
- The procedure is iterative: better features ⇒ better clusters ⇒ better pseudo-labels ⇒ better features (DeepCluster).

Reference

M. Caron, P. Bojanowski, A. Joulin, M. Douze. "Deep clustering for unsupervised learning of visual features". *ECCV 2018*.

47.5 Multi-Task Self-Supervised Learning

Combining several pretext tasks in a single network usually improves transfer:

- Deeper networks (ResNet vs. AlexNet) help.
- **Colour** and **Relative Position** are individually superior to **Exemplar**.
- Combinations (Rel. Pos. + Colour, Rel. Pos. + Exemplar, all three) further close the gap with strong supervision.

Example 47.2 (Multi-task transfer (ImageNet top-5 / PASCAL VOC mAP))**.** Rel. Pos.: 59.21 / 66.75; Colour: 62.48 / 65.47; Exemplar: 53.08 / 60.94; Rel. Pos. + Colour: 66.64 / 68.75; Rel. Pos. + Exemplar: 65.24 / 69.44; All three: 68.65 / 69.48; ImageNet labels (supervised): 85.10 / 74.17.

Reference

C. Doersch, A. Zisserman. "Multi-task self-supervised visual learning". *ICCV 2017*.

48 Context-Based Pretext Tasks (Videos)

Videos add a *temporal* axis. The supervision signal can be derived from frame ordering and the natural "arrow of time".

48.1 Frame-Order Recognition and Verification

Two related pretext tasks:

- **Frame-order recognition** (Lee et al., ICCV 2017): given a shuffled set of frames, predict their correct chronological order.
- **Frame-order verification** (Misra et al., ECCV 2016 — "Shuffle and Learn"): given a triplet of frames, decide whether the order is correct or shuffled. Three CNNs with shared parameters extract features for each frame; the features are fused and passed to a binary classifier (*correct order* / *incorrect order*).

To solve these tasks the network must understand motion, causality, and the typical dynamics of human actions.

48.2 Learning and Using the Arrow of Time

Pretext task (Wei et al., CVPR 2018): predict whether a video is being played *forward* or *backward*.

The task can require:

- **Low-level** understanding (e.g., physics — smoke rising vs. falling);
- **High-level** reasoning (e.g., semantics — a horse rider mounting vs. dismounting);
- Familiarity with very subtle effects (e.g., facial expressions during speech);
- Familiarity with camera conventions (e.g., trains entering vs. leaving a station).

The paper investigates three questions:

1. Can a reliable arrow-of-time classifier be trained on large-scale natural videos while *avoiding artificial cues* introduced during video production?
2. What does the model learn about the visual world in order to solve the task?
3. Can this learned commonsense knowledge be applied to other tasks? → **video representation learning** and **video forensics**.

References

- H.-Y. Lee, J.-B. Huang, M. Singh, M.-H. Yang. "Unsupervised representation learning by sorting sequences". *ICCV 2017*.
- I. Misra, C. L. Zitnick, M. Hebert. "Shuffle and learn: unsupervised learning using temporal order verification". *ECCV 2016*.
- D. Wei, J. Lim, W. Freeman, A. Zisserman. "Learning and Using the Arrow of Time". *CVPR 2018*.

49 Multimodal (Cross-Modal) Pretext Tasks: Sound + Vision

Videos provide a free pairing of two modalities: **vision** (frames) and **audio** (waveform). Naturally co-occurring audio-visual events are powerful supervision.

49.1 What Can Be Learnt by Watching and Listening?

- **Good representations**: both visual features and audio features.
- **Intra- and cross-modal retrieval**: aligned audio and visual embeddings.
- Localising **"what is making the sound"**: identify, in the image, the object that produced a given sound.

49.2 Two Types of Audio-Visual Proxy Task

1. **Audio-visual correspondence (AVC)** — predict whether an image and an audio clip come from the same video. Trains *semantic* alignment.
2. **Audio-visual synchronisation (AVS)** — predict whether the two streams are temporally synchronised. Trains *attention* / fine temporal alignment.

49.3 Audio-Visual Correspondence: L^3-Net and AVE-Net

Both networks share a two-stream architecture (Arandjelović & Zisserman, ICCV 2017 / ECCV 2018):

- A **vision subnetwork** processes a $224 \times 224 \times 3$ image, output $14 \times 14 \times 512$.
- An **audio subnetwork** processes a 257×200 log-spectrogram of 1-second 48 kHz audio, output $16 \times 12 \times 512$.

L^3-Net: pool_4 features ($1 \times 1 \times 512$) from each branch are concatenated (1024-dim), passed through fc1 ($1024 \to 128$) and fc2 (128×2), with a softmax over 2 classes (**corresponds: yes/no?**).

AVE-Net: each branch produces an L_2-normalised 128-dim embedding; the **Euclidean distance** between the two embeddings is the only feature passed to the classifier (fc3 + softmax). The network is forced to learn aligned audio-visual embeddings, which is what makes **cross-modal retrieval** possible.

Example 49.1 (Cross-modal/intra-modal retrieval, AudioSet-Instruments, average nDCG@30)**.** Random chance: 0.407 across all settings. AVE-Net: **im-im 0.604, im-aud 0.561, aud-im 0.587, aud-aud 0.665**, beating L^3-Net (0.567 / 0.418 / 0.385 / 0.653), L^3-Net + CCA, VGG16-ImageNet (im-im 0.600), and VGG16-ImageNet + L^3-Audio CCA.

49.4 Sound Localisation

A small modification of the AVC architecture allows pixel-level sound localisation. Instead of pooling the visual features to a single vector, the audio embedding is dot-producted with *every spatial position* of the visual feature map:

- Vision branch: convolutions output $14 \times 14 \times 128$.
- Audio branch: pool_4 + fc1 + fc2 produce a single 128-dim audio descriptor.
- All pairwise scalar products give a $14 \times 14 \times 1$ **per-location correspondence map**.
- A sigmoid converts each location into a correspondence score (**Where?**). Max-pooling the map gives a global "does it correspond? Yes/No" score.

References

- R. Arandjelović, A. Zisserman. "Look, Listen, and Learn". *ICCV 2017*.
- R. Arandjelović, A. Zisserman. "Objects that Sound". *ECCV 2018*.

49.5 Faces and Speech (Talking Heads)

Specialising audio-visual SSL to talking-head videos: *use faces and voice to learn from each other*. Two proxy tasks:

1. Predict audio-visual **correspondence**;
2. Predict audio-visual **synchronisation** — given a short window of mouth movements and an audio segment, decide whether they are aligned (*positive pair*) or shifted (*negative pair*).

The resulting network supports several practical applications: **audio-to-video synchronisation**, **active-speaker detection**, **voice-over rejection**, and **visual features for lip reading**.

Reference

J. S. Chung, A. Zisserman. "Out of time: automated lip sync in the wild". *Workshop on Multi-view Lip-reading, ACCV 2016.*

49.6 Audio-Visual Scene Analysis

Owens & Efros (ECCV 2018): the visual and audio components of a video are modelled **jointly** with a *fused multisensory representation*. The network is trained in a self-supervised way to predict whether the video frames and the audio are temporally aligned.

The learned representation is then used for several downstream applications:

(a) **Sound-source localisation** (visualising where in a frame the sound originates);

(b) **Audio-visual action recognition** (e.g., "cutting in kitchen");

(c) **On/off-screen audio-source separation**: separating the spectrogram of sounds whose source is visible in the frame from sounds coming from off-screen.

Reference

A. Owens, A. Efros. "Audio-Visual Scene Analysis with Self-Supervised Multisensory Features". *ECCV 2018.*

49.7 Biometric Embeddings (Face/Voice Correspondence)

The Question

How much can we infer from the voice about the face, and vice versa?
A famous illustration: in the film "Die Hard", John McLane (Bruce Willis) recognises Sgt. Al Powell at the end of the film, having only spoken to him by radio.

Two SSL approaches train cross-modal embeddings using face images and voice recordings of the same identities:

- **Cross-modal biometric matching** (Nagrani et al., CVPR 2018) — with modality-specific encoders followed by modality-shared layers and a softmax over N-way matching.
- **Learnable PINs** (Nagrani et al., ECCV 2018) — learns a **joint embedding space** where corresponding face and voice points lie close together (Face A near Voice A; Face B near Voice B). A face subnetwork f_θ produces a 256-dim embedding from a face frame; a voice subnetwork g_ϕ produces a 256-dim embedding from a 3-second spectrogram. Pairs are selected by *Curriculum Mining*, and a **contrastive loss** pulls positive pairs together and pushes negatives apart.

Evaluation:

- **Cross-modal verification**: given a face input and a speech segment, decide if they belong to the same identity.
- **Cross-modal retrieval**: given a query in one modality, retrieve all semantically matching templates in the other modality.
- **One-shot learning** for TV-show character retrieval.

References

- A. Nagrani, S. Albanie, A. Zisserman. "Seeing Voices and Hearing Faces: Cross-modal biometric matching". *CVPR 2018*.
- A. Nagrani, S. Albanie, A. Zisserman. "Learnable PINs: Cross-Modal Embeddings for Person Identity". *ECCV 2018*.

49.8 Recap of Multimodal SSL

- Self-supervised learning from images/video alone enables learning without explicit supervision and is roughly *on par with ImageNet supervised training*.
- Self-supervised learning **from videos with sound** additionally enables intra- and cross-modal retrieval and learns to localise sounds; the proxy tasks (synchronisation, attention) are not just proxies — they are directly applicable.
- The same paradigm extends to other domains with paired signals: face/voice, infrared/visible, RGB/depth, etc.

50 Comparing Pretext Tasks: Performance

A common evaluation protocol is to freeze the SSL-pretrained backbone (AlexNet) and train linear classifiers on the feature maps of each convolutional layer (ImageNet, Places). Higher numbers indicate more semantically informative features.

Linear Classification on AlexNet Feature Maps

Method	Type	conv1	conv2	conv3	conv4	conv5
ImageNet labels (sup.)	—	19.3	36.3	44.2	48.3	50.5
Random (scratch)	—	11.6	17.1	16.9	16.3	14.1
Colorful Colorisation	Generation	12.5	24.5	30.4	31.5	30.3
BiGAN	Generation	17.7	24.5	31.0	29.9	28.0
SplitBrain	Generation	17.7	29.3	35.4	35.2	32.8
ContextEncoder	Context	14.1	20.7	21.0	19.8	15.5
ContextPrediction	Context	16.2	23.3	30.2	31.7	29.6
Jigsaw	Context	**18.2**	28.8	34.0	33.9	27.1
Learning2Count	Context	18.0	30.6	34.3	32.5	25.7
DeepCluster	Context	13.4	**32.3**	**41.0**	**39.6**	**38.2**

Several patterns emerge:

- Context-based methods (especially DeepCluster) tend to dominate at deeper layers (conv3–conv5).
- No SSL method matches the supervised ImageNet baseline at the deepest layer (50.5), but DeepCluster (38.2) gets closer than older methods.
- The combination of clustering and self-supervised learning substantially closes the gap.

51 Self-Supervised, Contrastive Methods

Contrastive learning brought SSL to a new level of performance. The methods covered here are: **SimCLR**, **MoCo**, **BYOL**, **Supervised Contrastive (SupCon)**, and **SimCLR+**.

51.1 SimCLR — A Simple Framework for Contrastive Learning

Idea: SimCLR learns representations by **maximising agreement between differently augmented views of the same image** via a contrastive loss in a latent space.

Procedure:

1. Two augmentation operators $t \sim \mathcal{T}$ and $t' \sim \mathcal{T}$ are sampled from the same family $\mathcal{T}$ and applied to each input x, producing two correlated views $\tilde{x}_i, \tilde{x}_j$.
2. A base encoder $f(\cdot)$ produces representations $h_i = f(\tilde{x}_i)$, $h_j = f(\tilde{x}_j)$.
3. A projection head $g(\cdot)$ (an MLP) maps these to z_i, z_j where the contrastive loss is computed.
4. After pretraining the projection head g is **discarded**; only the encoder f and its representation h are used for downstream tasks. The paper shows h is more discriminative than z for transfer.

Loss (NT-Xent): from a mini-batch of N examples we obtain $2N$ augmented views. For a positive pair (i, j):

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \tag{14}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature. The other $2(N-1)$ views in the batch are treated as negatives. The final loss is the average over both directions of all positive pairs in the mini-batch.

Important Practical Note

SimCLR requires a **large batch size** to provide enough negative samples. This is a key limitation.

51.1.1 Importance of Data Augmentation

A grid of pairwise compositions of augmentations (crop, cutout, colour distortion, Sobel, noise, blur, rotate) was studied:

- **No single transformation** suffices to learn good representations, even though the model can almost perfectly identify positive pairs.
- **Composing** augmentations makes the contrastive task harder, but the quality of the representation *improves dramatically*. Crop + Colour is a particularly strong combination.

51.1.2 SimCLR Results

Example 51.1 (ImageNet linear evaluation)**.** With ResNet-50 (1×, 2×, 4× width multipliers), SimCLR substantially outperforms previous SSL methods (CPCv2, CMC, MoCo, BigBiGAN, AMDIM, PIRL, Rotation, InstDisc, ...) and approaches the supervised ResNet-50 baseline.

Example 51.2 (Transfer over 12 datasets, ResNet-50 (4×))**.** Linear evaluation: SimCLR is statistically tied with supervised on 7 of 12 datasets, slightly worse on others.
Fine-tuned: SimCLR significantly outperforms the supervised baseline on **5 of 12 datasets**; the supervised baseline wins on only 2 (Pets, Flowers); the remaining 5 are statistically tied.

Reference

T. Chen et al. "A simple framework for contrastive learning of visual representations". *ICML 2020*.

51.2 MoCo — Momentum Contrast

MoCo addresses SimCLR's main weakness (the dictionary of negatives is bounded by the mini-batch size) by decoupling the dictionary size from the batch size.

Idea: train a visual encoder by matching an encoded **query** q to a **dictionary of encoded keys** $\{k_0, k_1, k_2, \ldots\}$ using a contrastive loss. The dictionary keys are defined on-the-fly by a set of data samples.

Architecture:

- A **query encoder** f_q with parameters θ_q, updated by backpropagation.
- A **key encoder** (**momentum encoder**) f_k with parameters θ_k, **not** updated by gradients but by a **momentum update**:

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q, \tag{15}$$

  with momentum coefficient $m \in [0, 1)$.

Dictionary as a queue: keys are stored in a FIFO queue. Each mini-batch is enqueued and the oldest mini-batch dequeued, decoupling the dictionary size from the mini-batch size. Because f_k evolves slowly (large m), keys produced in different mini-batches are encoded by encoders that are *nearly* the same — the dictionary is large but consistent.

Contrastive loss (InfoNCE):

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+/\tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i/\tau)} \tag{16}$$

This is the log loss of a $(K+1)$-way softmax classifier that tries to classify q as k_+ (the positive key).

Remark. **Why a momentum encoder?** A queue makes the dictionary large but makes back-propagation through the keys intractable (the gradient should flow through every key in the queue). The momentum update sidesteps this: only θ_q is trained by gradients, while θ_k is a slow-moving average.

51.2.1 Comparison of Contrastive Mechanisms

- **(a) End-to-end**: query and key encoders are both updated by back-propagation; the dictionary is limited by mini-batch size.
- **(b) Memory bank**: keys are sampled from a memory bank of past representations. Supports a larger dictionary but the keys come from very different past encoders, hence inconsistent. About 2.6% worse than MoCo.
- **(c) MoCo**: keys are encoded on-the-fly by a momentum-updated encoder and stored in a queue — both large *and* consistent.

51.2.2 MoCo Results

Example 51.3 (ImageNet linear classification, unsupervised pretraining on ImageNet-1M)**.** With ResNet-50 (R50): 60.6% accuracy. Larger backbones (RX50, R50w2×, R50w4×) give 63.9, 65.4, **68.6**%. MoCo also outperforms its supervised pretraining counterpart on 7 detection/segmentation tasks (PASCAL VOC, COCO, ...), sometimes by large margins.

The main hypothesis is confirmed: *good features can be learned by a large dictionary covering a rich set of negative samples, while the key encoder is kept as consistent as possible.* The gap between unsupervised and supervised representation learning has been largely closed in many vision tasks.

Reference

K. He et al. "Momentum contrast for unsupervised visual representation learning". *CVPR 2020.*

51.3 BYOL — Bootstrap Your Own Latent

BYOL revisits and combines ideas from MoCo and SimCLR but **removes negative pairs entirely**.

Architecture:

- Two augmented views $v = t(x)$, $v' = t'(x)$ of the same image.
- An **online network** $(f_\theta, g_\theta, q_\theta)$: representation y_θ, projection z_θ, prediction $q_\theta(z_\theta)$.
- A **target network** (f_ξ, g_ξ): representation y'_ξ, projection z'_ξ. Its parameters ξ are an **exponential moving average** of the online parameters θ (similar in spirit to MoCo's momentum encoder).

Loss: BYOL is **not** contrastive — it *maximises similarity* between the prediction $q_\theta(z_\theta)$ and the (stop-gradient) target projection $\text{sg}(z'_\xi)$ via a **mean-squared-error**. Only the online parameters θ receive gradients; ξ is updated by EMA.

Process overview:

1. Take an unlabelled image and generate two views (which should share the same label).
2. Pass the top view through the online network (with the predictor).
3. Pass the second view through a delayed replica (the target network, no predictor).
4. Compute the MSE between the two latents and train the online network in a supervised fashion.

Key ingredients: image transformations, target network, additional predictor on the online network. *Without the predictor, the representation collapses.*

Takeaways:

- BYOL does **not** use negative examples;
- It is more resilient to transformation choice and batch size than contrastive methods (MoCo / SimCLR).
- At inference time, only f_θ is kept; the rest is discarded.

Example 51.4 (BYOL on ImageNet linear evaluation)**.** With ResNet-50 (and the larger ResNet-200 (4×)) BYOL achieves higher accuracy than state-of-the-art contrastive methods (SimCLR,

MoCo, MoCov2, AMDIM, CMC, CPCv2-L, InfoMin) and approaches the supervised baseline — without using negative pairs.

Example 51.5 (BYOL transfer over 12 datasets)**.** BYOL outperforms SimCLR on all 12 benchmarks and outperforms the Supervised-IN baseline on 7/12 (slightly worse on the remaining 5). The representation transfers well to small images (CIFAR), landscapes (SUN397, VOC2007) and textures (DTD).

Reference

J.-B. Grill et al. "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning". *NeurIPS 2020*.

51.4 Supervised Contrastive Learning (SupCon)

Idea: extend the contrastive framework to leverage class labels when they *are* available, merging the best of supervised and self-supervised contrastive learning.

- In the **self-supervised contrastive loss**, the only positive for an anchor is an augmented copy of the same image; *all* other batch images (regardless of class) are negatives.
- In the **supervised contrastive loss**, *all samples sharing the anchor's class* are positives; only the remaining batch elements are negatives. This produces an embedding space where same-class samples are more tightly clustered.

Two-stage training:

(a) **Supervised cross-entropy** (single stage): final 2048-D representation → 1000-D softmax with label "Dog".

(b) **Self-supervised contrastive** (two stages): Stage 1 contrastive 128-D head; Stage 2 freeze backbone, train classifier.

(c) **Supervised contrastive (SupCon)**: same two-stage pipeline as (b), but the contrastive loss in Stage 1 uses class labels to define positives.

Two formulations of the supervised contrastive loss:

$$\mathcal{L}_{out}^{sup} = \sum_{i\in I} \mathcal{L}_{out,i}^{sup} = \sum_{i\in I} \frac{-1}{|P(i)|} \sum_{p\in P(i)} \log \frac{\exp(z_i \cdot z_p/\tau)}{\sum_{a\in A(i)} \exp(z_i \cdot z_a/\tau)} \tag{17}$$

$$\mathcal{L}_{in}^{sup} = \sum_{i\in I} -\log \left\{ \frac{1}{|P(i)|} \sum_{p\in P(i)} \frac{\exp(z_i \cdot z_p/\tau)}{\sum_{a\in A(i)} \exp(z_i \cdot z_a/\tau)} \right\} \tag{18}$$

where $P(i)$ is the set of positives (same-class samples) and $A(i)$ all other batch elements.

Three benefits of these losses:

1. Generalisation to an arbitrary number of positives;
2. Contrastive power increases with more negatives (large batch size);
3. Intrinsic ability to perform hard positive/negative mining.

Connection with classical losses: triplet loss and N-pair loss are special cases of these formulations. The self-supervised contrastive loss is the degenerate case in which every image has a different label.

51.4.1 SupCon Results

Example 51.6 (Top-1 classification accuracy)**.**

- CIFAR-10: SimCLR 93.6, Cross-Entropy 95.0, Max-Margin 92.4, **SupCon 96.0**.
- CIFAR-100: SimCLR 70.7, Cross-Entropy 75.3, Max-Margin 70.5, **SupCon 76.5**.
- ImageNet: Cross-Entropy 78.2, Max-Margin 78.0, **SupCon 78.7** (ResNet-50).

On ResNet-200 with Stacked RandAugment, SupCon reaches **81.4% top-1** (95.9% top-5), a new SOTA at the time, beating cross-entropy by 0.5–1%. SupCon outperforms cross-entropy with standard augmentations (MixUp, CutMix, AutoAugment, Stacked RandAugment) consistently across architectures (ResNet-50/101/200, ResNeXt-101).

Reference

P. Khosla et al. "Supervised Contrastive Learning". *NeurIPS 2020*.

51.5 SimCLR+ (SimCLRv2) — Big Self-Supervised Models for Semi-Supervised Learning

SimCLR+ is the first systematic attempt to exploit contrastive SSL for **semi-supervised learning**, where only a small fraction (e.g., 1% or 10%) of the training data is labelled. It is a three-step pipeline:

Pretrain, Fine-tune, Distill

1. **Unsupervised pretraining** of a **task-agnostic Big CNN** on all unlabelled data, using SimCLR-style contrastive learning with a deeper projection head.
2. **Supervised fine-tuning** on the small labelled fraction.
3. **Distillation with unlabelled data**: a smaller *task-specific* student is trained on unlabelled data using soft targets from the fine-tuned teacher.

Key empirical findings:

- **Bigger self-supervised models are more label-efficient**: their gain over smaller models is much larger when fine-tuning with very few labels (1%) than with many (100%).
- "*The fewer the labels, the bigger the model.*"
- Increasing model size by 10× reduces the labels needed to reach a given accuracy by roughly 10×.
- **Distillation to smaller task-specific models** is an effective way to avoid serving big models in production.
- A **deeper projection head** $g(\cdot)$ improves not just linear-evaluation representation quality but also semi-supervised fine-tuning performance.

Use of unlabelled data occurs in two complementary ways:

- **Task-agnostic** — learning general visual representations via unsupervised pretraining.
- **Task-specific** — training a student on unlabelled data with imputed labels from the fine-tuned teacher (distillation).

51.5.1 Task-Specific Distillation Loss

Unlabelled case:

$$\mathcal{L}^{\text{distill}} = -\sum_{x_i \in \mathcal{D}} \left[\sum_y P^T(y \mid x_i; \tau) \log P^S(y \mid x_i; \tau) \right] \tag{19}$$

Semi-supervised case (combined fine-tune + distill):

$$\mathcal{L} = -(1-\alpha) \sum_{(x_i, y_i) \in \mathcal{D}^L} [\log P^S(y_i \mid x_i)] - \alpha \sum_{x_i \in \mathcal{D}} \left[\sum_y P^T(y \mid x_i; \tau) \log P^S(y \mid x_i; \tau) \right] \tag{20}$$

The first term is the supervised cross-entropy on the labelled subset $\mathcal{D}^L$, the second is the distillation term on the entire dataset $\mathcal{D}$, weighted by α.

51.5.2 Distillation with Unlabelled Data Improves All Model Sizes

Empirically (ImageNet, label fraction 10%):

- Label only: 12.3 (1%) / 52.0 (10%).
- Label + distillation loss on labelled set: 23.6 / 66.2.
- Label + distillation loss on labelled *and* unlabelled sets: 69.0 / 75.1.
- Distillation loss alone on labelled + unlabelled: 68.9 / 74.3.

Distillation on the labelled dataset alone is not sufficient; using unlabelled examples *largely* improves distillation.

51.5.3 State-of-the-Art on Semi-Supervised ImageNet

Example 51.7 (Semi-supervised ImageNet, label fractions 1% / 10%)**.** SimCLRv2 distilled (ours):

- ResNet-50: top-1 73.9 / 77.5; top-5 91.5 / 93.4.
- ResNet-50 (2× + SK): top-1 75.9 / 80.2; top-5 93.0 / 95.0.
- ResNet-152 (3× + SK), self-distilled: top-1 **76.6** / **80.9**; top-5 **93.4** / **95.5**.

This greatly improves the previous SOTA on ImageNet with 1% or 10% labels, beating earlier task-specific methods (Pseudo-label, VAT+EntMin, Mean Teacher, UDA, FixMatch, MPL, ...) and task-agnostic methods (InstDisc, BigBiGAN, PIRL, CPCv2, SimCLR, BYOL).

Reference

T. Chen et al. "Big Self-Supervised Models are Strong Semi-Supervised Learners". *NeurIPS 2020*.

52 Summary

Take-Home Messages

- Self-supervised learning derives supervision from the data itself via a **pretext task**; the learned features are then transferred to **downstream tasks**.
- Pretext tasks fall into four families: **generation-based** (autoencoder, GAN, colorisation, super-resolution, inpainting), **context-based spatial** (rotations, relative position, jigsaw, clustering), **context-based temporal** (frame-order verification/recognition, arrow of time), and **cross-modal** (audio-visual correspondence/synchronisation, face-voice).
- The choice of pretext task and the depth of the network strongly influence transfer: **multi-task** SSL further closes the gap with supervised pretraining.
- **Contrastive methods** (SimCLR, MoCo) and **non-contrastive methods** (BYOL) have largely closed the gap between unsupervised and supervised representation learning.
- Adding labels back into a contrastive framework (**SupCon**) outperforms cross-entropy on multiple benchmarks.
- In the **semi-supervised** regime, big self-supervised pretraining + supervised fine-tuning + distillation on unlabelled data (**SimCLRv2**) yields strong label-efficiency: bigger models help most when labels are scarce.

Part IX

Semi-Supervised Learning, Pseudo-Labelling and Noisy Labels

53 From Supervised to Semi-Supervised Learning

53.1 Supervised, Unsupervised, Semi-Supervised: A Quick Recap

- **Supervised Learning (SL)** learns from a fully labelled dataset and predicts outcomes for unseen data. Two main task families:
  - **Classification** — predict a discrete class.
  - **Regression** — approximate a mapping f from input variables X to a continuous target y.
- **Unsupervised Learning (UL)** works on a dataset *without* target labels. The model finds structure in the data by extracting features and analysing them. Typical tasks:
  - **Clustering** — find structure or patterns in a collection of points.
  - **Associations** — discover relationships between variables in large databases (e.g., people who buy a new home are likely to buy new furniture).
- **Semi-Supervised Learning (SSL)** sits between the two extremes: it uses a **small labelled** dataset together with a **relatively larger unlabelled** dataset.

Goal of Semi-Supervised Learning

Learn a predictor that performs better on future test data than the predictor learned from the labelled training data alone, by exploiting the unlabelled data.

53.2 Why Semi-Supervised Learning Matters

- In many real-world applications, collecting large *labelled* datasets is too expensive or simply infeasible.
- Large volumes of *unlabelled* data are typically available.
- SSL fits this scenario: it can leverage labels where present and derive structure from the unlabelled data to solve the overall task better.

Geometric intuition. If a model is trained only on the labelled data, the resulting decision boundary may not follow the actual contours of the data distribution suggested by the unlabelled points. The objective of SSL is precisely to use the unlabelled data to produce a decision boundary that better reflects the data's underlying structure — in the comparison of Oliver et al. (2018), Pseudo-Label, Π-model, Entropy Minimisation and VAT all bend the boundary to follow the data, while a purely supervised classifier remains a straight line.

Reference

A. Oliver et al. "Realistic evaluation of deep semi-supervised learning algorithms". `arXiv:1804.09170`, 2018.

54 Pseudo-Labelling: A Naive Semi-Supervised Method

54.1 The Idea (Lee, 2013)

Pseudo-Labelling (PL) is the simplest and most general SSL recipe. The network is trained in a supervised fashion using the labelled and unlabelled data *simultaneously*:

Pseudo-Labelling — Three Steps

1. Train the model on the labelled data only.
2. Use the trained model to predict labels for the unlabelled data.
3. Retrain the model with the pseudo-labelled and the labelled data together.

54.2 Pseudo-Label and Loss Function

For an unlabelled example, the **pseudo-label** is the class with the largest predicted probability:

$$y'_i = \begin{cases} 1 & \text{if } i = \arg\max_{i'} f_{i'}(x) \\ 0 & \text{otherwise} \end{cases} \tag{21}$$

i.e., the predictions are *hard-thresholded* into one-hot pseudo-labels. These are then treated as if they were true labels.

Because the number of labelled and unlabelled examples differs greatly, it is essential to balance their contribution to the loss:

$$L = \frac{1}{n} \sum_{m=1}^{n} \sum_{i=1}^{C} L(y_i^m, f_i^m) + \alpha(t) \, \frac{1}{n'} \sum_{m=1}^{n'} \sum_{i=1}^{C} L(y_i'^m, f_i'^m) \,, \tag{22}$$

or, in plain words:

$$\text{Loss per Batch} = \text{Labeled Loss} + W \cdot \text{Unlabeled Loss}.$$

54.3 The Weight Schedule $\alpha(t)$

The unsupervised weight α controls how much the unlabelled term contributes to the total loss; it is a function of training time (epochs) and is increased *slowly*:

$$\alpha(t) = \begin{cases} 0 & t < T_1 \\ \dfrac{t - T_1}{T_2 - T_1}\,\alpha_f & T_1 \le t < T_2 \\ \alpha_f & T_2 \le t \end{cases} \tag{23}$$

- For the first T_1 epochs (e.g., 100) the weight is 0 — training relies on labels only.
- Between T_1 and T_2 (e.g., 600) the weight *linearly* increases up to α_f (e.g., 3), gradually incorporating unlabelled data.
- After T_2 the weight saturates at α_f. The pair (T_2, α_f) controls the rate of increase and the steady-state value.

54.4 Implementation Outline

After pre-training on labelled data for 100 epochs, an alternation begins:

1. Forward-pass on a batch of unlabelled data; predict pseudo-labels.
2. Compute the unlabelled loss using these pseudo-labels.
3. After 50 unlabelled batches, train one epoch on the labelled data.
4. Repeat until convergence.

This alternation helps in two concrete ways:

- It **reduces overfitting** on the small labelled training set.
- It is faster: only **one** forward pass per batch on the unlabelled data is needed (instead of two as in the original paper formulation).

54.5 Empirical Behaviour: MNIST Example

Setup: 1,000 labelled (class-balanced) + 59,000 unlabelled training images; 10,000 test images.

Example 54.1 (Baseline vs. Pseudo-Labelling on MNIST)**. Supervised baseline** on the 1,000 labelled images:
Epoch 290 – Train Loss 0.00004 – Test Acc **95.57%** – Test Loss 0.233.
Pseudo-labelling (100 supervised + 170 SSL epochs):
Epoch 168 – $\alpha = 3.000$ – Test Acc **98.46%** – Test Loss 0.075.
The unlabelled data brings $\sim 3\%$ absolute accuracy improvement.

The plot of test accuracy vs. epoch shows that as α ramps up linearly, test accuracy slowly increases and saturates — evidence that the unlabelled term is being absorbed without destabilising training.

54.6 T-SNE Inspection of Pseudo-Labels

Visualising the network's features with t-SNE clarifies how PL operates:

- Faint background colours show the *true* clusters (computed from the full labelled set, only for visualisation).
- Small **circles** are the 1,000 labelled points used during the supervised phase.
- Small **stars** are pseudo-labels assigned by the model to unlabelled data at every epoch.

Observations

- Most pseudo-labels are *already correct* at epoch 0 (stars share the colour of their cluster) thanks to the high baseline accuracy.
- As training progresses, the percentage of correct pseudo-labels grows — reflected in the climbing test accuracy (e.g., 94.58% at epoch 0 vs. 98.33% at epoch 140 on the same 750 sampled points).
- Points that move from a wrong to a correct cluster are precisely those near the boundary between two clusters.

55 Why Pseudo-Labelling Works

PL is justified by two classical SSL assumptions:

Two Key Assumptions

- **Continuity (Smoothness) Assumption**: *points close to each other are more likely to share a label.* Small input changes (rotation, shearing, ...) do not change the label.
- **Cluster Assumption**: *the data tend to form discrete clusters, and points in the same cluster are more likely to share a label* — a special case of the smoothness assumption. Equivalently, the decision boundary lies in low-density regions between clusters (cf. maximum-margin classifiers in SVMs).

Consequences for PL:

- The initial labelled data is important: it lets the model learn the underlying cluster structure.
- Each pseudo-label is then assigned by exploiting that cluster structure; as training progresses, the structure itself is refined using the unlabelled data.
- If the initial labelled data is too small or contains outliers, PL will likely assign incorrect labels.
- Conversely, PL benefits from a classifier that already performs reasonably well on the labelled data alone — garbage-in, garbage-out.

56 When Pseudo-Labelling Fails

56.1 Case 1: Initial Labelled Data Insufficient to Determine Clusters

Repeating the MNIST experiment with progressively fewer labels:

- **10 labelled points** (1 per class): the model is no better than random ($\sim 11\%$ accuracy). With one point per class, the underlying class structure cannot be learned.

- **20 labelled points** (2 per class): some structure is captured ($\sim 43\%$ test accuracy). PL assigns correct pseudo-labels mainly when there are two labelled points close-by.
- **50 labelled points** (5 per class): performance improves substantially. Even so, points in the same true cluster but far from the labelled examples are systematically misclassified (e.g., a brown cluster gets mistakenly relabelled as "4" or "7" near its periphery).

Remark. For all of these experiments the *choice* of labelled points matters enormously: outliers can completely change the model and its pseudo-labels — a typical small-data pathology. Both **quantity and quality** of initial labels matter.

T-SNE itself is probabilistic and only gives an idea of the high-dimensional geometry; conclusions drawn from it should be treated as illustrative, not conclusive.

56.2 Case 2: Some Classes Are Missing from the Labelled Set

Setting: class "7" is absent from the labelled set, but present in the unlabelled set.

- After 100 epochs on labelled data only: Test Acc 85.63%, Test Loss 1.555.
- After SSL training: Epoch 99, $\alpha = 2.5$, Test Acc **87.98%**, Test Loss 2.98 — a small improvement that quickly saturates.

The model cannot learn the cluster structure of class "7" because it has never seen any labelled instance of it. PL fundamentally cannot recognise classes not present in the training labels. (Zero-Shot Learning techniques attempt to overcome this, but PL by itself does not.)

56.3 Case 3: No Benefit from Increased Data

In some cases the model lacks the *capacity* to exploit additional data. Conventional ML methods (logistic regression, SVMs) saturate quickly and gain little from more samples, while large deep networks almost always benefit from more data (Andrew Ng's classic *scale drives deep-learning progress* curve).

57 Challenges with Semi-Supervised Learning

57.1 Combining Unlabelled with Labelled Data

The core question: *how to use unlabelled data alongside labelled data?*

- Pseudo-Labelling uses a scheduled weight $\alpha(t)$ to slowly mix in the unlabelled loss. This implicitly assumes that model confidence increases over time and therefore that the unlabelled loss should grow linearly with t.
- This need not be the case: predictions can be wrong, and many wrong unlabelled predictions cause PL to act like a **bad feedback loop**, deteriorating performance further (*confirmation bias*, Arazo et al. 2019).
- One mitigation: use **probability thresholds** to admit only confident pseudo-labels (analogous to logistic regression).
- Other algorithms combine the two sources differently. For example, **MixMatch** (Berthelot et al. 2019) uses a 2-step process: first guess a label for unlabelled data, then perform Mixup augmentation between labelled and unlabelled samples.

References

- E. Arazo et al. "Pseudo-labeling and confirmation bias in deep semi-supervised learning". *IJCNN 2020*.
- D. Berthelot et al. "MixMatch: A holistic approach to semi-supervised learning". *NeurIPS 2019*.

57.2 Data Efficiency

A second challenge is to design SSL algorithms that work with very small amounts of labelled data. PL works best with $\sim$ 1,000 labelled samples; performance drops when the labelled set is shrunk further (e.g., 50 points). On the classical *two-moons* dataset, Oliver et al. (2018) show that:

- **VAT** (Miyato et al. 2019) and **Π-model** (Laine & Aila 2017) learn a near-perfect decision boundary with just 6 labelled points.
- **Pseudo-Labelling** fails completely: it learns a *linear* decision boundary instead. Reproducing the same model, PL needed 30–50 labelled points to recover the data structure.

References

- T. Miyato et al. "Virtual adversarial training: a regularization method for supervised and semi-supervised learning". *TPAMI 2019*.
- L. Samuli, T. Aila. "Temporal ensembling for semi-supervised learning". *ICLR 2017*.

57.3 Pseudo-Labelling: Conclusions

- PL is a simple heuristic, widely used in practice because of its *simplicity and generality*.
- Recent SSL algorithms have driven image-classification performance to surprising levels with very few labels: e.g., **Unsupervised Data Augmentation** (Xie et al. 2020) reaches **97.3%** on CIFAR-10 with just 4,000 labels, while DenseNet (Huang et al. 2017) achieved 96.54% on the *full* CIFAR-10 in 2017.
- ML and DS communities are increasingly moving towards approaches that need fewer labels (semi-supervised, zero/few-shot) or smaller datasets (transfer learning).

References

- Q. Xie et al. "Unsupervised data augmentation for consistency training". *NeurIPS 2020*.
- G. Huang et al. "Densely connected convolutional networks". *CVPR 2017*.

58 Ensembling Methods for Semi-Supervised Learning

Self-ensembling (Laine & Aila, ICLR 2017) is a simple, efficient way to train DNNs in an SSL setting. The idea: form a **consensus prediction of the unknown labels** using the outputs of the network at *different epochs*.

- Training operates on a single network; predictions on different epochs effectively ensemble many sub-networks (because of dropout regularisation).
- This ensemble is a better predictor than any single epoch's output.
- These predictions can therefore serve as training targets for unlabelled inputs.

- The method relies heavily on **dropout regularisation** and **versatile input augmentation**.

Two implementations are described: the **Π-model** and **temporal ensembling**.

58.1 The Π-Model

The Π-model encourages **consistent network output between two realisations** of the same input under *different dropout conditions* and augmentations.

Procedure:

- Each input x_i is evaluated twice, producing prediction vectors z_i and $\tilde{z}_i$. Differences arise from stochastic dropout, Gaussian noise, and random augmentations.
- The loss has two components:
  - **Cross-entropy** on the labelled inputs only.
  - **Mean-square difference** $\|z_i - \tilde{z}_i\|^2$ on *all* inputs.
- The two terms are combined via a time-dependent weight $w(t)$.
- Comparing entire output vectors — not only argmax decisions — asks for the *dark knowledge* between the two evaluations to be close, a stronger requirement than "same final classification".

Algorithm 1: Π-Model

```
Require: x_i = training stimuli
Require: L = indices of labelled inputs
Require: y_i = labels for i in L
Require: w(t) = unsupervised weight ramp-up function
Require: f_theta(x) = stochastic NN with parameters theta
Require: g(x) = stochastic input augmentation
for t in [1, num_epochs]:
    for each minibatch B:
        z_{i in B}    <- f_theta(g(x_{i in B}))    # forward pass 1
        ztilde_{i in B} <- f_theta(g(x_{i in B}))  # forward pass 2 (different dropout/au
        loss <- - (1/|B|) * sum_{i in B cap L} log z_i[y_i]      # supervised
                + w(t) * (1/(C|B|)) * sum_{i in B} ||z_i - ztilde_i||^2   # unsupervised
        update theta using e.g. ADAM
return theta
```

Important detail: $w(t)$ ramps up from zero along a Gaussian curve during the first $\sim$ 80 epochs. Initially the supervised term dominates; the unsupervised term grows in slowly. *The slow ramp-up is critical*: otherwise the network can get stuck in a degenerate solution with no meaningful classification.

58.2 Temporal Ensembling

Analysing Π-model: the two evaluations could be split into (i) classifying the training set once without updating θ, then (ii) training on the same inputs under different augmentations / dropout, using the previous predictions as targets for the unsupervised loss. Targets obtained from a single evaluation are noisy.

Temporal ensembling alleviates this by aggregating predictions of multiple previous network evaluations into an ensemble prediction. After every epoch, the network outputs z_i are accumulated:

$$Z_i \leftarrow \alpha\, Z_i + (1-\alpha)\, z_i, \tag{24}$$

where α is a momentum coefficient that controls how far back into history the ensemble reaches. Because of dropout and stochastic augmentation, Z is a weighted average of an ensemble of past networks f, with recent epochs weighted more heavily than distant ones.

The training targets $\widetilde{z}_i$ are corrected for startup bias by dividing by $(1-\alpha^t)$:

$$\widetilde{z}_i = Z_i/(1-\alpha^t).$$

On the first epoch, Z and $\widetilde{z}_i$ are zero, so $w(t)$ is also set to zero. As before, $w(t)$ ramps up via a Gaussian curve $\exp[-5(1-T)^2]$ in the first 80 epochs, with T advancing linearly from 0 to 1.

Algorithm 2: Temporal Ensembling

```
Require: x_i, L, y_i for i in L, alpha (ensembling momentum), w(t),
         f_theta, g
Z       <- 0_{N x C}             # ensemble predictions
ztilde  <- 0_{N x C}             # target vectors
for t in [1, num_epochs]:
    for each minibatch B:
        z_{i in B} <- f_theta(g(x_{i in B}, t))
        loss <- - (1/|B|) sum_{i in B cap L} log z_i[y_i]
                + w(t) * (1/(C|B|)) sum_{i in B} ||z_i - ztilde_i||^2
        update theta using e.g. ADAM
    Z      <- alpha * Z + (1 - alpha) * z
    ztilde <- Z / (1 - alpha^t)
return theta
```

58.3 Tolerance to Incorrect Labels

A striking property of temporal ensembling is its **robustness to label noise**: on SVHN with progressively randomised labels (0%, 20%, 50%, 80%, 90%):

- Standard supervised training: accuracy collapses very quickly — even 20% of randomised labels is enough to degrade test accuracy substantially; with 50% or more, accuracy drops to near-random.
- Temporal ensembling: *nearly perfect resistance* to disinformation when 50% of labels are random; over 90% accuracy retained when 80% of labels are random.

58.4 Results: Π-Model and Temporal Ensembling

Example 58.1 (CIFAR-10, 4,000 / all (50,000) labels)**.** Supervised-only with augmentation: 34.85 ± 1.65 / 6.05 ± 0.15.
Π-model: 16.55 ± 0.29 / —.
Π-model with augmentation: $\mathbf{12.36 \pm 0.31}$ / $\mathbf{5.56 \pm 0.10}$.
Temporal ensembling with augmentation: $\mathbf{12.16 \pm 0.24}$ / 5.60 ± 0.10.

Example 58.2 (SVHN, 500 / 1000 / all (73,257) labels)**.** Supervised-only with augmentation: 31.59 ± 3.60 / 19.30 ± 3.89 / 2.88 ± 0.03.
Π-model with augmentation: 6.65 ± 0.53 / 4.82 ± 0.17 / $\mathbf{2.54 \pm 0.04}$.
Temporal ensembling with augmentation: $\mathbf{5.12 \pm 0.13}$ / $\mathbf{4.42 \pm 0.16}$ / 2.74 ± 0.06.

58.5 Conclusions on Π-Model vs. Temporal Ensembling

- Temporal ensembling has two advantages: training is faster (the network is evaluated only once per input per epoch) and targets $\tilde{z}$ are less noisy.
- At each training step, the **Exponential Moving Average** (EMA) prediction of every example in the mini-batch is updated; the EMA prediction is an ensemble of the model's current and earlier versions.
- Using these EMA predictions as **teacher** targets improves results, but each target is updated only once per epoch — information is incorporated slowly.
- Drawbacks: temporal ensembling needs auxiliary storage (the matrix Z, possibly memory-mapped), plus a new hyperparameter α.

Reference

S. Laine, T. Aila. "Temporal ensembling for semi-supervised learning". *ICLR 2017*.

59 Mean Teacher

Mean Teacher (Tarvainen & Valpola, NIPS 2017) takes the temporal-ensembling intuition one step further: instead of averaging *label predictions*, average the *model weights*.

- Two networks: a **student** (trained by gradient descent) and a **teacher** (whose weights are an EMA of the student's weights).
- The teacher is the *average of consecutive student models* — hence the name.
- Averaging model weights over training steps tends to produce a more accurate model than using the final weights directly.
- Unlike temporal ensembling — which shares weights and updates targets only once per epoch — the teacher uses EMA weights of the student and can aggregate information *after every step*.
- Because the weight averages improve all layers (not just the output), the teacher has better intermediate representations.
- Mean Teacher requires fewer labels than Temporal Ensembling: without changing the architecture, it reaches **4.35%** error on SVHN with just **250 labels** — outperforming Temporal Ensembling trained with 1,000 labels.

59.1 Intuition: True Label, Teacher Label, Virtuous Cycle

- **Supervised case**: the true label *pulls* the model's prediction toward itself. For a labelled image of a dog, the prediction simplex is moved towards the "dog" vertex.
- **Unlabelled case**: there is no true label. We need *two* predictions — a student and a teacher — so that the student can learn from the teacher.
- Two non-exclusive ways to make this work:
  1. **Make the task harder**: distort the student's input so its task is more challenging; force the student to predict the easier task's output (i.e., the undistorted input's prediction).
  2. **Make the teacher better**: maintain an EMA of weights to create a stronger teacher (a *mean* teacher); let the student learn the teacher's predictions.

- Student and teacher therefore improve each other in a **virtuous cycle**: better student ⇒ better EMA ⇒ better teacher ⇒ better targets ⇒ better student.

59.2 Proposed Method

Conceptually:

1. Take a supervised model with parameters θ.
2. Make a copy of it with parameters θ' (the teacher).
3. At training step t, update the teacher weights as the EMA of the student weights:

$$\theta'_t = \alpha\,\theta'_{t-1} + (1-\alpha)\,\theta_t, \tag{25}$$

   where α is a smoothing-coefficient hyperparameter.

The **consistency cost** J is the expected distance between the student prediction (weights θ, noise η) and the teacher prediction (weights θ', noise η'):

$$J(\theta) = \mathbb{E}_{x,\eta',\eta}\left[\left\|f(x,\theta',\eta') - f(x,\theta,\eta)\right\|^2\right]. \tag{26}$$

Total loss = classification cost (on labelled data) + consistency cost (on all data).

Reference

A. Tarvainen, H. Valpola. "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results". *NIPS 2017*.

60 Dealing with Noisy Labels

60.1 The Problem

Real-world datasets often contain mislabelled examples. **Standard regularisation** (data augmentation, weight decay, dropout, batch normalisation) helps but is not sufficient.

Example 60.1 (Convergence under symmetric 40% noise (CIFAR-100, WideResNet-16-8))**.** Three regimes are compared: *Noisy w/o. Reg.*, *Noisy w. Reg.*, *Clean w. Reg.*.

- Noisy w/o. Reg. overfits aggressively (training accuracy near 100%, test accuracy below 25%).
- Adding regularisation to noisy data improves test accuracy somewhat (around 50%) but a large **gap** to the Clean+Reg. model (around 75%) remains.

Reference

H. Song et al. "Learning from noisy labels with deep neural networks: A survey". `arXiv:2007.08199`, 2020.

60.2 Co-Teaching: Robust Training under Extremely Noisy Labels

Co-teaching (Han et al., NeurIPS 2018) trains **two** networks simultaneously and lets them *teach each other* on every mini-batch:

1. Each network forwards all data and selects a small subset of *possibly clean* examples (those with the lowest loss).
2. The two networks **communicate** which examples should be used for training.

3. Each network back-propagates on the data selected by its *peer* and updates itself.

Because the two networks have different learning abilities (different decision boundaries / abilities to filter noise), they filter different types of error introduced by the noisy labels — error flow through the system is reduced mutually.

60.2.1 Algorithm

Algorithm 1: Co-Teaching

```
Input: w_f, w_g, learning rate eta, fixed tau, T_k, T_max, N_max
for T = 1, 2, ..., T_max:
    Shuffle training set D                    # noisy dataset
    for N = 1, ..., N_max:
        Fetch mini-batch \bar{D} from D
        Obtain \bar{D}_f = argmin_{D' : |D'| >= R(T)|\bar{D}|} l(f, D')   # f selects sma
        Obtain \bar{D}_g = argmin_{D' : |D'| >= R(T)|\bar{D}|} l(g, D')   # g selects sma
        Update w_f <- w_f - eta * grad l(f, \bar{D}_g)   # update f using g's selection
        Update w_g <- w_g - eta * grad l(g, \bar{D}_f)   # update g using f's selection
    Update R(T) = 1 - min{ T/T_k * tau, tau }
Output: w_f, w_g
```

Two design questions and their answers:

Q1: Why select small-loss instances with dynamic $R(T)$?

When labels are correct, small-loss instances are more likely to be the cleanly-labelled ones. Training only on small-loss instances is therefore noise-resistant.
The **memorisation effect** of DNNs helps: even on noisy data, networks learn the clean and easy patterns first, in the early epochs — so loss values at the beginning of training filter out noisy instances. With more epochs, however, the networks eventually overfit to noisy labels.
Solution: keep many instances early (large $R(T)$), then gradually *decrease* the keep-rate so that, once the network risks memorising noise, only the most likely-clean small-loss instances remain.

Q2: Why two networks and cross-update parameters?

A few wrong instances can deteriorate the whole model if a single network filters its own data.
Different classifiers learn different decision boundaries and therefore filter different types of label noise.
By **exchanging** the small-loss instances between f and g, the networks adaptively correct each other's training error if the selection is not fully clean. The error from one network is not directly fed back to itself, so co-teaching can deal with heavier noise than self-evolving (single-network) approaches.
The intuition is that of *peer review*: it is hard to find your own errors due to personal bias, but a peer can spot them more easily.

60.2.2 Results on CIFAR-10

Comparing Co-teaching with Standard, Bootstrap, S-model, F-correction, Decoupling and MentorNet under three noise patterns:

- **Pair-45%**: Co-teaching maintains $\sim 70\%$ test accuracy and rapidly raises label precision to $\sim 85\%$, while all other methods collapse below 55%.
- **Symmetry-50%**: Co-teaching stays at $\sim 75\%$ accuracy; competing methods drop and oscillate. Label precision climbs above 90%.
- **Symmetry-20%**: Co-teaching is competitive with the best methods (around 80–85%) and reaches the highest label precision ($\sim 95\%$).

Reference

B. Han et al. "Co-teaching: Robust training of deep neural networks with extremely noisy labels". *NeurIPS 2018*.

61 Summary

Take-Home Messages

- Semi-supervised learning sits between SL and UL and exploits both labelled and unlabelled data to learn a better predictor.
- **Pseudo-Labelling** is the simplest SSL recipe: train on labels, predict pseudo-labels, retrain. A scheduled weight $\alpha(t)$ slowly mixes in the unlabelled loss.
- PL relies on the **continuity** and **cluster** assumptions; it fails when initial labels are too few/biased, when classes are missing from the labelled set, or when the model lacks capacity to exploit extra data.
- Confirmation bias and data efficiency are core SSL challenges; **thresholds** and methods such as **MixMatch** or **UDA** mitigate them.
- **Self-ensembling** via the **Π-model** and **Temporal Ensembling** uses consistency between augmented/dropout-perturbed predictions and an EMA of past outputs as a regulariser. Temporal ensembling is also remarkably tolerant to noisy labels.
- **Mean Teacher** averages *weights* (not predictions), enabling per-step updates and stronger intermediate representations — much more label-efficient than Temporal Ensembling.
- For **noisy labels**, regularisation alone is insufficient. **Co-teaching** uses two networks that exchange small-loss (likely-clean) examples each mini-batch, exploiting the memorisation effect with a decreasing keep-rate $R(T)$ to handle even extreme noise levels.

Part X

Transfer Learning and Unsupervised Domain Adaptation

62 Motivation: Why Visual Models Break

62.1 The "Is There a Bird?" Story

A classical xkcd comic draws the line between trivial and almost-impossible computer-vision problems: *is the user in a national park?* is a quick GIS lookup; *is there a bird in the photo?* used to require a research team and years of work. Modern CNNs can answer the second question

well on **photos of birds**, regardless of species or scene context (real photos: *flamingo, cardinal, robin, owl, ...*).

The trouble starts when the visual style changes:

- **BAM (Behance Artistic Media)** paintings of birds: networks trained on natural photos become unsure — the silhouettes, colour palette and texture statistics differ.
- **Pen-and-ink drawings** of the same birds: performance collapses — the network has never seen this rendering style.

Real-World Example: COVID-19 Lung Ultrasound

Roy et al. (IEEE TMI 2020) train a CNN to predict COVID-19 markers on lung-ultrasound clips collected from several Italian hospitals (different sensors).

- Train and test on *the same hospital*: $\geq 85\%$ accuracy.
- Test on *a different hospital*: drop of up to 20%.
- Annotated data from every hospital is rarely available — a textbook motivation for transfer learning and domain adaptation.

62.2 Domain Shift

Definition 62.1 (Domain Shift)**. Visual appearance changes** between training and test data **degrade the performance of recognition systems**. The problem is well-known: a model that achieves $\sim 85\%$ on a CIFAR-10-style natural-image test set drops to $\sim 55\%$ on the same classes rendered as pencil sketches (Kulis, Saenko, Darrell, CVPR 2011).

62.3 Why Should We Care?

Domain shift hurts every system that operates outside its training distribution:

- **Self-driving cars and drones** must work *regardless* of:
  - **Seasonal/time changes** (winter vs. summer, day vs. night).
  - **Illumination conditions** (Hwang et al. multispectral pedestrian detection benchmark).
  - **Different sensors / modalities** (RGB vs. thermal infrared).
- **Costly or infeasible data collection**, e.g. rescue robots in earthquake-damaged buildings; the deployment environment is impossible to capture in advance.
- **Synthetic data** (SYNTHIA, GTA-V) is cheap to render but suffers a sim-to-real gap.

62.4 Domain Shift Is Ubiquitous

Common shift archetypes:

- **Time & environmental changes** (same scene at day/night).
- **Different modalities** (RGB $\rightarrow$ thermal).
- **Synthetic to real** images (game engines $\rightarrow$ real urban driving).
- **CAD to real** (3D models of chairs $\rightarrow$ real photographs).

62.5 Domain Shift and Dataset Bias

Torralba & Efros (CVPR 2011): "Unbiased look at dataset bias". Many object-recognition datasets contain a strong, characteristic "signature" — humans (and classifiers) can guess which dataset an image came from with surprising accuracy. PASCAL cars, SUN cars, Caltech-101 cars, ImageNet cars, and LabelMe cars look different in resolution, framing, colour and pose distribution. Models implicitly memorise these biases.

62.6 How Do We Solve It?

Annotate target data? A bottomless pit of effort:

- **Data collection is costly**: ImageNet has $\sim$ 14M hand-annotated images, only $\sim$ 1M with bounding boxes, more than 20,000 categories (Deng et al., CVPR 2009).
- **We cannot annotate everything**: every domain wants its own ImageNet (SpaceNet, MusicNet, Medical ImageNet, ShapeNet, EventNet, ActivityNet, ...).
- **Sometimes data collection is impossible** (rescue scenarios after a disaster).

The way out is to **transfer knowledge** learned on data we already have.

63 Transfer Learning

63.1 Traditional ML vs. Transfer Learning

In traditional machine learning, every task has its own **learning system** trained on its own data. In transfer learning, the system trained on *source tasks* produces **knowledge** that is reused to learn the *target task* more efficiently or with less data (Pan & Yang, IEEE TKDE 2009).

63.2 Definitions

Definition 63.1 (Domain)**.** A **domain** $\mathcal{D}$ is a two-element tuple

$$\mathcal{D} = \{\mathcal{X},\, P(X)\}$$

consisting of an **image/feature space** $\mathcal{X}$ and the **marginal probability** $P(X)$ over data points $X \in \mathcal{X}$.

Definition 63.2 (Task)**.** Given a specific domain $\mathcal{D} = \{\mathcal{X}, P(X)\}$, a **task** $\mathcal{T}$ has two components:

$$\mathcal{T} = \{\mathcal{Y},\, f(\cdot)\}$$

- A **label space** $\mathcal{Y}$.
- An **objective predictive function** $f(\cdot)$, not directly observed but learnable from training pairs $\{x_i, y_i\}$ with $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$. Equivalently, $\mathcal{T} = \{\mathcal{Y},\, P(Y \mid X)\}$.

Objective of Transfer Learning

Given a source domain $\mathcal{D}_S$, a corresponding source task $\mathcal{T}_S$, a target domain $\mathcal{D}_T$ and a target task $\mathcal{T}_T$, the goal is to learn the target conditional probability $P(Y_T \mid X_T)$ in $\mathcal{D}_T$ using the information gained from $\mathcal{D}_S$ and $\mathcal{T}_S$, where $\mathcal{D}_S \neq \mathcal{D}_T$ *or* $\mathcal{T}_S \neq \mathcal{T}_T$.

Typical assumption: only a *limited* number of labelled target samples is available — much smaller than the source — or only *unlabelled* target samples.

63.3 Four Scenarios

Four ways for source and target to differ:

1. $\mathcal{X}_S \neq \mathcal{X}_T$ — the **feature spaces** differ (e.g., documents in two different languages).
2. $P(X_S) \neq P(X_T)$ — the **marginal distributions** differ. **This is Domain Adaptation**.
3. $\mathcal{Y}_S \neq \mathcal{Y}_T$ — the **label spaces** differ (e.g., source labels: African elephant, wall clock, green snake, Yorkshire terrier; target labels: chair, background, person, TV/monitor).
4. $P(Y_S \mid X_S) \neq P(Y_T \mid X_T)$ — the **conditional distributions** differ (e.g., classes are unbalanced differently in the two corpora).

A canonical *different marginal* example: **MNIST** (handwritten digits, white-on-black, centred) vs. **SVHN** (Street View house numbers, colourful, cluttered).

63.4 The Three Key Aspects

What, When, How to Transfer

- **What to transfer?** Identify which part of the knowledge can be ported from source to target. Understand what is domain/task-specific and what is shared.
- **When to transfer?** Performance on the target task should *improve*, never degrade. Avoid **negative transfer**: source performance should also be preserved.
- **How to transfer?** Identify algorithmic solutions for moving the knowledge across domains/tasks.

64 Domain Adaptation

64.1 Notation

- $\mathcal{X}$ — input space; $\mathcal{Y}$ — output space.
- $X \in \mathcal{X}$ — input variable (image); $Y \in \mathcal{Y}$ — output variable (label).
- $D = \{\mathcal{X}, P(X)\}$ — a **domain**.
- $T = \{\mathcal{Y}, P(Y \mid X)\}$ — a **task**.

64.2 The DA Problem

Source: $D^s = \{\mathcal{X}^s, P(X^s)\}$, $T^s = \{\mathcal{Y}^s, P(Y^s \mid X^s)\}$.
Target: $D^t = \{\mathcal{X}^t, P(X^t)\}$, $T^t = \{\mathcal{Y}^t, P(Y^t \mid X^t)\}$.

Domain Adaptation

$$D^s \neq D^t, \quad T^s = T^t.$$

That is, $\mathcal{X}^s \neq \mathcal{X}^t$ *or* $P(X^s) \neq P(X^t)$, while $\mathcal{Y}^s = \mathcal{Y}^t$ (the same label space and the same conditional). The marginal input distributions differ, but the task is the same.

64.3 Unsupervised DA

Most practical setting: a **labelled source** domain (e.g., real images of *dog/elephant/house*) and an **unlabelled target** domain (e.g., cartoons of the same classes).

- $P(X^s) \neq P(X^t)$ — marginal distributions differ.
- $\mathcal{Y}^s = \mathcal{Y}^t$ — shared label set.

64.4 Covariate Shift Assumption

A canonical 1D illustration: training samples concentrate on the left of the input axis, test samples on the right. The *true function* (red) is non-linear; a model fit on training samples (green linear regressor) extrapolates badly. The conditional $P(Y \mid X)$ is the same for train and test, but the marginal $P(X)$ has shifted — exactly the assumption underlying most DA algorithms.

65 DA Scenarios

65.1 Label-Space Variants

- **Closed Set DA**: $\mathcal{Y}^s = \mathcal{Y}^t$. Both source and target share *exactly* the same set of classes (e.g., {dog, elephant, house} on both sides).
- **Partial DA**: $\mathcal{Y}^t \subset \mathcal{Y}^s$. The target uses only a subset of the source classes.
- **Open Set DA**: target contains *additional* unknown classes not present in the source. Some source classes may also be missing from the target.

65.2 Without Target Data: Domain Generalization

Sometimes target data is not available at training time at all. **Domain Generalization (DG)** addresses exactly this:

- **Single-source DG**: train on one labelled source domain, generalise to an unseen target.
- **Multi-source DG**: train on several labelled source domains, generalise to an unseen target. The intuition is that observing several sources teaches the model what is *domain-invariant* vs. what is domain-specific.

66 Benchmarks

66.1 Office31

A small but historically important DA benchmark (Saenko et al., ECCV 2010; Gong et al., CVPR 2012): three domains (**Amazon**, **DSLR**, **Webcam**) with 31 office-object classes; later extended with **Caltech-256**.

66.2 Office-Home, VisDA, DomainNet

DomainNet (VisDA Challenge 2019) is the largest cross-domain classification benchmark: 6 domains (*infograph, clipart, painting, real, quickdraw, sketch*), over 280K images across 12+ categories in the combined train/val/test splits; over 30K images and 18 categories for image segmentation.

66.3 Synthetic-to-Real Benchmarks

VisDA-2018 also targets a *synthetic source* $\rightarrow$ *real target* setting: large gap in appearance, smaller gap in spatial layout. Used for cross-domain object detection and semantic segmentation (e.g., GTA-V $\rightarrow$ Cityscapes for driving scenes).

67 Deep Domain Adaptation: Three Families

Following Csurka's survey (arXiv:1702.05374, 2017), deep DA methods split into three categories:

- **Discrepancy-based**: fine-tune the deep network with labelled or unlabelled target data to *reduce a measured distance* between source and target feature distributions.
- **Adversarial-based**: use a *domain discriminator* to encourage *domain confusion* via an adversarial objective.
- **Reconstruction-based**: use data *reconstruction* as an auxiliary task to enforce feature invariance.

67.1 Discrepancy: Maximum Mean Discrepancy (MMD)

MMD measures the distance between the embeddings of two distributions in a Reproducing Kernel Hilbert Space (Gretton et al., NIPS 2007). For source samples $\{x_i^s\}_{i=1}^{n^s}$ and target samples $\{x_i^t\}_{i=1}^{n^t}$:

$$\mathrm{MMD}^2 = \left\| \frac{1}{n^s} \sum_{i=1}^{n^s} \phi(x_i^s) - \frac{1}{n^t} \sum_{i=1}^{n^t} \phi(x_i^t) \right\|^2 . \tag{27}$$

Idea: in a deep network, the *bottom* layers capture general patterns whereas the *top* layers are tailored to the classification task and biased towards the source domain. Re-map the feature representations into a common embedding so this bias is removed; the re-mapping is induced by an MMD distance over a kernel function, applied at *multiple* layers (typically the higher fully-connected layers).

Reference

A. Gretton et al. "A kernel method for the two-sample problem". *NIPS 2007*.

67.2 Discrepancy: CORAL — Correlation Alignment

CORAL (Sun et al., 2017) aligns the **second-order statistics** of source and target features. It *recolours* the source covariance into the target covariance: starting from a strongly anisotropic source covariance and a different target covariance, two whitening + recolouring operations rotate and scale the source feature distribution until it matches the target's covariance shape. Intuitively, the source point cloud is reshaped from a tilted ellipsoid to one that lies in the same orientation as the target ellipsoid, so a classifier trained on the aligned features transfers better.

Reference

B. Sun et al. "Correlation alignment for unsupervised domain adaptation". *Domain Adaptation in Computer Vision Applications*, 2017.

67.3 CNNs and Universal Representations

A core empirical observation behind deep DA: **CNN features generalise much better than hand-crafted features**. Donahue et al. (ICML 2014) showed that DeCAF_6 activations cluster the same Office31 categories far more cleanly than SURF features — the same images that lie close in feature space across domains belong to the same semantic class.

68 Adversarial Domain Adaptation

68.1 The Domain-Classification Game

Consider two image collections: real photos vs. cartoons, both showing dogs, elephants and houses. A **linear classifier in pixel space** can easily separate the *real* from the *cartoon* images. If the same separation persists in feature space, the network has not learned a domain-invariant representation.

Goal: produce a feature space in which a domain classifier *cannot* tell source and target apart — effectively, the two coloured point clouds become indistinguishable.

68.2 DANN: Domain-Adversarial Training of Neural Networks

DANN (Ganin et al., JMLR 2016) plugs a **domain classifier** into a standard CNN and trains it adversarially.

Architecture:

- A **feature extractor** $G_f(\cdot;\theta_f)$ produces features $\mathbf{f} = G_f(x)$.
- A **label predictor** $G_y(\cdot;\theta_y)$ outputs the class label y with classification loss L_y.
- A **domain classifier** $G_d(\cdot;\theta_d)$ predicts the binary domain label $d \in \{\text{source}, \text{target}\}$ with loss L_d.

The **key trick** is the **Gradient Reversal Layer** (GRL) inserted between G_f and G_d:

- During the **forward pass**, the GRL is the identity.
- During the **backward pass**, it multiplies the gradient by $-\lambda$.

Therefore θ_y is updated to *minimise* the label loss; θ_d is updated to *minimise* the domain loss; and θ_f is updated to *minimise* the label loss *and maximise* the domain loss. The feature extractor is pushed to produce features that are simultaneously **discriminative for the class label** and **indistinguishable across domains**.

Example 68.1 (MNIST → MNIST-M, top feature-extractor layer)**.** Without adaptation, source (red) and target (blue) features form clearly separate clusters. After domain-adversarial training, the two distributions overlap, and class clusters survive — the network has built a domain-invariant feature space.

Reference

Y. Ganin et al. "Domain-adversarial training of neural networks". *JMLR*, 17(1):2096–2030, 2016.

68.3 ADDA — Adversarial Discriminative Domain Adaptation

ADDA (Tzeng et al., CVPR 2017) decouples the source and target encoders.

Three-stage pipeline:

1. **Pre-training**: train a **source CNN** + classifier on labelled source images with cross-entropy.
2. **Adversarial adaptation**: *freeze* the source CNN; train a **target CNN** (initialised from the source) so that a **discriminator** cannot tell apart features extracted from source and target images.

3. **Testing**: feed target images to the trained target CNN and reuse the source classifier (which works because target features have been pushed to look like source features).

The pictorial summary: in feature space, the **target encoder** pulls target features (blue) towards the cluster of source features (red), driven by the gradient of the **domain discriminator**.

Reference

E. Tzeng, J. Hoffman, K. Saenko, T. Darrell. "Adversarial discriminative domain adaptation". *CVPR 2017*.

68.4 GANs as a Building Block

Adversarial DA borrows the **GAN** machinery: a **generator** G tries to produce samples that fool a **discriminator** D, while D tries to distinguish real from fake. The minimax objective is

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]. \tag{28}$$

In adversarial DA, the same recipe is reused with a *domain* discriminator instead of a real/fake one.

68.5 DIFA: Domain Invariant Feature Augmentation

DIFA (Volpi, Morerio, Murino, Sangineto. CVPR 2018) extends ADDA with **feature augmentation**.

Three-step pipeline:

1. **Step 0**: train a deep classifier E_S, C on the source domain with cross-entropy loss.
2. **Step 1**: *adversarially* train a **sampler** S to learn the source feature distribution. With the source encoder E_S frozen, S takes noise $z \sim \Pi_{[-1,1]}$ and conditioning labels and is trained against a discriminator D_1 to produce features indistinguishable from E_S's outputs — effectively an *augmented feature space*.
3. **Step 2**: *adversarially* train a new **shared encoder** E_I to match source and target distributions. S is now frozen and used to generate source-like features; the discriminator D_2 pushes the target encoder to produce features that lie in the same distribution.

DIFA improves ADDA by:

1. Sampling source features to perform **augmentation**;
2. Making the encoder **domain-invariant** by anchoring it to the source distribution.

Reference

R. Volpi, P. Morerio, V. Murino. "Adversarial Feature Augmentation for Unsupervised Domain Adaptation". *CVPR 2018*.

68.6 Generalising ADDA-Style Methods

ADDA-style frameworks share a common skeleton (Tzeng et al., 2017):

- Source input → **source mapping** → source discriminator.
- Target input → **target mapping** → target discriminator.
- A shared **classifier** reads features from the target mapping at test time.

Three design questions individuate concrete methods:

1. **Generative or discriminative model?** (Are features generated from noise, or only adversarially aligned?)
2. **Weights tied or untied?** (Source and target encoders share parameters? or are they separate?)
3. **Which adversarial objective?** (Standard GAN, gradient reversal, Wasserstein, ...)

These three choices recover most of the methods in the deep-DA state of the art (Long et al. ICML 2015, Ghifary et al. ECCV 2016, Ganin et al. JMLR 2016, Tzeng et al. CVPR 2017, Rozantsev et al. TPAMI 2019, Bousmalis et al. NIPS 2016 / CVPR 2017, Volpi et al. CVPR 2018, Carlucci et al. CVPR 2018, ...).

69 Summary

Take-Home Messages

- **Domain shift** — visual appearance differences between training and test data — causes systematic accuracy drops in real-world deployments (sketches vs. photos, day vs. night, sensor changes, hospital-to-hospital, sim-to-real).
- Annotating *every* target domain is impossible: there will never be "the ImageNet of X" for every X. Hence the need for **transfer learning**.
- A **domain** is $\{\mathcal{X}, P(X)\}$ and a **task** is $\{\mathcal{Y}, P(Y \mid X)\}$. Transfer learning learns $P(Y_T \mid X_T)$ in the target by reusing knowledge from a source domain/task.
- The four scenarios cover different feature spaces, different marginals (**DA**), different label spaces and different conditionals.
- **Domain Adaptation**: $D^s \neq D^t$ but $T^s = T^t$. The *unsupervised* setting (labelled source + unlabelled target) is the most studied.
- DA scenarios further split by label-space relations (**closed/partial/open set**) and by data availability (**single-** and **multi-source domain generalization** when no target data is accessible).
- Deep DA methods cluster into three families: **discrepancy-based** (MMD, CORAL), **adversarial-based** (DANN with gradient reversal, ADDA, DIFA), **reconstruction-based** (autoencoder-style auxiliary tasks).
- The adversarial route reduces DA to a domain-classification game: align source and target so that no domain classifier can tell them apart, while preserving class discriminativeness.

Part XI

Generative Models

70 From Supervised to Generative Learning

70.1 Supervised vs. Unsupervised

Recall the two basic regimes:

- **Supervised learning**. Data $(\mathbf{x}, y)$, goal: learn $f : \mathbf{x} \mapsto y$. Examples: classification, regression, detection, segmentation, captioning.

- **Unsupervised learning**. Data $\mathbf{x}$ only (no labels), goal: discover underlying *hidden structure*. Examples: clustering, dimensionality reduction, feature learning, **density estimation**.

Within unsupervised deep learning, two broad families:

- **Non-probabilistic models**: autoencoders.
- **Probabilistic generative models**: VAEs, GANs, diffusion models.

70.2 What is a Generative Model?

A **generative model** takes unlabelled training samples drawn from some distribution $p_{\text{data}}(\mathbf{x})$ and learns a model $p_{\text{model}}(\mathbf{x})$ that represents that distribution. Three things this enables:

- **Density estimation** — evaluate how likely a sample is under the data distribution.
- **Representation learning** — learn embeddings useful for downstream tasks.
- **Sampling** — generate new data points (with the same or related distribution).

Definition 70.1 (Discriminative vs. generative)**.** Given a training set $\mathbf{X}$ with labels Y:

- **Discriminative models** learn the conditional $p(Y \mid \mathbf{X})$ — they draw a boundary.
- **Generative models** learn the joint or marginal $p(\mathbf{X})$ (or $p(\mathbf{X}, Y)$) — they model where the data *lives*.

Deep generative models are essentially **distribution transformers**: they map a simple prior distribution $p(\mathbf{z})$ (typically Gaussian) through a neural network G onto a complex target distribution $p(\mathbf{x})$.

$$\mathbf{z} \sim \mathcal{N}(0, I) \xrightarrow{G} \mathbf{x} \sim p_{\text{model}}(\mathbf{x})$$

70.3 Why Generative Models?

- **Realistic samples**: artwork, super-resolution, colorization, in-painting.
- **Simulation and planning**: reinforcement learning environments, sim-to-real (e.g., GTA5 $\rightarrow$ Cityscapes).
- **Data augmentation** for downstream tasks.
- **Latent representations**: interpretable and useful for editing, retrieval, transfer.

71 Autoencoders

71.1 Architecture

An **autoencoder** (Bourlard & Kamp, 1988) is an unsupervised approach for learning a lower-dimensional feature representation from unlabelled data. It is composed of two networks:

- **Encoder** f: maps the input $\mathbf{x}$ to a latent code $\mathbf{h} = f(\mathbf{x})$, usually with $\dim(\mathbf{h}) < \dim(\mathbf{x})$.
- **Decoder** g: produces a reconstruction $\hat{\mathbf{x}} = g(\mathbf{h})$ with a (typically mirrored) architecture.

The system is trained end-to-end to minimise the **reconstruction loss**, usually L_2:

$$L(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 = \sum_i (x_i - \hat{x}_i)^2.$$

The bottleneck $\mathbf{h}$ is forced to capture the meaningful factors of variation in $\mathbf{x}$ — there is no explicit supervision, but the reconstruction objective provides *implicit* supervision.

71.2 Implementation Variants

- **Originally**: linear layer + sigmoid non-linearity.
- **Later**: deep fully-connected networks.
- **Much later**: convolutional encoders/decoders with ReLU, suited for images.

Remark. A linear autoencoder with an L_2 loss and no non-linearities recovers **PCA**: the optimal encoder spans the top principal components of the data. Non-linear autoencoders generalise PCA to non-linear manifolds.

71.3 Convolutional Autoencoders

For image data, replace fully-connected layers with convolutional blocks in the encoder and *deconvolutional* (transposed convolution) blocks in the decoder. This preserves spatial structure and dramatically reduces parameter count.

71.4 Dimensionality of the Latent Space

The bottleneck dimension is the central hyperparameter:

- **Too small**: reconstructions become blurred (autoencoding compresses too aggressively).
- **Larger latent**: better reconstructions, but risk of trivial identity mapping if capacity is unconstrained.

71.5 Regularized Autoencoders

When the latent dimension equals or exceeds the input dimension, the model is **overcomplete** and could trivially copy the input. **Regularised autoencoders** avoid this by adding a penalty that encourages a useful structure on $\mathbf{h}$.

Sparse autoencoder. Apply an L_1 penalty on the code:

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \lambda \|\mathbf{h}\|_1$$

Forces most code activations to be zero $\Rightarrow$ sparse, disentangled features.

Contractive autoencoder. Penalise the Jacobian of the encoder, forcing the latent code to be locally invariant to input perturbations:

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2$$

71.6 Denoising Autoencoders (DAE)

Another way to force the autoencoder to learn useful features is to corrupt the input with random noise $\mathbf{w}$ and ask the network to recover the *clean* version (Vincent et al., 2008):

$$\text{minimise } L(\mathbf{x},\, g(f(\mathbf{x} + \mathbf{w}))).$$

- Architecture is unchanged — only the training data differs.
- The latent code must capture structure that is robust to input corruption.

- Useful for image denoising, watermark removal, and as a strong pretraining task for deep networks.

71.7 Autoencoders as Pretraining

After training, the decoder is *thrown away* and the encoder is reused to initialise a supervised model. A classifier head is attached on top of $\mathbf{h}$ and fine-tuned (possibly jointly with the encoder) on a small labelled dataset.

Transfer pipeline

1. Train autoencoder on *large unlabelled* dataset.
2. Discard decoder, keep encoder.
3. Add classifier on top of features $\mathbf{h}$.
4. Fine-tune end-to-end on *small labelled* dataset for the target task.

71.8 Applications

- **Image denoising / restoration**: DAE-style training.
- **Image colorization**: input grayscale, output color.
- **Watermark removal**, super-resolution, in-painting.
- **Anomaly detection**: a normal-data autoencoder fails to reconstruct out-of-distribution inputs; high reconstruction error flags anomalies.
- **Transfer learning** (above).

71.9 Autoencoder Summary

- Dimensionality reduction based on deep networks.
- Very flexible: works for images, text, time series, graphs, ...
- Active research threads: denoising AEs, sparse AEs, variational AEs.

A plain autoencoder is *not* a generative model: sampling a random point in latent space does not necessarily decode to a plausible $\mathbf{x}$, because the encoder may map data to an arbitrary, disconnected region of the latent space.

72 Variational Autoencoders (VAE)

72.1 Motivation

Plain autoencoders learn a deterministic mapping $\mathbf{x} \mapsto \mathbf{z}$. A **VAE** (Kingma & Welling, 2014) instead encodes each input as a *distribution* over the latent space:

$$\text{AE: } \mathbf{x} \to \mathbf{z} = e(\mathbf{x}) \to \hat{\mathbf{x}} = d(\mathbf{z})$$

$$\text{VAE: } \mathbf{x} \to q(\mathbf{z} \mid \mathbf{x}) \to \mathbf{z} \sim q(\mathbf{z} \mid \mathbf{x}) \to \hat{\mathbf{x}} = d(\mathbf{z})$$

The distributions are chosen to be **normal**: the encoder learns the *mean* $\boldsymbol{\mu}_{\mathbf{z}|\mathbf{x}}$ and *covariance* $\boldsymbol{\Sigma}_{\mathbf{z}|\mathbf{x}}$. Sampling from this distribution and decoding gives a probabilistic generative model: we can now draw new $\mathbf{z}$ from the prior $p(\mathbf{z})$ and decode to obtain a new $\hat{\mathbf{x}}$.

72.2 Probabilistic Setup

Assume the data is generated from unobserved latent variables $\mathbf{z}$:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{z})\, p_\theta(\mathbf{x} \mid \mathbf{z})\, d\mathbf{z}$$

- **Prior** $p_\theta(\mathbf{z})$: simple, typically $\mathcal{N}(\mathbf{0}, I)$.
- **Conditional** $p_\theta(\mathbf{x} \mid \mathbf{z})$: complex, parametrised by a **neural network** (decoder).

The goal is to estimate θ by **maximum likelihood**.

72.3 Intractability

Two quantities we would like, both **intractable**:

- **Likelihood** $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{z}) p_\theta(\mathbf{x} \mid \mathbf{z})\, d\mathbf{z}$ — integrating over all latents is infeasible for high-dimensional $\mathbf{z}$.
- **Posterior** $p_\theta(\mathbf{z} \mid \mathbf{x}) = p_\theta(\mathbf{x} \mid \mathbf{z}) p_\theta(\mathbf{z}) / p_\theta(\mathbf{x})$ — depends on the intractable marginal $p_\theta(\mathbf{x})$.

72.4 Solution: Approximate Posterior + ELBO

Introduce a second neural network — the **encoder** $q_\phi(\mathbf{z} \mid \mathbf{x})$ — that *approximates* the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. With this approximation, one can derive a tractable lower bound on the data log-likelihood, the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z}))}_{\text{regulariser}}$$

- **Reconstruction term**: forces the decoder to put high probability on the true $\mathbf{x}$ given samples from q_ϕ.
- **Regulariser**: KL divergence between approximate posterior and prior. Forces $q_\phi(\mathbf{z} \mid \mathbf{x})$ to stay close to the prior $p(\mathbf{z})$, which is essential for sampling: if posteriors collapse to isolated points, the latent space has "holes" and random $\mathbf{z}$ decode to garbage.

Remark. Since both networks output distributions, the VAE is intrinsically **probabilistic**: encoder and decoder both produce a mean and a covariance, not a single point.

72.5 Training and Sampling

Training. Maximise the ELBO with stochastic gradient ascent. The expectation under q_ϕ is estimated by sampling $\mathbf{z}$ from the encoder; gradients flow through the sampling step via the **reparametrisation trick** ($\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$).

Sampling. Once trained, generation is straightforward:

1. Sample $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$.
2. Pass $\mathbf{z}$ through the decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ to obtain a generated $\hat{\mathbf{x}}$.

72.6 Latent Space Properties

Diagonal Gaussian prior. The prior on $\mathbf{z}$ is typically a diagonal Gaussian: each latent dimension is independent. This simplification makes training tractable and tends to encourage **disentanglement** — each latent coordinate captures a separate factor of variation.

Interpretability. In practice, different dimensions of $\mathbf{z}$ end up encoding interpretable factors. In the classic VAE faces experiment (Kingma & Welling, 2014), varying z_1 changes the head pose while varying z_2 changes the degree of smile.

72.7 Conditional VAE (CVAE)

If labels (or any side information y) are available, condition both the encoder and the decoder on y: replace $p_\theta(\mathbf{x} \mid \mathbf{z})$ with $p_\theta(\mathbf{x} \mid \mathbf{z}, y)$ and $q_\phi(\mathbf{z} \mid \mathbf{x})$ with $q_\phi(\mathbf{z} \mid \mathbf{x}, y)$. The derivation is otherwise unchanged. CVAEs enable controlled generation: progression along attributes such as gender, age, hair colour, expression, or eyewear.

72.8 VAE Summary

VAE: pros and cons

Pros.

- Principled probabilistic framework for generative modelling.
- Yields a tractable approximate posterior $q_\phi(\mathbf{z} \mid \mathbf{x})$ that is a useful feature representation.
- Stable training compared to GANs.

Cons.

- Only maximises a *lower bound* on the likelihood — not the likelihood itself.
- Samples tend to be **blurrier** and of lower visual quality than GANs (a consequence of the Gaussian likelihood and KL regulariser).

Modern extensions. VQ-VAE (vector-quantised latents), VAEs combined with transformer decoders.

73 Generative Adversarial Networks (GANs)

73.1 Idea

Introduced by Goodfellow et al. (NIPS 2014). Unlike VAEs, GANs **do not** use an explicit density function. They focus purely on the *ability to sample*:

How do we sample from a complex high-dimensional training distribution?

Trick. Sample from a simple distribution (random noise) and train a neural network — the **generator** — to transform that noise into samples from the desired distribution (e.g., human faces, handwritten digits).

73.2 Two-Player Game

A GAN consists of two networks trained **adversarially**:

- **Generator** G: maps noise $\mathbf{z} \sim p(\mathbf{z})$ to $G(\mathbf{z})$, attempting to fool the discriminator.
- **Discriminator** D: receives both real samples $\mathbf{x} \sim p_{\text{data}}$ and fake samples $G(\mathbf{z})$, outputs the probability that the input is *real*.

The generator improves by trying to produce samples the discriminator misclassifies as real; the discriminator improves by becoming better at telling real from fake. The game is the deep-learning analogue of a forger versus a detective.

73.3 Objective Function

The generator and discriminator are trained jointly with a **minimax** objective:

$$\min_{\theta_g} \max_{\theta_d} \Big[\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D_{\theta_d}(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}[\log(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z})))]\Big]$$

- D wants $D(\mathbf{x}) \approx 1$ on real data and $D(G(\mathbf{z})) \approx 0$ on fakes — so D **maximises** the objective.
- G wants $D(G(\mathbf{z})) \approx 1$ — so G **minimises** the objective.

73.4 Training Algorithm

The minimax problem is solved by **alternating gradient updates**:

GAN training loop

Repeat:

1. **Gradient ascent on discriminator**:

$$\max_{\theta_d} \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}[\log D_{\theta_d}(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z})))]$$

2. **Gradient descent on generator**:

$$\min_{\theta_g} \mathbb{E}_{\mathbf{z}}[\log(1 - D_{\theta_d}(G_{\theta_g}(\mathbf{z})))]$$

73.5 Vanishing-Gradient Problem and the Non-Saturating Loss

The original generator loss $\log(1 - D(G(\mathbf{z})))$ has poor gradient behaviour:

- When samples are clearly fake ($D(G(\mathbf{z})) \approx 0$), the curve is **flat**: the gradient is tiny, and G learns slowly precisely when it needs to improve most.
- When samples are already good ($D(G(\mathbf{z})) \approx 1$), the gradient is **large** — but those samples don't need much updating.

The generator learns more from good samples than from bad ones — an ironic and unhelpful inversion.

Fix. Replace "minimise $\log(1 - D(G(\mathbf{z})))$" with "*maximise* $\log D(G(\mathbf{z}))$":

$$\max_{\theta_g} \mathbb{E}_{\mathbf{z}}[\log D_{\theta_d}(G_{\theta_g}(\mathbf{z}))]$$

Same fixed point, but the gradient is now large precisely in the region where samples are bad. This is the standard "non-saturating" GAN loss.

73.6 Issues with GAN Training

Common GAN pathologies

- **Hard to train**. Early in training, gradients are tiny; later, learning concentrates on already-good samples. The imbalance makes training slow or unstable.
- **Oscillation / divergence**. Parameters can oscillate; generator loss does not correlate with sample quality.
- **Hyperparameter sensitivity**. Behaviour very sensitive to learning rates, batch size, architecture.
- **Mode collapse**. Generator produces only a small subset of the data distribution.

A large literature improves stability, mostly by changing the **loss function**: LSGAN, WGAN, WGAN-GP, DRAGAN, EBGAN, BEGAN, ...

73.7 DCGAN

Radford et al. (ICLR 2016) introduced **Deep Convolutional GAN**, the first GAN to reliably scale to large-resolution image generation.

Generator. Upsampling network with **fractionally strided convolutions** from a $\mathbf{z}$ vector of dimension ~ 100 to a full image.

Discriminator. Convolutional classifier.

DCGAN architecture guidelines

- Replace pooling with **strided convolutions** (discriminator) and **fractional-strided convolutions** (generator).
- Use **batch normalisation** in both networks.
- Remove fully-connected hidden layers for deeper architectures.
- Generator: ReLU everywhere except the output (Tanh).
- Discriminator: LeakyReLU everywhere (sparse gradients from plain ReLU hurt training).

73.8 Latent-Space Arithmetic

Once a GAN has learned a smooth, meaningful latent space, semantic concepts emerge as **linear directions**.

Example 73.1 (Vector arithmetic on faces (DCGAN))**.**

$$\mathbf{z}_3 = \mathbf{z}_2 - \mathbf{z}_1 + \mathbf{z}_4$$

where $\mathbf{z}_1 \to$ man, $\mathbf{z}_2 \to$ man with glasses, $\mathbf{z}_4 \to$ woman. Then $G(\mathbf{z}_3)$ produces *a woman with glasses*. Analogous arithmetic works for "smiling man = smiling woman − neutral woman + neutral man".

The GAN implicitly learns high-level concepts (gender, glasses, expression, hair) and organises them as linear directions, enabling **latent-space editing** of generated content.

73.9 Modern GANs: Highlights

- **LSGAN, WGAN, WGAN-GP** (2017). Stability via least-squares and Wasserstein objectives.

- **Progressive GAN** (Karras 2018). Progressively grow resolution; first photorealistic faces.
- **CycleGAN** (Zhu 2017). Unpaired image-to-image translation (horse ↔ zebra, apple ↔ orange, summer ↔ winter Yosemite).
- **pix2pix** (Isola 2017). Paired translation: sketch → photo, map → aerial.
- **Text → Image Synthesis** (Reed et al. 2017). Generate images from captions.
- **BigGAN** (Brock et al. 2019). Large-scale, high-fidelity natural-image synthesis.

74 Evaluating Generative Models

74.1 Why Evaluation is Hard

Evaluating generative models is genuinely difficult:

- Showing pictures (user study) is not enough — especially for "easy" datasets like MNIST, CIFAR, faces.
- Likelihoods of high-dimensional samples cannot be computed directly; distributions cannot be directly compared.
- Many improvements claim *stability* or *ease of training*, which are difficult to quantify.

Two surrogate metrics dominate the literature.

74.2 Inception Score (IS)

A good generator should produce a *variety* of *recognisable* object classes. Pass generated images through a pretrained **Inception v3** classifier:

- For *recognisability*, each image's predicted class distribution should be **low-entropy** (confident on one class).
- For *diversity*, the marginal class distribution across many samples should be **uniform** (covers all classes).

Problems. A model that simply memorises the training set (**overfits**) or outputs a single canonical image per class (**mode dropping**) can still score well.

74.3 Fréchet Inception Distance (FID)

FID addresses some IS limitations by directly comparing the *statistics* of real and generated images in Inception feature space.

Algorithm.

1. Extract features from the `pool3` layer of Inception v3 for both real and generated images.
2. Fit a multivariate Gaussian to each set of features.
3. Compute the Fréchet distance (a.k.a. Wasserstein-2 distance) between the two Gaussians:

$$\mathrm{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \mathrm{Tr}\Big(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g)^{1/2}\Big)$$

   where $(\boldsymbol{\mu}_r, \boldsymbol{\Sigma}_r)$ and $(\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g)$ are the means and covariances of real and generated features.

Pros. Lower FID ⇒ generated images are more similar to real ones in content and diversity.

Cons. Cannot detect overfitting (memorising the training set yields excellent FID).

75 GANs vs. VAEs

GANs and VAEs are two complementary approaches:

- **GANs** provide a *sampling mechanism*. The adversarial loss progressively makes samples indistinguishable from real data, but the model does not explicitly represent any density.
- **VAEs** learn an *explicit (approximate) probability distribution* over the data. The distribution can be used to sample new data and to assess the probability of new data points.

75.1 Properties of Generative Models

A short checklist of desirable properties:

- **High-quality sampling**. Samples indistinguishable from training data.
- **Coverage**. Samples represent the entire training distribution, not just a subset.
- **Well-behaved latent space**. Every $\mathbf{z}$ corresponds to a plausible $\mathbf{x}$; smooth changes in $\mathbf{z}$ produce smooth changes in $\mathbf{x}$.
- **Interpretable latent space**. Each dimension of $\mathbf{z}$ corresponds to an interpretable factor.
- **Efficient likelihood computation**. For probabilistic models, evaluate $p(\mathbf{x})$ efficiently and accurately.

75.2 Comparison Table

Property	**GANs**	**VAEs**
Efficient sampling	✓	✓
High-quality samples	✓	× (blurry)
Coverage	× (mode collapse)	?
Well-behaved latent space	✓	✓
Interpretable latent space	?	?
Efficient likelihood	n/a (no density)	× (only ELBO)

76 Summary

Take-Home Messages

- Generative models learn $p_{\text{model}}(\mathbf{x})$ from unlabelled data, enabling density estimation, representation learning, and sampling.
- Deep generative models are **distribution transformers**: they push a simple prior (Gaussian noise) through a neural network onto a complex target distribution.
- **Autoencoders** are non-probabilistic; they learn a deterministic latent code by reconstruction. Variants: sparse, contractive, denoising, convolutional. Not generative on their own, but a strong pretraining tool.
- **VAEs** are probabilistic autoencoders: the encoder outputs a distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$; training maximises the ELBO (reconstruction − KL to prior). Stable to train but samples are blurry.
- **GANs** sidestep the density and train a generator/discriminator pair in a minimax game. Produce sharp samples but are hard to train (vanishing gradients, mode collapse, hyperparameter sensitivity). DCGAN, WGAN, Progressive GAN, CycleGAN, BigGAN, ...
- **Latent-space arithmetic** (DCGAN) reveals that GANs implicitly organise semantic concepts as linear directions, enabling generative editing.
- Evaluation is hard: **Inception Score** (recognisability + diversity) and **FID** (Fréchet distance in Inception features) are standard but imperfect — both can be fooled by memorisation.
- GANs vs. VAEs is a quality/coverage trade-off: GANs produce sharper samples but suffer mode collapse; VAEs cover the distribution but produce blurry samples. **Diffusion models** (covered elsewhere) combine many of the strengths of both.

Part XII

Introduction to Multimodal Learning

77 What is Multimodality?

77.1 Origin of the Term

The word **multimodality** originates in mathematics, where it refers to a probability distribution with multiple modes (peaks). In machine learning, the term has taken on a broader meaning tied to how information is acquired and expressed.

77.2 Modality

A **modality** refers to the way in which something is expressed or perceived. Modalities range from **raw** (close to the sensor) to **abstract** (far from the sensor):

$$\text{Raw} \longleftarrow \underbrace{\text{Speech signal, Image}}_{\text{close to sensor}} \longrightarrow \underbrace{\text{Language, Detected objects}}_{\text{intermediate}} \longrightarrow \underbrace{\text{Sentiment, Object categories}}_{\text{abstract}}$$

For humans, modalities correspond to the sensory channels: **vision, hearing, touch, taste, smell**.

77.3 Multimodality: A Working Definition

Two definitions are typically given:

- *Dictionary*: "with multiple modalities".
- *Research-oriented* (Morency, CMU): **Multimodal is the science of heterogeneous and interconnected data.**

Heterogeneous — Different modalities exhibit *diverse qualities, structures and representations*. A spectrum runs from very **homogeneous** (e.g. images from two cameras) through intermediate (text in two languages) to very **heterogeneous** (language + vision).

Interconnected — Modalities are typically related and share complementary information. Their interactions can be characterised at two levels:

- **Statistical**: *association* (correlation, co-occurrence) and *dependency* (e.g. temporal contingency).
- **Semantic**: *correspondence* (grounding — the word "car" linked to a region in an image) and *relationship* (knowledge-base-style links, e.g. "dad").

77.4 Multimodal is Multidisciplinary

Multimodal learning sits at the intersection of: psychology, medicine, language processing, computer vision, robotics, machine learning, multimedia, speech analysis.

77.5 Common Modalities

- **Natural language**: spoken, written.
- **Visual**: images, video, RGB, infrared (thermal), depth.
- **Audio**: voice, sound, music.
- **Biological signals**: EEG, ECG, GSR (galvanic skin response).
- **Haptics**: touch.
- **Motion capture**.

A finer-grained taxonomy for behavioural data uses the **LVTAPM** mnemonic: **L**anguage (lexicon, syntax, pragmatics), **V**isual (gestures, body language, eye contact, facial expressions), **T**ouch (haptics, motion), **A**coustic (prosody, vocal expressions), **P**hysiological (skin conductance, ECG), **M**obile (GPS, accelerometer, light sensors).

78 Multimodal Machine Learning

78.1 Definition

Multimodal ML takes multiple input modalities — e.g. language ("I really like this tutorial"), vision (a video of the speaker), and acoustics (a waveform) — and produces a feature vector or a prediction $\hat{y}$. All learning regimes apply: *unsupervised, self-supervised, supervised, reinforcement*.

78.2 A Historical Timeline

- **1970s–1980s**: Audio-visual speech recognition.
- **Late 1980s–1990s**: Content-based video retrieval.
- **1990s–2000s**: Affect and emotion recognition.
- **2000s–2010s**: Video event recognition (TrecVid), multimodal sentiment analysis.
- **∼2015**: Image captioning revisited ⇒ **birth of "Language & Vision"** research.
- **2016+**: Visual question answering, video QA, referring expressions.
- **2017**: Multimodal dialogue.
- **2018+**: Large-scale video retrieval (YouTube8M), language + vision + navigation, self-driving.

78.3 Application Examples

Audio-visual speech recognition. Speech recognisers based on *audio cues only* are sensitive to noise. Combining audio with *visual* cues (face tracking, mouth localisation) gives more robust "audio-visual speech recognition".

Affective computing. Systems that *recognise, interpret, process, and simulate* human emotions and internal states. Emotions are key for decision making.

Human–human–robot interaction analysis. Social Signal Processing (Social AI) integrates social-psychology concepts into AI: automatic detection, interpretation and synthesis of *social signals* carried by non-verbal behavioural cues.

Event / behaviour understanding. Action recognition, event segmentation, video summarisation — typically combining skeleton, depth, RGB and accelerometer modalities.

Content understanding. The ability of computers to ingest, relate and process information across modalities (text, speech, images, sensor data) and perform tasks using it.

Multimedia information retrieval. Image and video captioning, visual question answering, cross-modal retrieval.

79 The Five Multimodal Challenges

Following Baltrusaitis, Ahuja and Morency (TPAMI 2019), multimodal ML research can be organised around five technical challenges: **Representation, Alignment, Translation, Fusion, Co-learning**. A sixth meta-task — **Generation** — is increasingly distinguished by Liang, Zadeh & Morency (2023).

79.1 Challenge 1: Representation

How to represent and summarise multimodal data so that the *complementarity* and *redundancy* of multiple modalities is exploited.

Joint representations. All modalities are projected to a *single shared space*:

$$\mathbf{x}_m = f(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n).$$

Useful when modalities are tightly coupled and a single model can ingest all of them.

Coordinated representations. Each modality keeps its own representation $f(\mathbf{x}_1)$, $g(\mathbf{x}_2)$, ... but the spaces are *coordinated* by a constraint (typically a similarity, e.g. cosine distance):

$$f(\mathbf{x}_1) \sim g(\mathbf{x}_2).$$

More appropriate when modalities are fundamentally very different (e.g. image vs. text), where a forced joint space tends to wash out modality-specific structure.

79.1.1 Coordination via Contrastive Learning

Coordinated spaces are most often learned with a **contrastive loss**: pull *positive pairs* (corresponding instances across modalities) together, push *negative pairs* apart.

Simple contrastive (triplet-style) loss.

$$\max\{\, 0,\ \alpha + \mathrm{sim}(\mathbf{z}_A,\, \mathbf{z}_B^+) - \mathrm{sim}(\mathbf{z}_A,\, \mathbf{z}_B^-) \,\}$$

The similarity is typically cosine. Paired data $\{\triangle, \bigcirc\}$ (e.g. images and their captions) supply positives; unrelated cross-modal samples supply negatives.

79.1.2 CLIP — Contrastive Language-Image Pre-training

CLIP (Radford et al. 2021) is the flagship example of cross-modal coordination via contrastive learning. Two encoders, f_L for language and f_V for vision, produce embeddings $\mathbf{z}_L, \mathbf{z}_V$ coordinated by the **InfoNCE** (Noise Contrastive Estimation) loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\mathrm{sim}(\mathbf{z}_A^i, \mathbf{z}_B^i)}{\sum_{j=1}^{N} \mathrm{sim}(\mathbf{z}_A^i, \mathbf{z}_B^j)}$$

Key properties of CLIP

- f_L and f_V are strong encoders for any language-vision task.
- $\mathbf{z}_L$ and $\mathbf{z}_V$ live in *coordinated but not identical* spaces — not a single joint embedding.
- **Zero-shot** capability: classify images using natural-language label prompts at inference time, no fine-tuning required.
- Strong **generalisability**: applied far beyond natural images, including medical imaging (X-ray, CT, MRI, ultrasound, histology, dermatology, ECG) for classification, segmentation, detection, retrieval, VQA, report generation.

79.2 Challenge 2: Alignment

Alignment = finding relationships and correspondences between sub-components of instances from two or more modalities.

Example tasks.

- Given an image and a caption, find the image regions corresponding to each word/phrase.
- Given a movie, align it to its script or to book chapters.

79.2.1 Explicit vs. Implicit Alignment

Explicit alignment. Alignment is the *task itself*. Find correspondence between elements of different modalities, typically via a loss function. Classical method: **dynamic time warping** (e.g. aligning speech signal to transcript, or unaligned video to reference audio).

Implicit alignment. Alignment used as an *intermediate* (often latent) step inside a larger task. Modern method: **attention mechanisms** (self-attention, transformers). Example tasks: image/video captioning, VQA, cross-modal retrieval.

79.2.2 Why Attention is so Useful for Multimodal Data

- Assigns **different weights** to different parts of the input ⇒ focuses on the relevant parts of each modality.
- Effective for sequences of **varying lengths** (text length ≠ audio duration ≠ number of objects in an image).
- Helps **align modalities with different structures and temporal granularities** — learns which parts of one modality correspond to which parts of another.
- SOTA encoders — **BERT** (text), **ViT** (images), **CLIP** (vision-language) — all rely on attention to integrate information.

79.2.3 Semantic Alignment: Grounding

Grounding = establishing correspondences between modalities so that information in one modality becomes *meaningful in the context of another*.

Visual grounding. Linking specific regions or objects in an image/video to corresponding textual descriptions ("A *woman* reading a *newspaper*" → two bounding boxes).

Text grounding. Linking words or phrases in text to corresponding elements in other modalities (images, audio, video). E.g. VQA in text-rich images, where the model outputs both an answer and the bounding box that justifies it.

Referring Expression Segmentation (RES). Generate a segmentation mask for the object described by a natural-language expression ("the kid in red", "everyone except the kid in white"). A grounding → segmentation generalisation.

79.3 Challenge 3: Translation

How to map data from *one modality to another*: given an entity in modality A, generate the same entity in modality B.

Examples. Image → caption, text → image, speech → text, sketch → photo.

79.3.1 Two Families

Example-based models. Use a *dictionary* of paired examples and retrieve the closest one (image captioning by retrieval, media retrieval).

Generative models. Construct a model that *produces* the target modality. Classical pipeline: encoder-decoder — encode the source into a vector, decode it into the target. Examples: text ↔ image, sound synthesis, video description generation.

79.3.2 Translation Examples

CycleGAN. Unpaired image-to-image translation using cycle-consistent adversarial networks (Zhu et al., 2017). Monet ↔ photo, zebra ↔ horse, summer ↔ winter. *Adversarial loss* forces translated images to be indistinguishable from real images in the target domain.

Image → avatar. Cross-modal latent space alignment using a parametric set of facial attributes to render avatars (De Guevara et al., ICCV 2023).

DALL-E. Text-to-image. Two key components:

- **Discrete VAE (dVAE)**: encoder maps images to discrete latent codes; decoder reconstructs the image from those codes.
- **Autoregressive transformer (GPT-style)**: generates one element at a time, conditioning each subsequent element on the previous ones. Maps text tokens → image tokens.

DALL-E 2. Replaces the autoregressive image-token generator with a **diffusion model** conditioned on **CLIP image embeddings**, which at inference time are generated from CLIP *text* embeddings by a learned prior.

Stable Diffusion. Latent-space diffusion for text-to-image, image-to-image with text guidance, text-guided image editing ("swap sunflowers with roses", "add fireworks to the sky").

Stable Diffusion components (recap)

- **Forward diffusion**: gradually adds Gaussian noise to an image over many steps until it is pure noise.
- **Reverse diffusion**: a **U-Net** predicts and subtracts the noise step-by-step.
- **(CLIP) text encoder**: encodes the prompt; embedding conditions the U-Net.
- **Latent space**: diffusion runs in a low-dimensional latent space (VAE-encoded image), not in pixel space — huge cost savings.

79.4 Challenge 4: Fusion

How to *join* two or more modalities to perform a prediction.

79.4.1 Early vs. Late Fusion

Early fusion. Combine modalities at the *feature level* and train a *single model* on the concatenated/joined low-level features.

+ Single model ⇒ simpler training pipeline.
+ Exploits correlations and interactions between low-level features.

Late fusion. Train one model per modality, then combine their *decisions* (averaging, voting, weighted combination).

+ Each modality can use a specialised model tailored to it.
+ More flexible; easy to handle missing modalities at inference.

79.4.2 Classical Fusion Approaches

Multiple Kernel Learning (MKL). Kernel SVMs extended to use a different kernel per modality. Kernels act as similarity functions; different kernels allow better fusion of heterogeneous data.

Graphical models. Coupled and factorial Hidden Markov Models, dynamic Bayesian networks, multi-stream HMMs, conditional random fields. Generative (joint $p(X, Y)$) or discriminative (conditional $p(Y \mid X)$).

79.4.3 Neural Fusion

For temporal multimodal data: RNNs / LSTMs; image captioning with a CNN visual encoder + LSTM language decoder; modern: transformer-based fusion.

Neural fusion: pros and cons

Pros.

- Can learn from huge amounts of data.
- End-to-end training of representation + fusion components.
- Outperforms non-NN approaches on most tasks.

Cons.

- **Lack of interpretability**: hard to tell which modality / feature drove a prediction.
- Need large multimodal training datasets.

79.5 Challenge 5: Co-learning

Co-learning = transfer knowledge *between* modalities. Use knowledge learned from one modality to help a model that operates on another.

When it matters.

- One modality has *limited resources*: scarce annotations, noisy input, unreliable labels.
- The *helper modality* is often available **only at training time**, not at test time.

Co-learning objective	Purpose
Presence of modality	Handle missing modalities at train or test time
Data parallelism	Strongly / weakly / shared-modality paired data
Noisy modality	Robustness to noisy data and labels
Modality annotations	Annotated, partially, or unannotated modalities
Domain adaptation	Different domain at train vs. test time
Interpretability & fairness	Unbiased and explainable predictions

79.5.1 Co-learning via Representation

Transfer information from a *secondary modality* to a *primary modality* by **sharing representation spaces**. Crucially, do not confuse *multimodal co-learning* with *unimodal learning*:

Co-learning vs. unimodal

- **Train**: multimodal data + multimodal model.
- **Test**: only the primary modality available; the secondary modality slot is filled with zeros / dropped.
- Empirically: multimodal co-learning > language-only training, even when only language is available at test time.

Example: cross-modal autoencoder for diagnosis (Radhakrishnan et al., Nature Communications 2023). ECG and MRI are encoded into a shared latent space at training. At inference, ECG alone (cheap, available) is mapped through its encoder to the shared latent, from which MRI-derived phenotypes can be predicted — effectively translating one modality into another.

79.6 Challenge 6: Generation

Generation generalises translation. Liang, Zadeh & Morency (2023) distinguish three sub-tasks based on the relation between input and output information content:

Sub-task	Info content	Examples
Summarisation	Reduction (>)	Text from video, video summary from text
Translation	Maintenance (=)	Image → caption, text → image
Creation	Expansion (<)	Image+text → audio+text+video

79.6.1 Summarisation

Definition 79.1 (Summarisation)**.** The process of obtaining $X_{\text{sum}} = f(D)$ with $\text{length}(X_{\text{sum}}) \leq \text{length}(D)$, where D is the input data and f the summarisation function.

Multimodal summarisation. The input D comprises several partially disjoint sets of modality information $\{M_1, M_2, \ldots, M_n\}$ with $n \geq 2$ and non-empty shared latent information between at least one pair. If the output X_{sum} itself spans ≥ 2 modalities, the summary is *multimodal*; otherwise unimodal.

In practice, complementary modalities are used to *enhance the feature representation of the primary modality*, while the summary is generated in a single (often text) modality. Example: language-guided video summarisation.

Datasets. Text + image; video-based news articles (text + video); audio + transcript.

79.6.2 Creation

Simultaneously generates *multiple* modalities while maintaining coherence within and across them. Examples: language generation (captioning), high-resolution speech and sound generation, photorealistic image generation (text-to-image, speech-to-image). Modern multimodal LLMs (LLaVA, Mistral, ImageBind) combine text and images to caption, answer questions about visual content, or create new images from text.

Creation: four hard requirements

1. **Conditional**: preserve semantically meaningful mappings from the seed to the generated modalities.
2. **Synchronised**: semantically coherent across modalities.
3. **Stochastic**: capture many possible futures given a state.
4. **Autoregressive**: support long-range generation.

Hardest part: *evaluation*. User studies are the ideal — but are slow, costly, and subjective. Serious ethical issues with deepfakes, fake news, hate speech, lip-syncing.

80 Learning and Optimization Process

80.1 Generalisation Across Modalities and Tasks

Real-world platforms — phones, smart devices, self-driving cars, healthcare systems, robots — integrate many sensors beyond the prototypical text/video/audio triad. Generalising across *paired* and *unpaired* modality inputs (each task uses only a subset of all modalities) is still an open challenge.

80.2 Balanced and Efficient Training

Multimodal networks are prone to **overfitting** due to their large capacity, and different modalities *overfit and generalise at different rates*. Training all branches with a single optimisation strategy is sub-optimal.

OGR (Overfitting-to-Generalisation Ratio) (Wang et al., CVPR 2020). Measure the gap between training and validation loss *per modality* and use it to decide how much to train each branch.

80.3 Performance–Robustness–Complexity Trade-offs

(MultiBench, Liang et al., NeurIPS 2021.)

- **Performance**: prediction accuracy.
- **Robustness**: generalisation, resilience to noise/corruption/adversarial inputs, fault tolerance.
- **Complexity**: model size, inference time, scalability.

There is a structural trade-off: high-performance multimodal architectures often score worse on robustness.

81 Modality Biases

81.1 Unimodal Biases and Modality Collapse

A multimodal model can degenerate into a *unimodal* model. Classic VQA failure: a model is asked "what colour is the banana?" and answers *yellow* regardless of the image, because $\sim 80\%$ of bananas in the training set are yellow. The image branch is effectively ignored.

Solutions (Liang, Zadeh & Morency, 2023):

- **Balancing modalities**: curate datasets with adversarial / counterfactual examples (e.g. VQA v2 by Goyal et al., CVPR 2017).
- **Balancing training**: training regularisers (e.g. RUBi) that prevent the model from relying on a single modality.

81.2 Fairness and Social Biases

Sources.

- **Bias in historical data**: training data carries unfair or discriminatory patterns.
- **Lack of representation**: insufficient data for some groups ⇒ inaccurate predictions and unfair treatment.

Image captioning example (Hendricks et al., ECCV 2018, "Women also Snowboard"):

- *Right for the wrong reasons*: a baseline captioning model labels images of women near laptops as "a man sitting at a desk".
- *Right for the right reasons*: a debiased model attends to the actual person before predicting gender.

Spurious correlations link **gender** to objects (kitchen, snowboard, tennis racquet) and to **actions**.

81.3 Cross-modal Interactions Can Worsen Biases

(Srinivasan & Bisk, NAACL 2022, "Worst of Both Worlds".) In VL-BERT, the language-only mask-filling task predicts gendered objects (purse, briefcase) with a small bias; *adding visual information makes the model more confident in reinforcing the stereotype*, not less. Cross-modal interactions can therefore **compound**, not mitigate, social biases.

82 Summary

Take-Home Messages

- A **modality** is a way information is expressed or perceived; **multimodal learning** is the science of *heterogeneous and interconnected* data.
- Modalities differ in their qualities and structure (homogeneous → heterogeneous) and interact statistically (association, dependency) and semantically (correspondence, relationship).
- Multimodal ML organises around five (or six) technical challenges: **Representation, Alignment, Translation, Fusion, Co-learning, (Generation)**.
- **Representations**: *joint* (single shared space) vs. *coordinated* (separate spaces linked by a constraint). **CLIP** is the canonical contrastive coordinated example (InfoNCE loss, image + text encoders, zero-shot transfer).
- **Alignment**: explicit (DTW) or implicit (attention, transformers). Underlies grounding, referring-expression segmentation, captioning.
- **Translation**: example-based vs. generative; CycleGAN, DALL-E, DALL-E 2, Stable Diffusion.
- **Fusion**: early (feature level) vs. late (decision level); classical MKL/graphical models vs. neural; neural fusion wins on performance but loses on interpretability.
- **Co-learning**: transfer between modalities, especially when secondary modality is only available at training time. Important for medical, low-resource, noisy settings.
- **Generation** decomposes into *summarisation* (info ↓), *translation* (info =), and *creation* (info ↑); the hardest open problem is *evaluation*.
- **Open issues**: generalising across modality subsets, balanced training (OGR), the performance–robustness–complexity trade-off, modality collapse, fairness and the compounding of social biases across modalities.

Part XIII

Multimodal Foundation Models, LLMs and VLMs

83 Multimodal Foundation Models (MFM)

83.1 Definition

A **Multimodal Foundation Model (MFM)** is a large-scale model capable of processing multiple types of inputs, where each *modality* is a specific data type:

- Text, Images, Audio, Video, ...

A canonical application is **Visual Question Answering (VQA)**: given an input image and a natural-language question about it, the model produces a natural-language answer. Other applications: text-image retrieval, image generation, multimodal chat (e.g. explaining a meme).

83.2 Two Common Approaches

Restricting to vision-language MFMs, there are two main architectural recipes.

A. Unified Embedding Decoder. Inputs from different modalities (image, text) are mapped into a **shared embedding** space; a single decoder (typically a language-style LLM) generates the output (answer, caption, ...) from this unified representation.

- Single decoder model — often an unmodified LLM such as GPT-2.
- Images are converted into tokens (vectors) of the *same embedding size* as text tokens.
- Image and text tokens are concatenated and processed together.
- Examples: LLaVA, MiniGPT-4, Qwen-VL.

B. Cross-Modality Attention. The model uses a **cross-attention** mechanism to integrate information *inside the attention layer*: one modality (text) directly attends to features from another modality (image) during processing.

- Text queries the image embeddings, attending to the visual regions relevant to it.
- Lets the LLM reason about language conditioned on visual content.
- Examples: Llama 3.2-V, Aria, NVLM-X.

83.3 Self-Attention vs. Cross-Attention

Both share the same formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The difference is in *where* Q, K, V *come from.*

- **Self-attention**: Q, K, V all from the same modality $\Rightarrow$ captures relations *within* a modality (text-to-text, image-to-image).
- **Cross-attention**: Q from one modality (e.g. text), K, V from another (e.g. image) $\Rightarrow$ fuses information *across* modalities.

83.4 Vision-Language MFM Building Blocks

Three main components:

1. **Image Encoder**: produces patch embeddings.
2. **Text Tokenizer + Embedding Layer**: produces token embeddings.
3. **GPT-like LLM**: processes the joint sequence of embeddings.

83.4.1 Image Encoder

- Split image into **smaller patches**; flatten and project to *patch embeddings.*
- Pass patch embeddings through a pre-trained **Vision Transformer (ViT)**.
- Output: a set of patch embeddings capturing local + global context.

Projection layer. Patch embeddings cannot be fed directly to the LLM:

- Their dimension may not match the LLM embedding size (e.g. 768 vs. 4096).
- They live in a space the LLM was *not* trained to interpret.

A **linear projection** reshapes them to the LLM embedding dimension and aligns them with the LLM language-based representation.

83.4.2 LLM Input Text Format

LLMs cannot process raw text (categorical, incompatible with NN math). Text must be turned into continuous vectors — an *embedding*. Two sequential steps:

1. **Tokenizer**: splits text into tokens, converts each to an integer ID.
2. **Embedding Layer**: maps each ID to a learned high-dimensional vector.

Tokenizer. Many splitting strategies: Word Tokenization, Byte-Pair Encoding (Sennrich et al., 2015). Word tokenizer splits by spaces, treats punctuation as a separate token, then maps each string to an integer using a **vocabulary** (dictionary built once from the training corpus by tokenising, sorting, and de-duplicating).

Token Embedding Layer. A *lookup* mechanism mapping integer IDs to high-dimensional vectors.

- Implemented as a matrix of shape (`num_embeddings`, `embedding_dim`).
- Given an integer index, return the corresponding row.
- Weights are **learnable**: initialised randomly, updated via backprop like other parameters.

84 Large Language Models (LLMs)

84.1 Definition

A **Large Language Model (LLM)** is a deep-learning model designed to understand, generate and work with human language:

- Performs a wide variety of tasks (chat, summarisation, translation, code generation, ...).
- Based on the **Transformer** architecture.
- Trained on *vast* text corpora.
- Initially designed for text only.

84.2 Architecture Overview

- Stack of multiple **Transformer blocks**.
- Each block uses **Masked Self-Attention (MSA)**: a variant of self-attention that models relationships between tokens *without leaking future information*.

84.3 Autoregressive Modelling

LLMs are **autoregressive**: they feed their previous outputs back as input. Each new token is chosen conditioned on the preceding sequence.

"This" → "This is" → "This is an" → "This is an example"

84.4 Two Training Phases

1. **LLM Pre-training**: produces an initial *base / foundation* model with broad language understanding. After this phase the model can only *complete* text.
2. **Instruction Fine-tuning**: adapts the base model to specific applications (summarisation, translation, QA, ...). The model now *understands* instructions, not just completes.

84.5 Training Task: Next-Token Prediction (NTP)

The pretraining task is **Next-Token Prediction**: predict the upcoming token given the preceding ones.

- Builds a foundational understanding of how words and phrases fit together.
- A form of **self-supervised learning**: the labels (next tokens) are already in the data — no annotation needed.

Training data format. Input-target pairs are built with a **sliding window**: input is the window, target is the input shifted one position to the right. All tokens past the target are masked.

84.6 Masked Self-Attention (MSA)

Why mask? Standard self-attention lets each token attend to all others. For *sentence translation* or *sentiment analysis* that's fine; for **next-token prediction** it leaks future information — during training the model would have access to the very tokens it must predict.

Mask. Use a **lower-triangular binary mask** on the attention matrix: each token attends only to itself and previously-seen tokens.

Implementation. Recall $\text{Softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$. *Zero* cannot be used as a masking value because $e^0 = 1$ still contributes to the denominator. Use $-\infty$: $e^{-\infty} = 0$, and the masked positions are ignored exactly.

84.7 Positional Encoding

Self-attention measures token similarity, not order: it has no built-in notion of position. Without additional positional awareness, the same word in different contexts gets the same vector after tokenisation ⇒ the model loses any sense of sequence.

Absolute Positional Embeddings. A **learnable matrix** of size (`max_sequence_len`, `embedding_dim`) that is **added** (not concatenated) to the token embeddings before the Transformer blocks. `max_sequence_len` (e.g. 1024, 2048) is the maximum input length the LLM can handle.

84.8 Output Layer

- The last Transformer block outputs an embedding for every token, but *only the last token's embedding* is retained for next-token prediction — it has integrated all preceding context via masked attention.
- The Output Layer projects this embedding to a vector of dimension equal to the **vocabulary size** (e.g. 50257 for GPT-2).
- **Softmax** converts logits to a probability distribution over the vocabulary.

- **Argmax** selects the highest-probability ID, which is looked up in the vocabulary to produce the next word.

84.9 Training Objective

LLM training aims to make the model assign the highest probability to the correct next token given the preceding context. The likelihood of the corpus $\mathcal{U}$ is:

$$\mathcal{L}(\mathcal{U}) = \sum_{i=1}^{N} \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}, \Theta)$$

Since standard optimisers *minimise*, the training loss is the **negative log-likelihood**:

$$\mathcal{L}(\mathcal{U}) = -\sum_{i=1}^{N} \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}, \Theta)$$

Minimising the negative log-likelihood is equivalent to maximising the log-likelihood.

Different LLMs, same recipe

Each LLM component carries learnable weights Θ. Different LLMs differ in:
- Number of Transformer blocks.
- Embedding size.
- Sub-components inside each block.

The training objective is essentially identical: next-token prediction with negative log-likelihood.

85 GPT: Generative Pretrained Transformer

85.1 History

GPT is a type of LLM introduced by OpenAI in 2018.

Model	Year	Params
GPT-1 (Radford et al., 2018)	2018	117M
GPT-2 (Radford et al., 2019)	2019	1.5B
GPT-3 (Brown et al., 2020)	2020	175B

- **GPT-1**: introduced the Transformer into a language model; could be fine-tuned for simple tasks (e.g. text classification).
- **GPT-2**: significant leap in model size; trained on *WebText* (8M web pages); much better language understanding.
- **GPT-3**: dramatic scale-up; trained on books + web; creative writing, QA, translation, ...

85.2 Model Scaling

All GPT models up to GPT-3 share the *same fundamental architecture*; the difference is scale:

Model	# Transformer blocks	Embedding size
GPT-1	12	768
GPT-2	48	1600
GPT-3	96	12288

85.3 Emergent Behaviours

Despite the simple next-token objective, GPT models acquire abilities they were never explicitly trained for as they scale up: **emergent behaviours**.

Example 85.1 (Translation emerges at scale)**.** Input: "*English: The dog is playing in the yard. Italian:*". A sufficiently large GPT outputs "*Il cane sta giocando in giardino*".

Emergent behaviours include arithmetic, code completion, summarisation, common-sense reasoning, translation, jokes, physics QA — a consequence of scale + diversity of training data.

85.4 Model Architecture

A GPT model contains all the standard LLM components (Embedding Layer, Positional Encoding, N Transformer Blocks, Output Layer). The only *additional* elements are:

- **Initial Dropout layer** (after token + positional embedding).
- **Final LayerNorm layer** (before the output projection).

85.5 Transformer Block

Each block has two sub-blocks, each with a residual connection.

Attention sub-block. Blends information from earlier tokens using Masked Self-Attention. A residual connection adds the input back to the output, preserving the original token identity after attention mixing.

Feed-Forward sub-block. Re-encodes each token **separately** (no cross-token mixing) to add depth and complexity. A residual connection preserves the signal from the attention block.

Order inside one block:

$$\mathbf{x} \to \text{LayerNorm1} \to \text{MSA} + \text{Dropout} \to +\mathbf{x} \to \text{LayerNorm2} \to \text{FeedForward} \to +(\text{prev})$$

85.6 Layer Normalization

- Deep networks can suffer from instabilities (**vanishing/exploding gradients**) as activations grow uncontrolled.
- LayerNorm adjusts the activations of a layer to have **zero mean and unit variance**.
- Normalisation is computed across the *embedding dimension*: each token is normalised *independently.*
- Result: tighter, more centred distribution; more balanced gradients; more stable training.

85.7 Dropout in GPT

Dropout (Srivastava et al., 2014) randomly ignores a fraction of units at each training step, preventing the model from over-relying on any specific subset.

Two locations in GPT.

1. **Inside the attention mechanism**, after the softmax: a Dropout mask zeros out random attention weights. Prevents the attention from focusing too heavily on a single position; spreads attention more evenly.

2. **On the input embeddings**: forces the model to learn from a reduced set of features each step, encouraging robustness. Applied independently to every embedding element with a fixed probability.

Dropout is used only during training and disabled at inference.

85.8 Feed-Forward Module

The FF module **expands** the embedding into a higher-dimensional space (typically $\times 4$) via a first linear layer, applies a **GELU** non-linearity, then **contracts back** to the original dimension via a second linear layer.

Why expand-then-contract? Temporary projection into a much higher-dimensional space lets the model capture more expressive transformations, then compress them back.

GELU activation. Combines ReLU and sigmoid ideas: rather than a hard cutoff at zero, weights inputs by their value *and* by their probability of being positive under a Gaussian:

$$\mathrm{GELU}(x) \approx 0.5\,x\left(1+\tanh\left[\sqrt{\frac{2}{\pi}}(x+0.044715\,x^3)\right]\right).$$

85.9 GPT Training Summary

GPT is a stack of Transformer blocks trained *autoregressively* with Next-Token Prediction. Training minimises the negative log-likelihood:

$$\mathcal{L}(\mathcal{U}) = -\sum_{i=1}^{N} \log P(u_i \mid u_{i-k},\ldots,u_{i-1},\Theta).$$

All learnable components (token embeddings, positional encoding, Transformer-block weights, final LayerNorm, output projection) are updated via backpropagation.

86 CLIP: Contrastive Language-Image Pre-training

86.1 Motivation

Standard supervised vision models need large *annotated* datasets (e.g. ImageNet); annotation is expensive. **CLIP** (Radford et al., ICML 2021) sidesteps this: **image-text pairs are readily available on the Internet** — captions, alt-text, Wikipedia ⇒ vast supervision without manual labelling.

86.2 Idea

Train a vision-linguistic model over pairs of images and textual descriptions so that it can solve different tasks **without task-specific training or labelling**. CLIP learns visual concepts from image captions by training to identify which image-caption pairs are *valid*. The matched pairs teach the model what visual features correspond to words (“dolphin”, “night sky”); mismatched pairs prevent incorrect associations.

86.3 Architecture

Two encoders:

- **Image encoder**: ResNet or ViT. Only the *first token* (CLS) is retained to summarise visual information.
- **Text encoder**: decoder-only Transformer. Only the *last token* is retained to summarise the textual information.

86.4 Measure of Similarity

To make image and text outputs directly comparable:

1. **Project** both to a space of equal dimensions via separate Linear layers.
2. Apply L_2 **normalisation** to remove scale: vectors compared by direction, not magnitude.
3. Measure similarity with the **cosine**:

$$\cos\theta = \frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\,\|\mathbf{B}\|}.$$

   For unit vectors this is just their dot product.

86.5 Training Objective: Contrastive Loss

Maximise the cosine similarity of valid image-text pairs; minimise it for all other pairs in the batch. Two symmetric components:

- **Image-to-text loss**: each image embedding compared against *all* text embeddings in the batch; cross-entropy along the text dimension (column axis).
- **Text-to-image loss**: each text embedding compared against all image embeddings; cross-entropy along the image dimension (row axis).

In a batch of N pairs, the similarity matrix $I_i \cdot T_j$ has size $N \times N$; the **positive pairs are on the diagonal**. The ground-truth vector for the cross-entropy is simply $(0, 1, 2, \ldots, N-1)$.

This encourages a joint embedding space where image and text encoders produce *similar* embeddings for related image-caption pairs.

86.6 Zero-Shot Classification

CLIP is trained to predict whether an image and a caption are paired; this capability can be repurposed for **zero-shot classification**: classify images for tasks CLIP was never fine-tuned on.

Procedure. Given a set of candidate classes:

1. Craft text prompts of the form "*a photo of a <class_name>*".
2. Embed each prompt with the CLIP text encoder.
3. Embed the image with the CLIP image encoder.
4. Predict the class whose prompt has the **highest cosine similarity** to the image embedding.

87 LLaVA: Large Language and Vision Assistant

87.1 Overview

LLaVA (Liu et al., NeurIPS 2023) is a **Vision-Language Model** that works with both image and text modalities. It follows the **Unified Embedding-Decoder** paradigm and is built *on top of a pre-trained LLM* (LLaMA, Vicuna) with a vision encoder (CLIP ViT).

Use cases. Image captioning, VQA, multimodal multi-turn conversation (multimodal chatbot).

87.2 Architecture

Three key components:

1. **Vision Encoder** (frozen CLIP ViT): extracts visual features Z_v from the image X_v.
2. **Multimodal Projector** W: maps visual features to the language model's token space, producing H_v. Implemented as a Linear layer (LLaVA 1.0) or an MLP (LLaVA 1.5+).
3. **Language Model** f_ϕ: processes H_v together with the tokenised instruction H_q, generating the language response X_a.

The projector **re-encodes the visual embeddings** into a format the LLM can interpret as if they were text tokens, bridging the gap between image and text.

87.3 Autoregressive Modelling

LLaVA follows the autoregressive paradigm: each new token is generated conditioned on the visual input + question + previously generated tokens.

87.4 Training Procedure

LLaVA is trained with the standard Next-Token Prediction objective, conditioned on the image:

$$\mathcal{L}_{\text{LLaVA}} = -\sum_{i=1}^{L} \log P(u_i \mid \mathbf{X}_v, \mathbf{X}_{\text{instruct}}, \mathbf{X}_{a,<i}, \Theta)$$

- $\mathbf{X}_v$ — visual input,
- $\mathbf{X}_{\text{instruct}}$ — user's instruction,
- $\mathbf{X}_{a,<i}$ — assistant tokens already generated before position i.

Training is split into **two stages**.

87.4.1 Stage 1: Pre-training for Feature Alignment

Goal: align the vision encoder's output with the LLM's input embedding space. The two were pre-trained *separately* and live in incompatible spaces.

- **Only the projection layer** is trained (Θ for W); vision encoder and LLM stay frozen.
- Dataset: diverse image-text pairs (e.g. CC3M, 3M image-caption pairs) for broad concept coverage.
- Captions converted into **instruction-following data** as *single-turn conversations*: "User: what is depicted in the image? Assistant: <caption>".

87.4.2 Stage 2: Visual Instruction Tuning (Fine-tuning)

Goal: teach the model to act as an *assistant* on complex multimodal reasoning tasks.

- Updates both the LLM and the projection layer; vision encoder stays frozen.
- Training data: **multi-turn conversations** (not just single-turn captioning).
- Only after this stage does LLaVA *behave like a chatbot* — remembering context across turns and answering a series of questions conversationally.

LLaVA training summary

1. **Pre-training** (single-turn, CC3M): align visual features with LLM tokens. Only *projector* trained.
2. **Visual Instruction Tuning** (multi-turn): teach assistant behaviour. *Projector + LLM* trained.

Vision encoder (CLIP ViT) is always **frozen**.

88 Gemini: A Family of Highly Capable Multimodal Models

88.1 Overview

Gemini (Gemini Team, Google, 2023) is a family of multimodal AI models designed from the start to process *multiple* modalities: audio, images, text, video.

Key differentiator vs. GPT. GPT accepts text-only prompts. Gemini supports **interleaved sequences of different modalities as inputs** and can produce **interleaved text + image outputs** — handles a natural conversation-like flow with mixed modalities, both in and out.

88.2 Capabilities

Multimodal reasoning. Given a handwritten student's physics solution + the problem statement, Gemini explains the student's error and produces the correct solution with LaTeX.

Multilingual. Trained on high-quality English plus 40+ other languages. Example: explain a Chinese family-tree chart with the correct Mandarin kinship terms.

88.3 Architecture (Hypothesis)

Gemini's full architecture is not disclosed, but the technical report gives two hints:

- *Gemini models are built on top of Transformer decoders.*
- *Gemini models are multimodal from the beginning.*

A reasonable architecture sketch:

1. A **multimodal embedding model** transforms text / audio / image / video inputs into a unified representation.
2. A **multimodal Transformer** trained *from scratch* (not bolted on top of a pre-trained text LLM) processes the unified sequence.
3. Separate output heads — **image decoder** and **text decoder** — specialise the Transformer outputs for the desired modality.

88.4 Mixture of Experts (MoE)

Gemini 1.5 introduces **Mixture of Experts (MoE)**: the network is divided into smaller "expert" sub-networks, and a **learned routing function** directs each input to one or a few experts.

- Each expert specialises in different input patterns.
- Only a *subset* of the model is active at inference ⇒ lower compute (FLOPs).
- Allows the total parameter count to grow far beyond a dense model with the same per-token compute.

88.4.1 Where in the Transformer?

MoE typically **replaces the Feed-Forward modules** inside Transformer blocks. Instead of a single FFN, a MoE block contains several parallel FFNs; the router picks the appropriate expert(s) for each token. During training the router learns to route in a way that encourages experts to specialise.

88.4.2 The Router

The router decides:

- Which expert(s) handle each token.
- How much weight to give each expert's output (in the top-k setting).

Single-expert (top-1) implementation.

1. A **Linear layer** projects the embedding to a vector of dimension equal to the number of experts.
2. **Softmax** converts logits to a distribution over experts.
3. **Argmax** selects the expert with the highest probability.

89 Summary

Take-Home Messages

- **Multimodal Foundation Models (MFMs)** are large-scale models processing multiple input modalities; canonical task is VQA. Two architectures: *Unified Embedding Decoder* (LLaVA, MiniGPT-4, Qwen-VL) and *Cross-Modality Attention* (Llama 3.2-V, Aria, NVLM-X).
- **Self-attention** captures within-modality relations; **cross-attention** fuses across modalities (Q from one, K, V from another).
- Vision-language MFM blocks: *image encoder* (ViT + patch projection), *tokenizer + embedding layer* for text, *GPT-like LLM* on the joined sequence.
- **LLMs** are stacks of Transformer blocks with **Masked Self-Attention**, trained *autoregressively* with Next-Token Prediction (a self-supervised task) and minimising the negative log-likelihood. Positional encoding (learnable, additive) injects order; output layer projects to vocabulary size and uses softmax + argmax.
- **GPT-1/2/3** share an architecture and differ only in scale (blocks, embedding size). Scaling produces *emergent behaviours* — translation, arithmetic, reasoning the model was never explicitly trained on. Inside each block: LayerNorm, Masked Self-Attention with Dropout, residual, FeedForward (expand $\times 4$ with GELU, then contract).
- **CLIP** learns a joint image-text space contrastively from web-scale image-caption pairs. Two encoders (image: ResNet/ViT; text: Transformer), L_2-normalised projections, cosine similarity, symmetric image-to-text + text-to-image cross-entropy. Enables **zero-shot classification** via prompt templates ("a photo of a <class>").
- **LLaVA** stacks a frozen CLIP-ViT vision encoder, a trainable multimodal projector (Linear or MLP), and a pre-trained LLM (Vicuna / LLaMA). Two stages: *pre-training* (projector only, single-turn) for feature alignment, then *visual instruction tuning* (projector + LLM, multi-turn) to act as a chatbot.
- **Gemini** is multimodal from the ground up: handles interleaved sequences of audio/image/text/video as inputs and produces interleaved text + image outputs. Built on Transformer decoders trained from scratch on multimodal data. Gemini 1.5 uses **Mixture of Experts**: MoE blocks replace FFNs; a learned router (Linear $\rightarrow$ Softmax $\rightarrow$ Argmax) sends each token to one or a few experts, enabling much larger total parameter counts at fixed per-token compute.